E1803D All-In-One Scanner Controller

XY2-100, XY2-200,
XY2-100E, XY2-200E,
XY3-100

SL2-100, NX-02, SDP optionally

Users Manual

© 2016-2026 by HALaser Systems GmbH

Table of Contents

1 Copyright....4
2 History....8
3 Safety....10
4 EU Cyber Resilience Act (CRA) Information....11
4.1 General....11
4.2 Firmware Bill of Material....11
4.3 Software Security Information....11
5 Overview....12
5.1 Features....12
6 Position Within The System....13
7 Board And Connectors....14
7.1 Ethernet....14
7.1.1 Ethernet Configuration With Windows 10....15
7.1.2 Ethernet Configuration With Windows 11....16
7.1.3 Ethernet Configuration With Linux....16
7.2 USB....17
7.3 Power....17
7.4 User LEDs....19
7.5 microSD-Card....20
7.5.1 Firmware Update....31
7.6 Scanner Signals....32
7.7 Laser Signals....34
7.8 Digital Interface....36
7.8.1 Marking On-The-Fly Signals....37
7.8.2 Opto-Configuration Jumpers....39
7.8.3 Output State LEDs....39
7.8.4 Input State LEDs....39
7.9 Serial Interface....40
7.10 Extension Connectors....40
8 Stand-Alone Operation....41
8.1 Create Stand-Alone Data with BeamConstruct....41
8.2 Stand-Alone Configuration Parameters....42
8.3 Stand-Alone Control....44
9 Matrix Laser Dot Marking Mode....46
9.1 Dot Mode Configuration Parameters....46
9.2 Dot Mode Hardware Interface....47
9.3 Dot Mode Control....48
10 Multi-IO Extension Board....49
10.1 Board Connectors....49
10.2 Multi-IO Interface....49
11 Intelli-IO Extension Board v1....51
11.1 Board Connectors....51
11.2 Intelli-IO Interface in IO mode....51
11.3 Intelli-IO Interface in motion mode....52
12 Intelli-IO Extension Board v2....54
12.1 Board Connectors....54
12.2 State LED Signals....54
12.3 Intelli-IO Interface in IO mode....55
12.4 Intelli-IO Interface in motion mode....56
13 NX-02 Extension Board....57
13.1 Board Connectors....57
13.2 NX-02 Interface....57
13.3 Analogue Input Interface....58
14 SDP Extension Board....59
14.1 Board Connectors....59
15 E1803dock Extension Board....60
15.1 E1803dock MOPA....60

15.2 E1803dock YLM....64
15.3 E1803dock SPI....66
15.3.1 Connector to laser....68
15.4 E1803dock CO_2/YAG....69
15.5 E1803dock NCC....72
16 E1803base Mounting Kit....74
17 Quick Start into E1803D....75
18 Command Interface....76
18.1 General Commands....76
18.2 Stand-Alone Control Commands....78
18.3 Hardware Commands....85
18.4 Mark Control Commands....86
19 Supported CNC G-Code Commands....95
19.1 General G-Code Characters....95
19.2 Supported “G”-codes....96
19.3 Supported “M”-codes....97
19.4 Supported “T”-codes....99
19.5 Control Protocol....99
20 Programming Interfaces....101
20.1 E1803D Easy Interface Functions....101
20.1.1 General functions....102
20.1.2 Laser and scanner related functions....108
20.1.3 Digital interface functions....122
20.1.4 Serial interface functions....125
20.1.5 Intelli-IO extension functions (IO-mode)....127
20.1.6 Intelli-IO extension functions (motion mode)....127
20.1.7 Miscellaneous functions....132
20.1.8 Writing of stand-alone data....133
20.1.8.1 Example....136
20.1.9 Error Codes....137
20.2 RTC4 Compatibility Functions....137
20.3 USC1/2 Compatibility Functions (SCI interface)....141
APPENDIX A – Wiring between E1803D and specific laser types....143
Wiring between E1803D and IPG YLP Series Type B, B1 and B2, GZTech YFPN series fiber laser....143
Wiring between E1803D and JPT YDFLP series fiber laser (“MOPA”) or IPG YLP Series Type D fiber laser or Raycus RFL PMX/PQB Series fiber laser....144
Wiring between E1803D and IPG YLP Series Type E fiber laser....145
Wiring between E1803D and IPG YLP Series Type G fiber laser....146
Wiring between E1803D and IPG YLR Series laser....147
Wiring between E1803D and IPG YLM Series laser....148
Wiring between E1803D and MaxPhotonics MFP fiber laser....149
Wiring between E1803D and SPI G4 Pulsed Fibre Laser / TRUMPF TruPulse nano series....150
Wiring between E1803D and Raycus fiber laser....151
Wiring between E1803D and Raycus C500 Series laser....152
Wiring between E1803 and DAVI D-Series RF CO_2 Laser....153
Wiring between E1803 and Raycus RFL-C Series 24V Control CW Laser....154
Wiring between E1803D and GZTECH S355 series DPSS Laser....155
APPENDIX B – XY2-100 / XY2-200 protocol description....156
APPENDIX C – XY3-100 protocol description....157
APPENDIX D – SL2-100 protocol description....158
APPENDIX E – RL3-100 protocol description....159
APPENDIX F – IDC connector pin numbering....160
APPENDIX G – Mechanical Dimensions....161

1 Copyright

This document is © by HALaser Systems GmbH.

E1803D boards, their hardware and design are copyright / trademark / legal trademark of HALaser Systems GmbH.

IPG and others are copyright / trademark / legal trademark of IPG Laser GmbH / IPG Photonics Corporation.

Scanlab, RTC4, RTC5, RTC6, SL2-100 and others are copyright / trademark / legal trademark of Scanlab AG.

SCAPS, USC1, USC2, USC3 and others are copyright / trademark / legal trademark of SCAPS GmbH.

Raylase, SP-ICE, RL3-100 and others are copyright / trademark / legal trademark of Raylase AG.

Rofin, Rofin-Sinar, Visual Laser Marker and others are copyright / trademark / legal trademark of Raylase AG.

Sunny, CSC-USB and others are copyright / trademark / legal trademark of Beijing Century Sunny Technology CO., LTD

CTI, Cambridge Technology, Novanta and others are copyright / trademark / legal trademark of Novanta Inc.

Han's, Han's Laser and others are copyright / trademark / legal trademark of Han's Laser Technology Industry Group Co., Ltd.

Newson, SDP, Serial Data Protocol and others are copyright / trademark / legal trademark of Newson NV

All other names / trademarks are copyright / trademark / legal trademark of their respective owners.

Portions of the E1803D firmware are based on lwIP 1.4.0 (or newer):

Copyright (c) 2001, 2002 Swedish Institute of Computer Science.
All rights reserved.
Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. The name of the author may not be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR ``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT
OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY
OF SUCH DAMAGE.

Portions of the E1803D firmware are based on FatFS R0.10a (or newer):

FatFs module is an open source software to implement FAT file system to small embedded systems. This is a free software and is opened for education, research and commercial developments under license policy of following terms.

Copyright (C) 2014, ChaN, all right reserved.

- The FatFs module is a free software and there is NO WARRANTY.
- No restriction on use. You can use, modify and redistribute it for personal, non-profit or commercial product UNDER YOUR RESPONSIBILITY.
- Redistributions of source code must retain the above copyright notice.

Portions of the E1803D firmware are based on StarterWare 2.0 (or newer):

Copyright (C) 2010 Texas Instruments Incorporated – http://www.ti.com/

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of Texas Instruments Incorporated nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright (c) 2008-2010 Texas Instruments Incorporated. All rights reserved.

Software License Agreement

Texas Instruments (TI) is supplying this software for use solely and exclusively on TI's microcontroller products. The software is owned by TI and/or its suppliers, and is protected under applicable copyright laws. You may not combine this software with "viral" open-source software in order to form a larger program.

THIS SOFTWARE IS PROVIDED "AS IS" AND WITH ALL FAULTS. NO WARRANTIES, WHETHER EXPRESS, IMPLIED OR STATUTORY, INCLUDING, BUT NOT LIMITED TO, IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE APPLY TO THIS SOFTWARE. TI SHALL NOT, UNDER ANY CIRCUMSTANCES, BE LIABLE FOR SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, FOR ANY REASON WHATSOEVER.

This is part of AM1808 Sitaraware USB Library and reused from revision 6288 of the Stellaris USB Library.

Portions of the E1803D firmware are based on libzint-backend 2.0 (or newer):

libzint - the open source barcode library, Copyright (C) 2008-2017 Robin Stuart <rstuart114@gmail.com>

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. Neither the name of the project nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

The Motion Extension v1 firmware bases on motion5 version 1.1 or newer*:

Copyright (c) 2018 Oxygenic, (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC, Copyright (c) 2009-2011 Simen Svale Skogsrud

motion5 is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.
motion5 is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

*) GPL notice: the motion extension firmware is running separately and completely independent from the main controller firmware, they are neither linked nor compiled together with each other. The related motion5 sources, as used on motion extension, can be found at https://sourceforge.net/p/axis5/code/ci/master/tree/

The Motion Extension v2 firmware bases on grblHAL STM32H7xx:

COPYRIGHT NOTICE FOR grblHAL STM32F3xx driver:

grblHAL - Embedded CNC g-code interpreter and motion-controller

Copyright (c) 2019-2021 Terje Io

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <http://www.gnu.org/licenses/>.

COPYRIGHT NOTICE(S) FOR WORK CONTAINED IN THIS SOFTWARE:

GNU General Public License:
Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
Copyright (c) 2009-2011 Simen Svale Skogsrud

COPYRIGHT(c) 2017 STMicroelectronics

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:
1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. Neither the name of STMicroelectronics nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

STM32 USB driver files
MCD Application Team

Copyright (c) 2015 STMicroelectronics. All rights reserved. </h2>

This software component is licensed by ST under Ultimate Liberty license SLA0044, the "License"; You may not use this file except in compliance with the License. You may obtain a copy of the License at: ww.st.com/SLA0044

Copyright (c) 2009-2018 Arm Limited. All rights reserved.

SPDX-License-Identifier: Apache-2.0

Licensed under the Apache License, Version 2.0 (the License); you may not use this file except in compliance with the License. You may obtain a copy of the License at www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an AS IS BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

FatFs - Generic FAT Filesystem Module R0.13c

Copyright (C) 2018, ChaN, all right reserved.

FatFs module is an open source software. Redistribution and use of FatFs in source and binary forms, with or without modification, are permitted provided that the following condition is met:

Redistributions of source code must retain the above copyright notice, this condition and the following disclaimer.

This software is provided by the copyright holder and contributors "AS IS" and any warranties related to this software are DISCLAIMED. The copyright owner or contributors be NOT LIABLE for any damages caused by use of this software.

2 History

Date	Changes in document
10/2026	Description of commands "cmapz" and "cstal" added
10/2026	Description of configuration parameter mapz added
09/2026	Description of configuration parameters "clipwa" and "cliprot" added
09/2026	Intelli-IO v2 description added
08/2026	Description of configuration parameters "cjsor", "cmsor", "cpwor", "cfror", "cpuor" added
02/2026	Description of functions E180X_jump_abs_list() and E180X_mark_abs_list() added
12/2025	Description of cxcnc-mode control format added
09/2025	Wiring/configuration description for GZTECH DPSS UV laser added
06/2025	Wiring description for Raycus RFL-C Series 24V controlled CW lasers added
06/2025	Configuration parameter cxcnc added
06/2025	Command cxcnc added
06/2025	Added support for G2/G3 G-codes
05/2025	Description of configuration parameters errout and motout added
11/2024	Added PID control loop API function description
10/2024	Specification of SDP Extension Board added
08/2024	Description of pilot laser pin added
07/2024	Description of E1803dock NCC added
06/2024	Description of limitations of dynamic texts clarified
06/2024	Description of parameter "iothres" clarified
03/2024	Added parameter and command cswaf
03/2024	Description of function E180X_uart_write() updated
02/2024	Description of A/E-LED updated
10/2023	Configuration parameter u0stop added
10/2023	Voltage levels of RS232 interface specified more exact
10/2023	Windows 11 Ethernet configuration description added
07/2023	Reference to header files and programming examples added
03/2023	Behaviour of A/E LED clarified
03/2023	Numbering of pins of extension boards clarified
12/2022	Description of E180X_stop_execution() and cvers extended
12/2022	Missing description of flags E180X_PIXELMODE_ added
11/2022	Added new function E180X_get_pos()
11/2022	Default values of G-Code parameters described
10/2022	Added new stand-alone commands cspof, cfror and cpuor
10/2022	Electrical behaviour of digital IOs clarified
10/2022	New confoguration parameter busid added
10/2022	Code for d-command "Set Position Offset" corrected
06/2022	New function E180X_motion_move_abs_async()
06/2022	New commands cslp8, cslgt and cslmo added
05/2022	Description of E1803dock MOPA version 1.2 added
02/2022	Tune-flag added to invert input logic of ExtStart input
02/2022	Tune-flag now also can read hex-values with 0x prefix
02/2022	Added tune-flags to invert LP8 and MO outputs
12/2021	New configuration parameter lasergate
11/2021	Configuration parameter u0bypass added
11/2021	Tune-flag 524288 added
10/2021	Description of E180X_set_xy_correction() corrected
10/2021	E1803dock pinout description clarified
08/2021	New parameters tunereadyout and tunemarkout added
07/2021	Inappropriate language and naming removed ("master", "slave", ...)
07/2021	Description of HALdrive mounting possibilities added
07/2021	Added description of ctlxy command
07/2021	Wiring of JPT/MOPA laser with pulse width serial interface clarified

07/2021	Description of CSV-support in stand-alone mode added
06/2021	Added wiring scheme when no dual power supply is used
04/2021	New “d”-commands 0x18 / delay and 0x19 / Latch added
04/2021	Description of new configuration parameter “eth=2” for Ethernet interface polling added
04/2021	Description of Ethernet configuration updated for Windows 10
03/2021	Added function `E180X_get_serial_number()`
01/2021	Added description of new function `E180X_motion_set_pulsewidth()`
12/2020	Added commands “`cgmtx`” and “`csmtx`”
12/2020	Added new stand-alone function and configuration parameters “`iolatch`” and “`iothres`”
11/2020	Description of jumper configuration of digital interface extended
10/2020	Pinout of IPG YLM and SPI G4 corrected
08/2020	Description of parameter „tunexy3“ added
06/2020	XY3-100 description and pinout added
04/2020	Added description of `E180X_set_scanner_mode()` and introduced 200 kHz output modes
01/2020	Added description of commands „`csbuf`“ and „`cgbuf`“
01/2020	Added new stand-alone mode „`idxselect`“
01/2020	Added “`adcfreq`” configuration parameter
11/2019	Usage of digital inputs of Intelli-IO extension clarified
10/2019	Added wiring scheme for MaxPhotonics fiber lasers
10/2019	Example in description of `pethd`-parameter corrected
07/2019	Extended description of card state flags
07/2019	Added description about how to write stand-alone data via API
05/2019	Numbering of DIn and DOut of second digital port (Intelli-IO extension) corrected
02/2019	Added “`pethd`” configuration parameter
01/2019	Added PID control loop API function description
12/2018	Added “`haltedloopbuffer`” configuration parameter
11/2018	Stepper motor pinout added for Intelli-IO extension
10/2018	Added commands „`cscor`“ and „`cgcor`“
10/2018	Added “`haltedlooptimeout`” configuration parameter
09/2018	New tune-flag added
09/2018	Added description of Intelli-IO Extension Board and related API functions
05/2018	Added description of Motion Extension Board and motion API functions
04/2018	Description of new parameters “`digiinit`” and “`digimask`” added
04/2018	Description of new tune-flags added
04/2018	Added description of command `E180X_digi_pulse()`
03/2018	Added description of Multi-IO Extension Board
02/2018	New “d”-command 0x45 to update firmware
02/2018	Added description of flag `E180X_COMMAND_FLAG_ASYNC`
01/2018	Added description for `u0bits` and `u0parity` configuration parameters
12/2017	Added description for matrix-”d”-commands 0x40 and 0x41
10/2017	Added wiring description for IPG YLM lasers
08/2017	Added description for config parameters `wetout` and `mipout`
08/2017	Description for `E180X_set_sync()`/`E180X_get_sync()` added
07/2017	Description of USB license retrieval clarified
07/2017	TrueType support in stand-alone mode
04/2017	Added description of command `cscnc`
04/2017	Added wiring scheme for IPG YLR lasers
03/2017	Added description of supported G-Code commands
02/2017	Images updated
01/2017	Added wiring scheme for IPG type E lasers with APD index mode
01/2017	Added “`iohaltedloop`” stand-alone mode
12/2016	Description of `corrtable0` parameter corrected
12/2016	Initial version

3 Safety

The hardware described within this document is designed to control a laser scanner system. Laser radiation may effect a person's health or may otherwise cause damage. Prior to installation and operation compliance with all relevant safety regulations including additional hardware-controlled safety measures has to be secured. The client shall solely be responsible to strictly comply with all applicable and relevant safety regulations regarding installation and operation of the system at any time.

Beside of that some laser equipment can be damaged in case it is controlled with wrong signals or signals outside a given specification. Thus it is highly recommended to check the output generated by this hardware using e.g. an oscilloscope to avoid problems caused by wrong configurations. This should be done prior to putting a system into operation for the first time, whenever some parameters have been changed or whenever any kind of software update was installed.

The hardware described here is shipped without any cover and without prefabricated equipment for electric installation. It is intended to be integrated in machines or other equipment. It is not a device for use "as is", but a component which is intended to be used as part of a larger device, e.g. for integration in a machine with own housing or within an electrical cabinet. Prior to operation compliance with all relevant electric / electromagnetic safety regulations including additional hardware-controlled safety measures has to be secured. The client shall solely be responsible to strictly comply with all applicable and relevant regulations regarding installation and operation of the system at any time.

The hardware described here is an electrostatic sensitive device. This means it can be damaged by common static charges which build up on people, tools and other non-conductors or semiconductors. To avoid such a damage, it has to be handled with care and including all relevant procedures (like proper grounding of people handling the hardware, shielding/covering to not to let a person touch the hardware unwanted, proper packaging in ESD-bags, ...). For more information please refer to related regulations and standards regarding handling of ESD devices. The EMC Directive (2014/30/EU) does not apply to this hardware as it is not intended for an end user (a person without knowledge of EMC) and as it is not otherwise made available on the market.

The Low Voltage Directive (2014/35/EU) does not apply to this hardware as the voltage supply is below the 50V AC / 75V DC limit.

This control board is considered partly completed machinery in accordance with the EU Machinery Directive (2006/42/EC). It cannot operate independently and is intended to be integrated into a larger machine or system. The final integrator is responsible for ensuring that the complete machine or system complies with all applicable safety and regulatory requirements in the intended market (such as CE- certification).

This document describes the E1803D-hardware but may contain errors and/or may be changed without further notice.

4 EU Cyber Resilience Act (CRA) Information

4.1 General

The firmware and hardware described in this manual is intended for operation only in accordance with the conditions and limitations specified in this manual. In particular, it shall be operated exclusively within a closed and isolated network environment without direct or indirect connection to the Internet.
The user is responsible for ensuring the integrity and authenticity of any software update prior to installation. Before installing an update, the user shall verify the provided checksums and/or cryptographic fingerprints against the values published by the manufacturer in the applicable product documentation.
Software updates shall only be installed if the integrity and authenticity of the update have been successfully verified.

4.2 Firmware Bill of Material

The software described in this manual makes use of the following external components and 3rd party software products:

- Texas Instruments Starter Ware 02.00.01.01
- lwIP 1.4.0
- FatFS R0.10a
- libzint 2.4.3
- monocypher 4.0.3

4.3 Software Security Information

Before installing an update, the user shall verify the digital signature and/or cryptographic fingerprint of the update against the corresponding information published by the manufacturer in the applicable product documentation.

All software components come with an integrated or separate software signature which allows it to cross-check them against a finger print in order to verify software integrity. Here two fingerprints are used:

Signature fingerprint for the Windows software package and device drivers:
`BD5D2A78F7F198C1A89385ED9E869966D5ADE1B5`

Signature fingerprint for the Linux software package and controller firmware:
`CBE3C92581559B68480EFF5E764715C27BEBA260`

To verify the integrity of the fingerprints given here, please cross-check them against earlier documentations and information that contain these fingerprints too and that are issued prior to any potential incident (such as other manuals or invoices or email signatures in mails from HALaser Systems).

5 Overview

This document describes the E1803D compact scanner controller board, its electrical characteristics and usage.

The E1803D scanner controller board is designed for controlling galvanometric scanner systems with two or three axes. It also supplies extensive signals for laser and external control. The communication between the host system and the controller boards is done via Ethernet or USB.

This is an all-in-one controller which provides all interfaces that are required to control common lasers and additional hardware. For a modular controller which can be configured according to some special requirements please refer to https://halaser.systems/E1803.php.

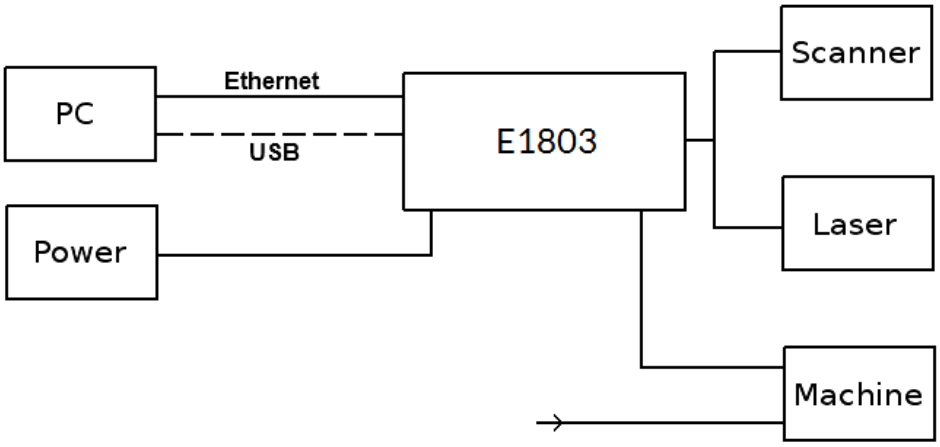
5.1 Features

The E1803D controller can be used to control 2D or 3D scanheads that come with a XY2-100 interface. It offers the following features:

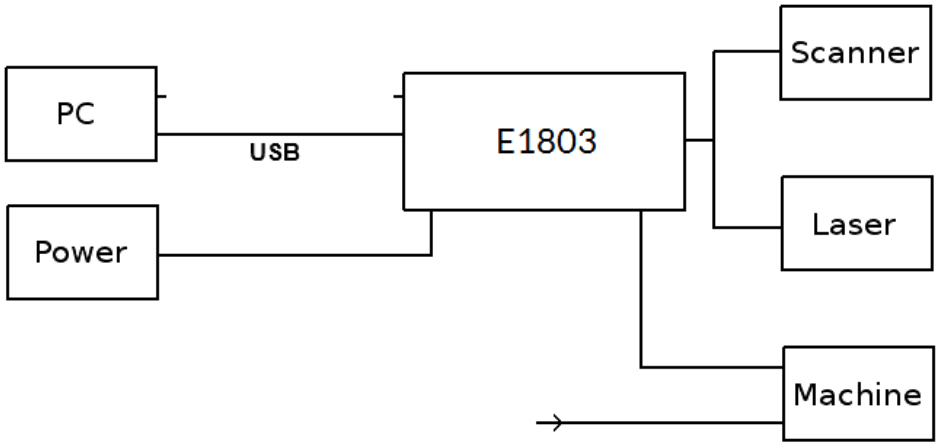
- XY2-100 and XY2-100-E interface to scanhead with X, Y and optional Z channel
- 100 Mbit Ethernet connection
- USB 2.0 connection
- online XYZ grid correction with support for several correction table file formats (like SCAPS™ .ucf, Scanlab™ .ctb and .ct5, Raylase™ .gcd, Rofin™ .fcr, Han's™ .crt, CTI™ .xml, Sunny™ .txt)
- high-definition online XYZ grid correction with BeamConstruct HD correction files (.bco)
- switching between up to 16 preloaded grid correction tables
- 10 microseconds vector cycle time and resolution (microstep period)
- command execution time down to 0,5 microseconds
- realtime processing of laser and scanner signals
- 26 bit internal resolution (for better accuracy also with 16 bit or 18 bit hardware output)
- 512 MByte DDR3 RAM
- 1 GHz CPU clock
- support for microSD and microSDHC cards
- optional matrix laser dot marking mode with up to 13 dots, up to two independent lines of text and up to 2 MHz dot frequency
- internal command and vector data list with more than 20 million entries
- continuous list concept, no need to swap between buffers
- BeamConstruct PRO license included
- open source compatibility library that emulates existing programming interface for fast and easy usage with existing software (contains e.g. Scanlab™ RTC4™, SCAPS™ USC™/SCI and other compatible interfaces)
- LP8 8 bit CMOS level parallel digital output e.g. for controlling laser power or laser waveform type
- LP8 latch CMOS level digital output for usage with IPG™ and compatible laser types
- Main Oscillator CMOS level digital output for usage with IPG™ and compatible laser types
- 12 bit 0..10V analogue output e.g. for controlling laser power
- two laser CMOS level digital outputs for usage with YAG, CO_2, IPG™, SPI™ and compatible laser types (outputs can provide PWM frequency, Q-Switch, FPK-pulse, CW/continuously running frequency, stand-by frequency) running with frequencies of up to 20 MHz
- 8 freely usable digital outputs providing either CMOS level or electrically insulated outputs via external power supply
- 8 freely usable digital inputs expecting either CMOS level or electrically insulated inputs via external power supply
- 4 digital inputs usable for quadrature encoder signals for 1D and 2D marking on-the-fly applications
- RS232/RS485 serial interface for communication with external devices
- scanhead power supply via controller card to save additional wiring

6 Position Within The System

The E1803D scanner controller system can be connected to the host via Ethernet or USB to receive laser marking data from BeamConstruct laser marking application or from any other application which makes use of one of the provided programming possibilities (as described below). When using Ethernet connection, it optionally can be connected via USB too. In this case USB connection is used to retrieve BeamConstruct PRO license from the board:

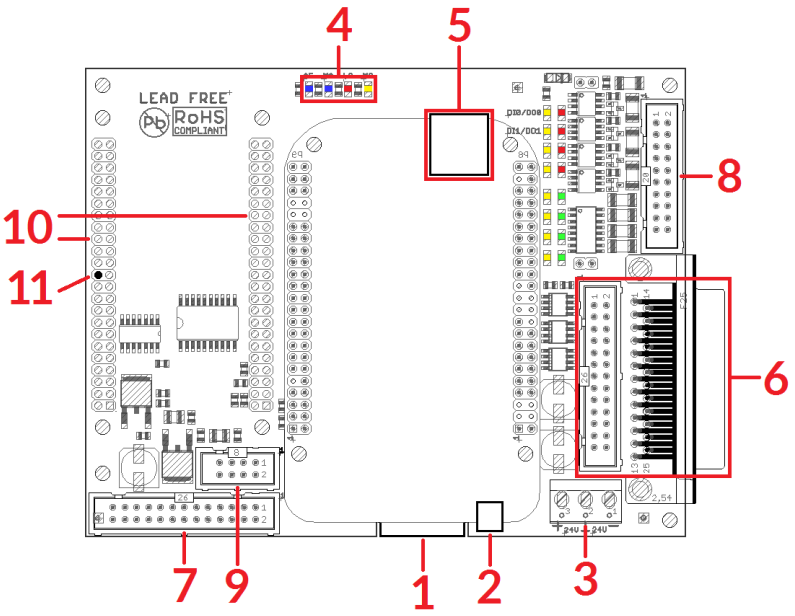


Since 100 Mbit Ethernet provides much faster data transfer than USB 2.0, this connection type is preferred. Especially in case complex marking data with many short lines that result in many separate jump and mark commands are used, Ethernet connection is more responsive.
When using USB connection with such data, time from sending data to the card until marking operation can be started may be longer (up to several seconds in worst case) caused by slower USB data transfer:



In both cases the board itself is connected with the scan head to submit 2D or 3D position information to it. Beside of that it is connected to a laser to submit motion-synchronous laser data. Additional communication channels between the E1803D scanner controller board and a connected machine can be done via separate IOs of the digital interface.

7 Board And Connectors



The E1803D Digital Laser Scanner Controller Card provides following connectors and interfaces:

1. Ethernet – for communication with the host system, marking information are submitted via this path
2. USB – via microUSB connector for providing BeamConstruct PRO license to host system and optionally for submitting marking data from host to E1803D card (in case Ethernet is not used)
3. Power – connect with power supply
4. User LEDs – show operational and error states of card
5. microSD-card – storage place for firmware and extended configuration file, can be used to upgrade firmware, to change the card's IP and other things more
6. Scanner signals – 26 pin and D-SUB25 scanner output connectors which provides XY2-100 scanner signals and power to scanhead
7. Laser signals – connector with different signal for controlling a laser and for starting/stopping mark operation
8. Digital interface – in- and output connector for control of external devices and for connecting marking on-the-fly encoder(s)
9. Serial interface connector for connections to RS232 or RS485 devices
10. Extension connectors
11. Code-pin for correct placement of extension boards

7.1 Ethernet

This is a standard RJ45 Ethernet plug for connection of the board with the host system. When the controller board is accessed via this connection, all scanner and laser data are sent via Ethernet. Thus it is recommended for security reasons to have a separate machine network that contains the control-PC, the scanner controller card(s) and other Ethernet-devices for the machine, but has no physical connection to the “outer world”, means no access to the internet.
Ethernet connection is initialised during start-up only, thus Ethernet cable connecting E1803D board and host system needs to be plugged before the board is powered up.
By default the E1803D board is using IP 192.168.2.254, thus the Ethernet network the card is connected with needs to belong to subnet 192.168.2.0/24.
PLEASE NOTE: For security reasons it is highly recommended to not to mix a standard communication network with an E1803D network or to connect the scanner controller card with a standard network. Here it may be possible someone else in that network (accidentally) connects to that scanner controller and causes laser emission.
The IP of the scanner controller can be changed. This is necessary e.g. in case an other subnet has to be used or in case the E1803D board has to be operated in multi-head environments where more than one card will be

accessed at the same time. The IP can be configured using e1803.cfg configuration file that is placed on microSD-card. To change the IP, please perform the following steps:

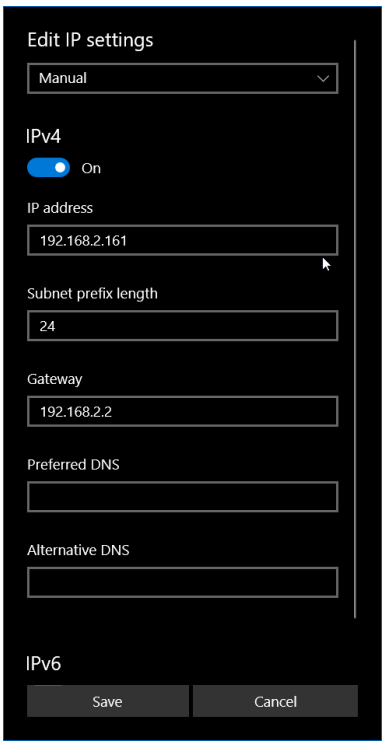
1. disconnect E1803D board from power and USB
2. remove microSD-card
3. put microSD-card into a desktop computer, this may require a microSD- to SD-card-adapter
4. open the drive that is assigned to the card
5. open file e1803.cfg using a text editor like Notepad or kwrite
6. add a line or edit an existing line "`ip0=`", here the desired IP has to be appended (as example: when you want to configure IP 192.168.2.13 the line has to be "`ip0=192.168.2.13`" – without any quotation signs
7. save the file
8. eject the drive the card is assigned to
9. place the microSD-card in E1803D board (place without the use of force, notice correct orientation with connectors of SD-card to top!)
10. power up card

When User LEDs do not light up as described below, please check if microSD-card is placed in board correctly.

7.1.1 Ethernet Configuration With Windows 10

When E1803D scanner controller is accessed via Ethernet, it is recommended to use a separate network for security reasons. Since the controller is working with a static IP (default is 192.168.2.254) the Ethernet port on host PC has to be configured with an IP of same subnet in order to allow access to it. For Windows 10 (and similar) this configuration has to be done using following steps:

1. right-click the network-symbol in your taskbar
2. Select “Open network and internet settings”
3. Select “Ethernet” on the left
4. find the network interface E1803D has to be connected with and select it
5. Click the “Edit” button in section “IP settings”
6. now a window opens where “IPv4” has to be turned on and that has to be configured as follows:

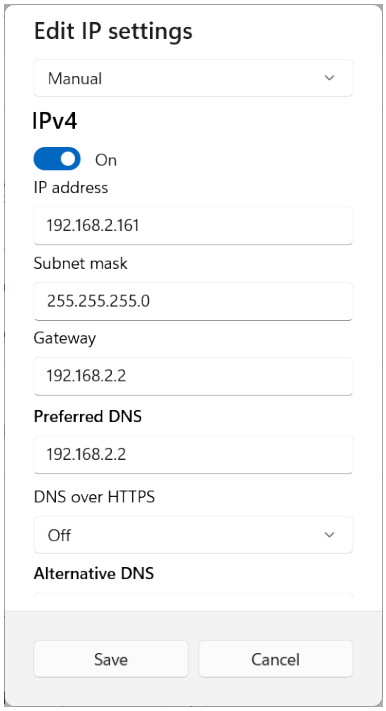


There you can specify an IP for your host PC. It has to belong to network 192.168.2.xxx and can be any number except than 192.168.2.254 (this is already the IP of the scanner card), 192.168.2.0 or 192.168.2.255.

7.1.2 Ethernet Configuration With Windows 11

When E1701 scanner controller is accessed via Ethernet, it is recommended to have a separate network for security reasons. Since the controller is working with a static IP (default is 192.168.2.254) the Ethernet port on host PC has to be configured with an IP of same subnet in order to allow access to it. For Windows 11 (and similar) this configuration has to be done using following steps:

1. right-click the network-symbol in your taskbar
2. Select “Network and internet settings”
3. Select “Ethernet” in the opened list
4. find the network interface E1701D has to be connected with and select it
5. Click the “Edit” button right beside “IP assignment”
6. now a window opens where “Edit IP Settings” has to be switched from “Automatic (DHCP)” to “Manual”
7. next “IPv4” has to be turned on and the remaining parameters in this window have to be configured as follows:



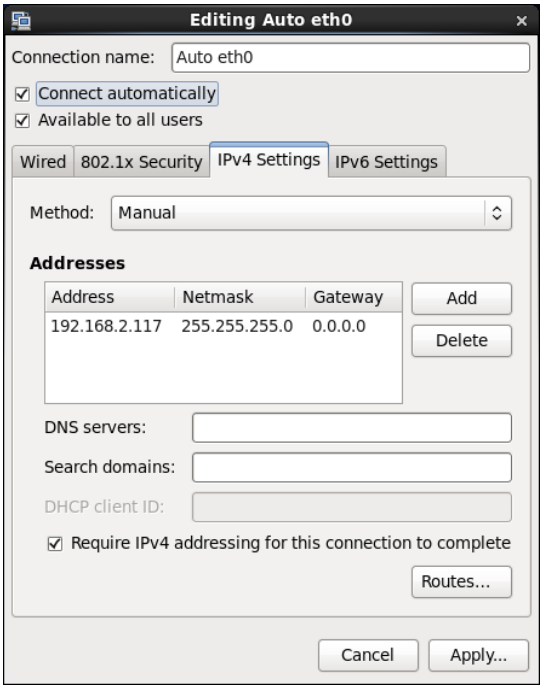
There you can specify an IP for your host PC. It has to belong to network 192.168.2.xxx and can be any number except than 192.168.2.254 (this is already the IP of the scanner card), 192.168.2.0 or 192.168.2.255.

7.1.3 Ethernet Configuration With Linux

When E1803D scanner controller is accessed via Ethernet, it is recommended to use a separate network for security reasons. Since the controller is working with a static IP (default is 192.168.2.254) the Ethernet port on host PC has to be configured with an IP of same subnet in order to allow access to it. For Linux (with NetworkManager) this configuration has to be done using following steps:

1. right-click the network-symbol in taskbar
2. click "Edit Connections..."
3. select the "Wired" network interface the scanner card is connected with and press button "Edit"

4. go to tab-pane "IPv4 Settings" and configure it as shown below:



There you can specify an IP for your host PC. It has to belong to network 192.168.2.xxx and can be any number except than 192.168.2.254 (this is already the IP of the scanner card), 192.168.2.0 or 192.168.2.255.

7.2 USB

This is a standard microUSB-connector for connection of the board with the host system. It is used to retrieve BeamConstruct PRO license and optionally to send marking data to the card. When USB is used for sending all scanner and laser data, Ethernet cable does not need to be connected.

PLEASE NOTE: USB 2.0 is much slower than a standard 100 Mbit Ethernet connection, so expect slower execution in case of complex marking data!

The required device driver is installed automatically during the installation of the HALsetup software package (Windows) or comes with operating system by default (Linux). E1803D card appears as COM-interface on Windows using any free number for the port. With Linux it appears as /dev/ttyACMx where "x" is any number. These numbers are provided by the operating system automatically.

When no external power supply is connected, USB provides 5V power supply too. So whenever the card has to be stopped, both USB and power have to be disconnected in order to shut it down completely. It is not recommended to use USB as power supply, additional, external power should be connected in order to operate E1803D controller correctly. When E1803D is powered via USB only, not all functions are available. Here things like power supply of connected scanhead and 0..10V analogue output signals AOut0 and AOut1 will not work.

Depending on the capabilities of the used USB host, there also may be other failures and limitations caused by power-brownouts and drop-outs.

When the controller is connected via USB, a BeamConstruct PRO license is provided via this interface automatically. This is done without the need to configure anything, and as long as following conditions are true:

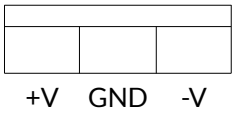
- physical USB connection from controller to host PC exists
- the COM-port (Windows) has a number smaller than COM20
- the controller is working and the Alive-LED in blinking

It is also possible to have the USB-connection for license retrieval only and to use the Ethernet-connection to transfer marking data to the controller, both can exist beside each other.

7.3 Power

Power supply for E1803D scanner controller board is done via 3 pin screw connector. Here a power in range +12..+24V or +-12..+-24V can be connected. This connector powers the board and optionally can be used to

power the scanhead too as this power is routed to the DB25-connector directly and from there can supply the scanhead directly via a standard 1:1 DB25 cable (for details please refer to description of XY2-100 / XY3-100 connector below). No matter if the power is routed via the DB25 connector to the scanhead or not, it is recommended to use the same power supply as for the scanhead here.

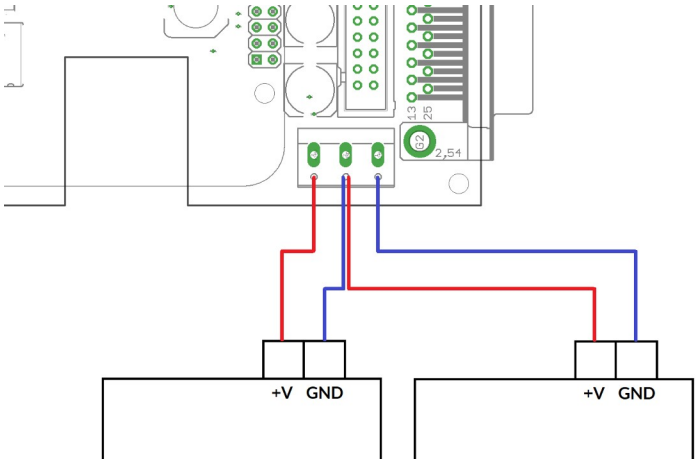


Following possibilities to connect power exist:

+V	GND	-V	Supported Features / Remarks
12..24 V	connected	unused	Power E1803D and analogue outputs AOut0 and AOut1
12..24V	connected	12..24V	Power E1803D, analogue outputs AOut0 and AOut1 and scanhead via XY2-100 / XY3-100 connector, input voltage has to be the same voltage that is required to operate the scanhead
12V	connected	unused	Mandatory power supply in case the SDP Extension Board is used (as described in section "14 SDP Extension Board" below)
12..24V	unused	12..24V	Not allowed!
unused	connected	12..24V	Not allowed!
unused	unused	unused	Power supply needs to be done via USB, then only E1803D is powered; scanhead needs a separate power supply connection and the wires 9..13 and 22..25 of the D-SUB25 connector need to be interrupted between scanhead and controller!
other	other	other	Not allowed!

When all three inputs are connected to a bipolar power supply providing +-12..+-24V to power both, the controller and a connected scanhead, the input voltage is feed to the scanhead directly via XY2-100 connector. Here the voltage has to be equal to the voltage required by the scanhead (typically +-15V or +-24V). Power supply needs to provide 1A plus current required for connected scanhead. For more details please refer to section about XY2-100 connector below.

When no power supply with symmetric/bipolar outputs is available, it is possible to combine two standard power supplies, here GND of the first power supply has to be connected with +V of the second power supply and with the GND-screw-connector of E1803D:

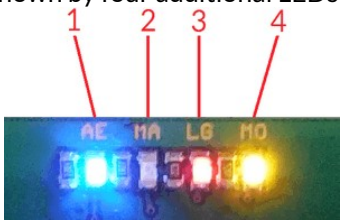


ATTENTION: When connecting wires to the screw terminals of the power connector, do not transmit any force to the PCB where the green connector is soldered at! While screwing tight the wires, hold the connector by hand to catch the force but **do not hold the PCB only!**

ATTENTION: due to the undefined behaviour of some power supplies with high peaks in some specific situations, the power to the controller never should be toggled just by pulling and reconnecting a cable which is on power (hot-swap). Always turn off the power the regular way via the power supplies input/a regular switch. Otherwise this can cause serious damage to the controller card or power supply.

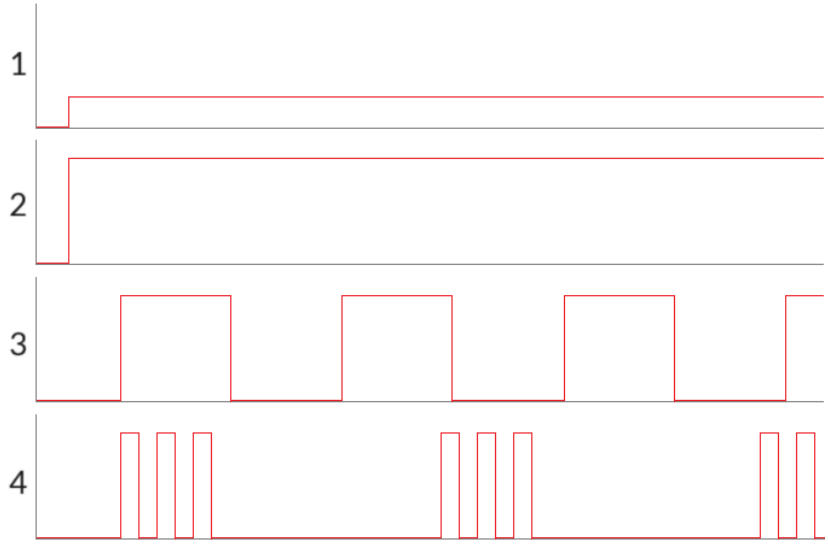
7.4 User LEDs

The real operational state of the card is shown by four additional LEDs described here:



1. A/E (Alive/Error) – blue – this LED is turned on permanently (with full brightness) as soon as the card was powered up and the firmware boots properly. When it is not turned on with full brightness after some seconds, please check if the microSD-card is placed properly and if it contains a working firmware file (for details please refer below).
   After boot process has completed successfully, it starts blinking slowly and with same on and off times. This is an alive-notification, as long as it blinks, the board is working and ready for operation. During operations the blink frequency may vary.
   When this LED starts blinking with a changed on-time (LED is off for a long time and flashes two or three times for a short time only), a fatal error has occurred that normally should never happen. When this happens, in some cases the board can't continue with operation until the reason for error is removed and the board is restarted.
   In case this LED flashes signalling an error-state, please:
   - check if you are using valid E1803D extension boards only (and no other 3rd party hardware)
   - check if you are using latest firmware and host software
   - check all connections and cables
   - undo your latest changes in hardware and configuration
   If these steps do not help, please contact us for further assistance.

   So this LED can signalise four operational states:



   1 – Powered on but firmware not yet loaded – turned on permanently but with low brightness
   2 – Not ready / booting – turned on permanently with full brightness
   3 – Operational – blinking, on and off times are the same
   4 – Error – flashing three times, LED is off most of the time and is turned on for a very short time only (with some previous firmware version the error is signalled by flashing two times)

   Please note: during start-up and when the configuration parameter “eth=2” is set in e1803.cfg, the

blinking frequency of operational state 2 can be much lower. This is the case as long as the controller tries to detect an Ethernet connection. It ends and switches to faster blink frequency as soon as this detection is timed-out or as soon as a connection via Ethernet or USB is established.

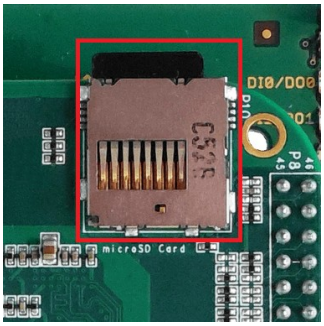
2. MA (Marking Active) – blue – this LED is turned on as long as a marking operation is running. This LED does not correspond to the laser gate signal, comparing to it it's also enabled during jumps when laser is turned off but marking operation itself is active.

3. LG (LaserGate) – red – this LED shows modulation state of the laser and signal the state of the laser gate output. It is turned on as long as the laser is turned on and the laser gate is high. This LED does NOT signal the same like the marking active LED described above since it will be turned off during jumps.

4. MO (Main Oscillator) – yellow – this LED is specific to the Main Oscillator output signal described below. As long as the signal is on (HIGH-signal at output pin), the LED is turned on.

7.5 microSD-Card

The microSD-card is storage place for firmware and configuration files. Here SD and SDHC cards with a capacity of up to 32 GB are supported. It is plugged with the contacts of the SD-card oriented to upper side.



To remove the microSD-card, first disconnect all power from the E1803D board completely (including USB). Next press microSD-card gently into the board until you can hear a click-noise. Then you can pull it out of the board. To place a microSD card, the same has to be done in reverse order: place it into the E1803D board's card slot and press it gently until a noise signals locking of the card. Now the board can be powered.
E1803D board is shipped with a card containing firmware and configuration files:

- e1803.fwi – firmware file that is used to operate the board, to be replaced when a firmware update is provided;
- e1803.cfg – configuration text file, can be edited using a text editor in order to modify cards configuration
- e1803.dat – additional data file that is used to operate the board, to be replaced when a firmware update is provided

Starting with firmware version 23, following additional files can be found on the SD-card which belong to the security mechanisms as per the EU CRA (please refer to "4 EU Cyber Resilience Act (CRA) Information"):

- signatures.bin – a specific signature file that is used internally to verify the integrity of the firmware; this file is locked to e1803.fwi and e1302.dat, means none of these files may be exchanged separately. Doing so prevents the firmware from proper booting
- sha256 contains the SHA256 checksums of all relevant files on the SD-card, they can be used to verify if the files are correct or if they have been modified by some 3rd party; checking of the integrity of these files can be done e.g. by calling "`sha256sum -c sha256`" in Linux command line
- sha256.asc contains the signature for the file sha256 and ensures the checksums mentioned there have not been changed by some 3rd party; checking the integrity of the checksums can be done e.g. by calling "`gpg --verify sha256.asc sha256`" in Linux command line, this call should return the same signature fingerprint as listed in section "4.3 Software Security Information" above, otherwise the whole firmware is to be assumed as manipulated and potentially insecure

To use an other microSD card than the one shipped with the board, following conditions have to be met:

- maximum total size of 32 GB (SD or SDHC card)

- FAT32 formatted
- using only one partition
- BOOT-flag is set
- e1803.fwi, e1803.dat and signatures.bin files available on card (e1803.cfg is optional)

An additional file e1803.cfg can be placed on the card too. It contains plain ASCII text, acts as configuration file and can contain several parameters and its values which are separated by an equal-sign. Every of the possible parameter/value pairs has to be located in an own line. Following configuration parameters are possible within this file:

Parameter	Description	Example
adcfreq	Specifies the ADC (analogue digital converter) sampling frequency. With this value the update rate of the analogue input (requires an extension board) can be modified. The higher this value is, the more fast changes of the analogue input value can be detected. On too big values the analogue sampling may affect the timing accuracy of the output, thus this value should be kept as low as possible. The default value is 5000, the possible range is from 1000 to 3000000. This parameter requires a firmware version 8 or newer.	`adcfreq=10000` set a sampling frequency of 10000 Hz
corrtable0	Specifies a correction table file in .bco, .ctb, .ct5, .ucf, .gcd, .xml .crt or .txt format to be loaded on start-up. When this parameter is set, the specified correction table is used exclusively and all correction data possibly sent from the host are ignored. The correction file itself has to be located on microSD-card too. When the Error-LED is turned on after a correction table file was configured, E1803D board was not able to load it for some reason.	`corrtable0=0:/D2_200.ctb` use file D2_200.ctb as correction file and ignore all correction tables possibly sent from host application
corrtable<idx>	Specifies one of up to 16 correction table file in .bco, .ctb, .ct5, .ucf, .gcd, .xml .crt or .txt format to be loaded on start-up. When this parameter is set, the specified correction table is used exclusively and all correction data possibly sent from the host are ignored. The correction file itself has to be located on microSD-card too. This method has also to be used when running the controller in stand-alone mode with .EPR files that require such a correction. When the Error-LED is turned on after a correction table file was configured, E1803 baseboard was not able to load it for some reason. `<idx>` can be any value in range 0..15 and specifies the storage location index of the correction file to be loaded. Later the related correction file can be used via command `cscor`. When `<idx>` has to be set to values greater than 0, a firmware version 6 or newer is needed.	`corrtable7=0:/200_200.bco` use file 200_200.bco as correction file at index position 7 and ignore all correction tables possibly sent from host application
ip0	Configures IP of Ethernet port. Here only IPs in xxx.xxx.xxx.xxx notation are allowed but no host or domain names.	`ip0=192.168.2.100` specifies IP 192.168.2.100 to be used for Ethernet interface on next startup

Parameter	Description	Example
passwd	Specifies an access password that is checked when card is controlled via Ethernet connection. This password corresponds to password specified with function `E180X_set_password()`, please refer below for a detailed description. When a client computer connects to the card without sending the correct password, Ethernet connection to this host is closed immediately. PLEASE NOTE: this password does not replace any network security mechanisms and does not give the possibility to operate E1803D controller via insecure networks or Internet! It is transferred unencrypted and therefore can be "hacked" easily. Intention of this password is to avoid collisions between several E1803D cards that operate in same network and are accessed by several software instances. Maximum allowed length of the password is 48 characters. It is recommended to not to use any language-specific characters.	`passwd=myCardPwd` set a password "myCardPwd"
standalone	This command can be used to disable or enable a specific stand-alone operation mode. For a detailed description of possible parameters, operation modes and usage please refer related section below.	
iolatch	When using one of the digital-input-controlled stand-alone modes, this option can be used to latch the digital states in via DIn7. For details please refer to section “8 Stand-Alone Operation” below	`iolatch=1` enable the latch-function via DIn7
iothres	In stand-alone mode there are two conditions that cause a loaded EPR file to be ready: it is fully loaded into the secondary, marking buffer or a minimum amount of data is available in secondary buffer. Something similar is true for host-controlled operating mode: here marking starts when `E180X_execute()` is called or when a minimum amount of data has been transmitted to the controller. That minimum amount of marking data can be modified with the parameter “iothres”. The smaller this value is, the faster marking will start, but in this case it also may happen there are not enough data available so that interruptions occur during marking. So a balance between speed and a secure, non-interrupted marking process need to be found when this value is modified. By default “iothres” is 80000 which should fit to most applications, the maximum allowed value is 280000 and it should not become smaller than 10000	`Iothres=120000` Set the threshold for availability of the stand-alone marking data to 120000
haltedlooptimeout	This parameter is used in stand-alone modes “haltedloop” and “iohaltedloop” (please refer to section “8 Stand-Alone Operation” for detailed information). It defines a timeout for the laser in unit seconds. If the current operation is active for a longer time, the laser is turned off. It then can be turned on only by toggling the enable-input (ExtStart) again. This parameter requires firmware version 6 or newer.	`haltedlooptimeout=5` sets the laser timeout to 5 seconds

Parameter	Description	Example
haltedloop buffer	This parameter is used in stand-alone modes “haltedloop” and “iohaltedloop” (please refer to section “8 Stand-Alone Operation” for detailed information). It defines a maximum buffer size for the marking data. The buffer size should have a size of 20000000 at max. The minimum size depends on the specific application, in fact, when it is set to some too small values, drop-outs in marking operation may occur. Data which are already buffered in this marking mode can't be modified any longer. So any change on marking speed, laser power or similar (done e.g. by commands “`cjsor`”, “`cmsor`” or “`cpwor`”) will apply only to data which are not yet buffered. And as bigger as this buffer is, as longer it takes until the first new data after change of any of these parameters can be emitted. This parameter requires firmware version 5 or newer.	`haltedloopbuffer=100000` set the buffer to a maximum size of 100000 commands which is similar to data for about 1 second marking time

autofile	Loads a special .EPR stand-alone file or .CNC G-Code file from SD-card in some specific stand-alone modes. For a detailed description of possible parameters, operation modes and usage please refer related section below. For a description of supported G-Code commands, please check out related section “19 Supported CNC G-Code Commands“	`autofile=0:/markdata.epr`
		loads a file markdata.epr from disk; here 0:/ specifies the SD-card to be used. The .EPR-file itself can be generated within BeamConstruct out of a normal .BEAMP project file
		
		`autofile=0:/markdata.cnc`
		same as above but a G-Code file is provided which contains marking information

iobuff	Pre-loads one or more .EPR files to the RAM of the controller to allow faster switching in “ioselect” or “idxselect” stand-alone mode. This command can not be used to load file “0.EPR”	`iobuff=1`
		`iobuff=3`
		pre-load files 1.EPR and 3.EPR on board start-up

mipout	Configure a Digi I/O output pin to be used as “mark in progress”-signal by default; here an output bit number in range 0..7 has to be configured which will be set to HIGH as long as a marking operation is in progress, the value given here can be overwritten by API-function `E180X_digi_set_mip_output();` this parameter requires firmware version 3 or newer	`mipout=1` use DOut1 for mark-in-progress signal
wetout	Configure a Digi I/O output pin to be used as “wait for external trigger”-signal by default; here an output bit number in range 0..7 has to be configured which will be set to HIGH as long as a marking operation is in progress and the controller is waiting for an external trigger signal to arrive at ExtStart input, the value given here can be overwritten by API-function `E180X_digi_set_wet_output();` this parameter requires firmware version 3 or newer	`wetout=0` use DOut0 for mark-in-progress signal
errout	Configure a Digi I/O output pin to be used as “error”-signal by default; here an output bit number in range 0..7 has to be configured which will be set to HIGH as soon as a fatal error occurs that can’t be recovered by the scanner card and that sets the A/E-LED into a state where it blinks three times fast; once this output is set, it can be reset only by rebooting the controller card; this parameter requires firmware version 20 or newer	`errout=2` use DOut2 for the error output signal

Parameter	Description	Example
motout	Configure a Digi I/O output pin to be used as “motion in progress”-signal by default; here an output bit number in range 0..7 has to be configured which will be set to HIGH as long as a motion axis of the optional Intelli-IO extension (see “11 Intelli-IO Extension Board v1”) board is moving; this parameter requires firmware version 20 or newer	`motout=3` use DOut3 for the motion active output signal
digiinit	Initialises the digital outputs on firmware start-up with the given defaults. This overrides the hardware defaults. The default digital values set here are NOT available on power up but a few seconds later after firmware has been loaded and started. This function requires firmware version 5 or newer.	`digiinit=2` set DOut1 to HIGH initially and all other outputs to LOW
digimask	Masks the digital inputs and specifies which inputs can be read. All input bits which are ignored by this command by setting the related value to 0, are no longer read. This may be useful for applications where encoder inputs are used together with a “ioselect” stand-alone operation and where the random state of the encoder has to be masked out. This function requires firmware version 5 or newer.	`digimask=253` use only DIn2..DIn7 as input and ignore DIn0 and DIn1
digidebc	Sets a debouncing time / filter time for the digital inputs of the digital interface in order to not to let the inputs react on noise or bouncing of mechanical inputs. The debouncing value is given in time-units where every time-unit is equal to 31 usec. By default 7 time-units are set.	`digidebc=10` set the debounce-time to 310 usec
lasergate	By default, the laser on/off information is provided via the LaserGate output and with CMOS logic voltage level. With this parameter, a digital output of the digital interface can be specified to provide the laser gate signal in parallel. Please refer to “7.8 Digital Interface” for further details about the digital interface. This configuration parameter requires firmware version 14 or newer	`lasergate=3` Use DOut3 to provide the laser gate signal
u0brate	Set the bitrate of UART0 RS485/RS232 serial interface on E1803D. By default this port is initialised with a speed of 115200 bps, this value can be changed with this parameter. Setting an u0brate of 0 disables the serial port completely	`u0brate=9600` set a new bitrate of 9600 bps for UART0 E1803D on-board serial port
u0bits	Set the number of data bits of UART0 RS485/RS232 serial interface on E1803D. By default this port is initialised with 8 data bits, this value can be changed to a word length of 5, 6, or 7 bits with this parameter. This parameter requires firmware version 4 or newer.	`u0bits=7` set a new word length of 7 bits for UART0 E1803D on-board serial port
u0parity	Set the parity of UART0 RS485/RS232 serial interface on E1803D. By default this port is initialised no parity (=0). For odd parity a value of 1 has to be set, for even parity a value of 2 has to be used. This parameter requires firmware version 4 or newer.	`u0parity=2` enable even parity for UART0 E1803D on-board serial port
u0stop	Configures the number of stop-bits used within a single UART0 serial frame: • when this parameter is not set or when it is set to 1, each frame ends with one stop bit • when this parameter is set to 2 and when u0bits configures 5 data bits, each frame ends with 1.5 stop bits • when this parameter is set to 2 and when u0bits configures, 6, 7 or 8 data bits, each frame ends with 2 stop bits This parameter requires a firmware version 17 or newer.	`u0stop=1` each frame ends with 1 stop bit

Parameter	Description	Example
u0bypass	Bypasses the on-board serial port to a specific function, here following values are possible: 0 – no bypassing, the serial port can be used out of the contro ldata stream as usual 1 – use as control interface, when set to this value, the on-board serial port 0 accepts the same commands and data as the Telnet Ethernet interface and the USB serial port This parameter requires firmware version 14 or newer.	`u0bypass=1` allow control communication with E1803D via serial port
u1brate	Set the bitrate of UART1 RS485/RS232 serial interface on E1803D Multi-IO Extension Board (deprecated). By default this port is disabled and has to be activated by setting a bitrate. This parameter exists in firmware versions 4 to 13 and requires a Multi-IO Extension Board.	`u1brate=115200` set a new bitrate of 115200 bps for UART1 serial port
u1bits	Set the number of data bits of UART1 RS485/RS232 serial interface on E1803D Multi-IO Extension Board (deprecated). By default this port is initialised with 8 data bits, this value can be changed to a word length of 5, 6, or 7 bits with this parameter. This parameter exists in firmware versions 4 to 13 and requires a Multi-IO Extension Board.	`u1bits=7` set a new word length of 7 bits for UART1 serial port
u1parity	Set the parity of UART1 RS485/RS232 serial interface on E1803D Multi-IO Extension Board (deprecated). By default this port is initialised no parity (=0). For odd parity a value of 1 has to be set, for even parity a value of 2 has to be used. This parameter exists in firmware versions 4 to 13 and requires a Multi-IO Extension Board.	`u1parity=1` enable odd parity for E1803D on-board serial port
busid	This command requires the parameter u0bypass set to 1: with the busid it is possible to connect several E1803D controller cards via RS285 serial interface in bus topology. The identifier in range 0..99 given with this command then can be used as identifier prior to every c- or d-command in order to let the correct controller card react on a command. With the busid set, echoing (command "`cecho`") is turned off automatically for the serial interface. This parameter requires firmware version 16 or higher.	`busid=9` Specifies the ID 9 for the controller card. With this ID set, all d- and c-commands have to be preceded by a string "b09" to let the controller react on them
tunereadyout	In stand-alone modes, the ready-state of a loaded stand-alone project is signalled via DOut0 by default (please refer to section "8.3 Stand-Alone Control" for further details). Using this parameter, the used output can be changed. Here following values can be given: • 0 – DOut0 (default) • 1 – LaserA (has to be configured as GPO via the related tune-flag) • 2 – LaserB (has to be configured as GPO via the related tune-flag) This parameter requires firmware version 13 or newer.	`tunereadyout=1` use LaserA to signal state "ready" in stand-alone mode
tunemarkout	In stand-alone modes, the ready-state of a loaded stand-alone project is signalled via DOut1 by default (please refer to section "8.3 Stand-Alone Control" for further details). Using this parameter, the used output can be changed. Here following values can be given: • 0 – DOut1 (default) • 1 – LaserA (has to be configured as GPO via the related tune-flag) • 2 – LaserB (has to be configured as GPO via the related tune-flag) This parameter requires firmware version 13 or newer.	`tunemarkout=2` use LaserB to signal state "ready" in stand-alone mode

Parameter	Description	Example

tune	Enables special functions and features that are not activated by default. As parameter a number can be handed over that specifies the functions to be enabled. Starting with firmware version 14 the number can also be specified as hexadecimal value when it is prefixed with “0x”. Several of these functions can be combined by adding their related numbers:
	
	1 (0x01) – use DIn7 of digital interface connector as external trigger, this disables ExtStart input on laser signal connector
	
	2 (0x02) – use additional marking encoder inputs on DIn2 and DIn3 for 2D marking on-the-fly operations
	
	4 (0x04) – enable storage of serial number count values to microSD card; this option is useful in case of stand-alone operation mode when dynamic data with serial number counting is used. When it is set, the current count value of all used serial numbers is stored and reloaded on next power up. Thus their values are not get lost when power was turned off. The values are stored in a file with the same name like the "autofile" or the currently loaded .epr file but with extension ".ser".
	 
	ATTENTION: The file is saved on the FatFS formatted microSD card. FatFS is NOT fault-proof, means it can be corrupted when power is turned off during writing. So when this option is enabled, user has to ensure power is NOT turned of while the card writes to disk. Writing of serial number states is always done in case they have changed, then it is started when Alive/Error LED of E1803D board is switched off. Write operation is finished when this LED is turned back on the next time. So to ensure data are written successfully, it is recommended to let this LED blink two times after last mark operation has been finished or to wait for about 4 seconds.
	 
	ATTENTION: due to this limitation it is not recommended to work with this option but to save the state of the serial numbers by sending ASCII command "`cssta`" instead (please refer below for details)!
	
	8 (0x08) – invert LaserGate output to work as active HIGH signal; when this option is set, logic of LaserGate-LED changes too, it is on as long as laser is turned off and it is off as long as laser is on
	
	16 (0x10) – invert LaserA output to work as active HIGH signal
	
	32 (0x20) – invert LaserB output to work as active HIGH signal
	
	64 (0x40) – use LaserA output as GPO (general purpose output pin); when this flag is set, LaserA output is no longer able to emit a frequency but can be used as digital output pin; when this value is set, a tune-value of 16 (invert LaserA) is ignored. This flag has to be set e.g. when LaserA has to be used together with `tunereadyout` or `tunemarkout` parameter.
	
	128 (0x80) – use LaserB output as GPO (general purpose
		`tune=1`
		disables ExtStart input and switches over external trigger function to DIn7 input
		
		`tune=0x1000`
		operate the scanner output in XY2-100E mode

Parameter	Description	Example
	output pin); when this flag is set, LaserB output is no longer able to emit a FPK pulse but can be used as digital output pin; when this value is set, a tune-value of 32 (invert LaserB) is ignored.. This flag has to be set e.g. when LaserB has to be used together with `tunereadyout` or `tunemarkout` parameter. 4096 (0x1000) – operate in enhanced XY2-100 18 bit mode; when this value is added to the tune-parameter, the controller outputs more accurate 18 bit position data instead of the standard 16 bit values in normal operation mode; this mode needs to be supported by the connected scanhead, elsewhere the results are unpredictable. 8192 (0x2000) – operate in XY3-100 mode with extended resolution; when this value is added to the tune-parameter, the controller outputs more accurate position data instead of the standard 16 or 18 bit values in normal operation mode; this mode needs to be supported by the connected scanhead, elsewhere the results are unpredictable. 32768 (0x8000) – invert the mark-in-progress signal (requires firmware version 5 or newer) 65536 (0x10000) – invert the wait-external-trigger signal (requires firmware version 5 or newer) 524288 (0x80000) – inverts the logic of the ExtStop input; by default, the stop-input is LOW and has to be set to HIGH in order to stop a running operation. When this flag is set, this is inverted, ExtStop has to kept HIGH for normal operation and a stop is performed as soon as it goes to LOW. This flag requires firmware version 14 or newer. 4194304 (0x400000) – invert the LP8 signal of laser interface (requires firmware version 14 or newer) 8388608 (0x800000) – invert the MO (main oscillator) signal of laser interface (requires firmware version 14 or newer) 16777216 (0x1000000) – inverts the logic of the ExtStart input. By default, the start-input reacts on a rising edge. When this flag is set, this is inverted and a falling edge is expected to release an external trigger. This also has an effect on the behaviour of tune-flag 0x2000000, it is inverted too. This flag requires firmware version 14 or newer.	

Parameter	Description	Example
tunexy3	Enables special functions and features related to the XY3-100 scanner interface. The parameters set with this function are set only when the XY3-100 data protocol is used. They are submitted after enabling the XY3-100 mode and prior to first transmission of position data to the connected scanhead. When the connected scanhead does not support the related functions, they will not have any effect This parameter requires a firmware version 38 or newer. As parameter a number can be handed over that specifies the functions to be used. Several of these functions can be combined by OR-concatenating their related numbers: 1 – turn auto-calibration function on (can't be combined with flag value 2) 2 – turn auto-calibration function off (can't be combined with flag value 1) 4 – turn dynamic temperature compensation on (can't be combined with flag value 8) 8 – turn dynamic temperature compensation off (can't be combined with flag value 4)	`tunexy3=5` Turns on the auto calibration and dynamic temperature compensation of a scanhead
sntp0 	Allows to specify the IP of an SNTP time server. This option can be used in case of Ethernet usage to synchronise controller with an external time source. E1803D tries to connect to this server after initialisation of Ethernet interface and – if not successful – a few more times. These additional connection attempts are done whenever the Alive/Error-LED is switched on. ATTENTION: when this function has to be used, the network or host-computer the controller is connected with needs to be able to route this request. This is a potentially dangerous operation because a connection between encapsulated machine network and open and dangerous Internet has to be established. Since this is NOT RECOMMENDED in general, this option should be used ONLY when it is 100% sure there is no possibility for people from outside to intrude the machine network! Instead of that is is recommended to set system time manually using host-computer and ASCII command "cstime" (please refer below). Alternatively it is also possible to contact an own, network-internal NTP-server. When this option is used, the gateway and netmask have to be configured for the controllers Ethernet interface	`sntp0=83.170.1.42` – IP of time server at 3.de.pool.ntp.org is used for SNTP time retrieval (not recommended since this requires a connection to potentially dangerous Internet!)
sntp0offset	This value corresponds to sntp0 parameter above, it is used when system time is retrieved from an external time server to set an offset to the time returned from this server. The offset has to be specified in unit seconds.	`sntp0offset=-3600` – specifies an offset of minus one hour to the time returned from time-server. So when the time server would return a current time of 11:42:17, the system time of the controller would be set to 10:42:17 with this value
gw0	Specifies a gateway-address for the scanner controllers Ethernet interface. This option belongs to parameter "ip0" and has to be set in case "sntp0" is used.	`gw0=192.168.2.1` – use 192.168.2.1 as gateway

Parameter	Description	Example
nm0	Specifies the netmask for the scanner controllers Ethernet interface. This option belongs to parameter "ip0" and has to be set in case "sntp0" is used.	`nm0=255.255.255.0` – use upper 24 bits of current IP for netmask
node0	Specifies the IP of an external HALnode Compact Ethernet IO module to be used for additional 8 digital in- and outputs (lower 8 bits of an additional 16-bit IO port). Once configured, the HALnode's digital ports are used as they were a part of the controller. This functionality requires firmware version 4 or newer.	`node0=192.168.1.253` – use the digital IOs of the HALnode at the IP 192.168.2.253
node1	Specifies the IP of an external HALnode Compact Ethernet IO module to be used for additional 8 digital in- and outputs (upper 8 bits of an additional 16-bit IO port). Once configured, the HALnode's digital ports are used as they were a part of the controller. This functionality requires firmware version 4 or newer.	`node1=192.168.1.252` – use the digital IOs of the HALnode at the IP 192.168.2.252
usb	When this parameter is set to 0, USB interface is disabled completely. This means it is no longer possible to connect to E1803D USB serial interface via terminal software or via BeamConstruct and it is also no longer possible to retrieve BeamConstruct PRO license via USB. This option can be used to suppress illegal access to USB and saves some power.	`usb=0` – turn off USB interface
eth	This parameter specifies the behaviour of the Ethernet interface. Here following values can be set: • 0 – Ethernet network interface is disabled completely. This means it is no longer possible to connect to E1803D via Telnet or via BeamConstruct. All SNTP-functionalities are disabled too. This option can be used to suppress illegal access to Ethernet, to save several seconds of startup-time and some power. • 1 – this mode enables the Ethernet interface and checks once at the beginning if some Ethernet hardware is connected to the controller card; when the "eth"-parameter is not specified at all, the resulting behaviour is the same • 2 – this enables Ethernet polling mode; instead of checking for an Ethernet device only once during boot, in this mode the interface is polled regularly until an electrical connection is detected. As long as the controller is polling, the A/E-LED blinks very slow and toggles once in about 20 seconds, when an Ethernet device was detected, the blink frequency changes to normal speed; PLEASE NOTE: when this mode is used, access via USB is limited, so "eth" should be set to "2" only when no communication via USB is intended. The "eth"-value of 2 requires a firmware version 12 or newer	`eth=0` – turn off Ethernet interface completely
pethd	When Ethernet connection is used, it has to be established on power-up of the controller card as this connection is set-up and configured by the controller only once during boot. There may be situations where the other side of the Ethernet connection can not boot up as fast as E1803. In such cases this parameter can be used. It delays initialisation of Ethernet by the time given as parameter. The time is specified in unit "delayticks" where one "delaytick" is equal to about 0,5 seconds. This feature requires a firmware version 7 or newer.	`pethd=20` – halt initialisation of the controller for about 10 seconds prior to initialisation of Ethernet interface

Parameter	Description	Example
cswaf	Sets a factor in unit bits per mm that is used with all commands that specify some distance or position (such as cspof). When this factor is set to a proper value, the given positions/distances can be specified in unit um instead of bits. This parameter requires a firmware version 18 or newer.	`cswaf=671089` – set the conversion factor from bits to mm for a 100x100 mm working area
cxcnc	Configure the controller to accept G-Code data at the Ethernet-interface (via Telnet communication) or at the USB-serial interface instead of regular commands. When this option is set, the controller can be operated with a stream of G-Code data. Different to the command "`cxcnc`" (as described in section "18.2 Stand-Alone Control Commands" below) this one is permanent, means the G-Code operation mode stays active also when the current CNC-program is marked to being complete by the code M2. This parameter requires a firmware version 20 or newer.	`cxcnc=1` enable the G-Code reception mode for the controller card
cjsor	This parameter has the same meaning as the related command (as described in section "18.2 Stand-Alone Control Commands" but sets the initial and default value. So the override-value specified here is active from the very beginning. This parameter requires firmware version 23 or newer.	
cmsor	This parameter has the same meaning as the related command (as described in section "18.2 Stand-Alone Control Commands" but sets the initial and default value. So the override-value specified here is active from the very beginning. This parameter requires firmware version 23 or newer.	
cpwor	This parameter has the same meaning as the related command (as described in section "18.2 Stand-Alone Control Commands" but sets the initial and default value. So the override-value specified here is active from the very beginning. This parameter requires firmware version 23 or newer.	
cfror	This parameter has the same meaning as the related command (as described in section "18.2 Stand-Alone Control Commands" but sets the initial and default value. So the override-value specified here is active from the very beginning. This parameter requires firmware version 23 or newer.	
cpuor	This parameter has the same meaning as the related command (as described in section "18.2 Stand-Alone Control Commands" but sets the initial and default value. So the override-value specified here is active from the very beginning. This parameter requires firmware version 23 or newer.	

Parameter	Description	Example
clipwa	By default the scanner controller uses its full range to emit positions in a way that makes it possible to use the full range of the connected scanhead. In some situations it may be necessary to limit that range and to clip input coordinated to that limited range. Here the clipwa-parameter can be used to define a smaller output rectangle the emitted coordinates have to be located into. Coordinate values outside that rectangle then are clipped to the maximum specified here. This command expects coordinates in unit bits as coordinates x1,y1,x2,y2 that specify the upper left corner (x1,y1) and the lower right corner (x2,y2) of that clipping rectangle. The values itself have to be specified in 26 bit range of the controller. By default the clipping rectangle is set to 0,0,67108864,67108864 which is equal to the full available range. This function requires firmware version 24 or newer.	`clipwa=0,16777216,33554432,50331648` Set a clipping area which covers only halve of the full available working area of the hardware and which is centred in Y direction and aligned to the left side in horizontal direction
cliprot	This parameter is related to the previous configuration option clipwa and allows it to rotate an existing clipping area. It expects a value in unit 1/1000 degrees and specifies the amount the clipping rectangle is rotated with. This function requires firmware version 24 or newer.	`cliprot=45000` Rotate the clipping area by 45 degrees
mapz	Using this parameter the Z-axis hardware output can be used for other coordinates than the default Z. Here following values can be set: x – the X-axis-coordinates are emitted at the Z-channel -x – inverted X-coordinates are emitted at the Z-channel y – the Y-axis-coordinates are emitted at the Z-channel -y – inverted Y-coordinates are emitted at the Z-channel In all cases the Z-values are no longer available at an hardware output. During runtime, the Z-mapping can be changed by using command `cmapz` as described below in section “18.3 Hardware Commands”. This parameter requires firmware version 24 or newer.	`mapz=-y` Use the Z-channel to emit inverted Y-coordinates in parallel to the non-inverted y-coordinates at the regular Y-channel.

7.5.1 Firmware Update

As described above, the firmware is located on microSD-Card and therefore can be updated easily:

1. remove the microSD-Card as described above
2. download a new firmware from https://halaser.systems/download/Firmware/E1803/ (the higher the number in the file name, the newer the firmware is)
3. copy the contents of this ZIP-file to microSD-Card (please take care about e1803.cfg in case it contains a changed configuration)
4. reinsert microSD-Card as described in previous section

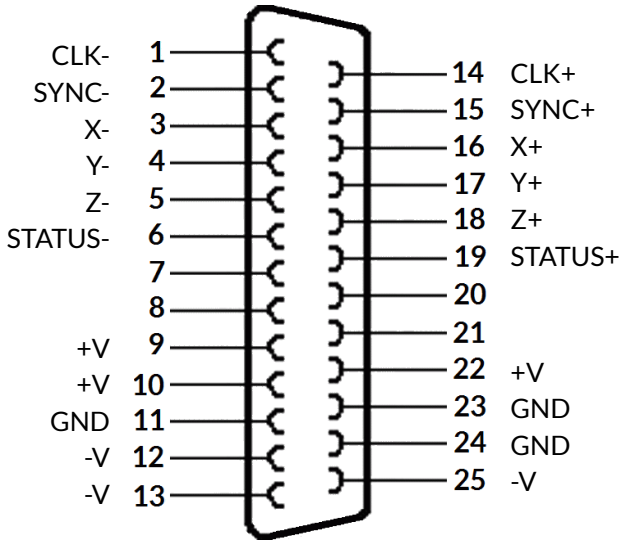
7.6 Scanner Signals

The 26 pin connector provides signals to be used to control up to three galvos of a scanhead and to power it up.

The connector provides following signals when operated in XY2-100, XY2-100E, XY2-200 or XY2-200E mode:

Upper Row Of Pins	Signal	Voltage	Remarks	Lower Row Of Pins	Signal	Voltage	Remarks
1	CLK-		XY2-100-compatible signals	**2**	CLK+		XY2-100-compatible signals
3	SYNC-			**4**	SYNC+		
5	X-			**6**	X+		
7	Y-			**8**	Y+		
9	Z-			**10**	Z+		
11	STATUS-			**12**	STATUS+		
13				14			
15				16			
17	+V	+12..24V	Power supply to scanhead (output)	**18**	+V	+12..24 V	Power supply to scanhead (output)
19	+V	+12..24V		**20**	GND	GND	
21	GND	GND		**22**	GND	GND	
23	-V	-12..24V		**24**	-V	-12..24V	
25	-V	-12..24V		26			

The D-SUB25 connector provides the same signal as described above on a default XY2-100 connector:



The connections -V, GND and +V can be used to power the scanhead with 12..24V and **max. 3A**. This requires a bipolar external power supply connected to the controllers three-pin power connector described above. Power from this power connector is routed to the -V, GND and +V pins directly, so the provided voltage should be stabilised according to the requirements of the scanhead.

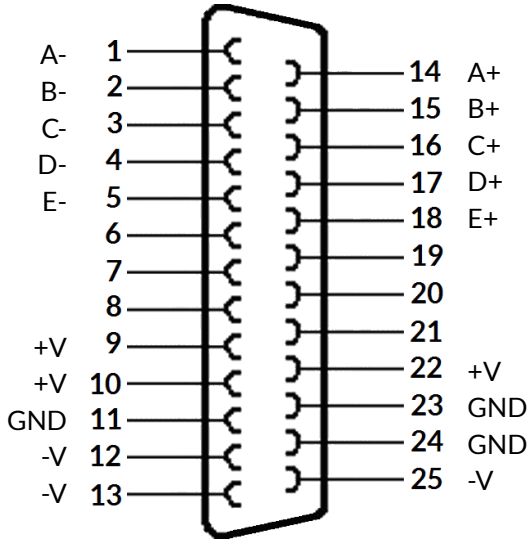
PLEASE NOTE:

- do not connect scanheads that consume more than 3A (peak and continuously), this may damage the controller and voids warranty!
- do not feed more than 24V into the three-pin power connector of E1803!
- feed a stabilised voltage into E1803D controller according to requirements of connected scanhead!
- when E1803 card is powered via three-pin power connector but scanhead has not to be powered out of the card, the 9 lines for -V, GND, +V (9..13 and 22..25) need to be disconnected, means the used D-SUB25 cable needs to leave these pins open!
- **Violating one of these rules may damage the E1803D card or scanhead irreversibly!**

E1803D can be configured to work in XY3-100 mode too. This configuration requires a firmware version 10 or newer for 2D XY3-100 signals and a firmware version 11 or newer for 3D XY3-100 signals. The scanner signal connector provides following signals when operated in XY3-100 mode:

Upper Row Of Pins	Signal	Voltage	Remarks	Lower Row Of Pins	Signal	Voltage	Remarks
1	A-		XY3-100-compatible signals	**2**	A+		XY3-100-compatible signals
3	B-			**4**	B+		
5	C-			**6**	C+		
7	D-			**8**	D+		
9	E-			**10**	E-		
11				**12**			
13				14			
15				16			
17	+V	+12..24V	Power supply to scanhead (output)	**18**	+V	+12..24 V	Power supply to scanhead (output)
19	+V	+12..24V		**20**	GND	GND	
21	GND	GND		**22**	GND	GND	
23	-V	-12..24V		**24**	-V	-12..24V	
25	-V	-12..24V		26			

The D-SUB25 connector provides the same signal as described above:



The connections -V, GND and +V can be used to power the scanhead with 12..24V and **max. 3A**. This requires a bipolar external power supply connected to the controllers three-pin power connector described above. Power from this power connector is routed to the -V, GND and +V pins directly, so the provided voltage should be stabilised according to the requirements of the scanhead.

PLEASE NOTE:

- do not connect scanheads that consume more than 3A (peak and continuously), this may damage the controller and voids warranty!
- do not feed more than 24V into the three-pin power connector of E1803!
- feed a stabilised voltage into E1803D controller according to requirements of connected scanhead!
- when E1803 card is powered via three-pin power connector but scanhead has not to be powered out of the card, the 9 lines for -V, GND, +V (9..13 and 22..25) need to be disconnected, means the used D-SUB25 cable needs to leave these pins open!
- **Violating one of these rules may damage the E1803D card or scanhead irreversibly!**

7.7 Laser Signals

The 26 pin connector provides several signals to be used to control a laser source. It can be used e.g. together with YAG, CO_2, IPG™, SPI™, fiber and compatible lasers since it provides additional signals and frequencies these laser types may require for proper operation.

The connector provides following signals:

Upper Row Of Pins	Signal	Voltage	Remarks	Lower Row Of Pins	Signal	Voltage	Remarks
1	LP8_0	CMOS, 0/5V, max 8 mA		**2**	GND	GND	
3	LP8_1	CMOS, 0/5V, max 8 mA		4	Pilot	CMOS, 0/5V, max 8 mA	Pilot laser control [1)]
5	LP8_2	CMOS, 0/5V, max 8 mA		**6**	5V	5V	Output
7	LP8_3	CMOS, 0/5V, max 8 mA		**8**	MO	CMOS, 0/5V, max 8 mA	Main Oscillator
9	LP8_4	CMOS, 0/5V, max 8 mA		10			
11	LP8_5	CMOS, 0/5V, max 8 mA		**12**	AOut0	0..10V, max 15 mA	Analogue output
13	LP8_6	CMOS, 0/5V, max 8 mA		**14**	AOut1	0..10V, max 15 mA	Analogue output
15	LP8_7	CMOS, 0/5V, max 8 mA		**16**	ExtStart	CMOS, 0/5V	Input control signal
17	LP8 Latch	CMOS, 0/5V, max 8 mA		**18**	5V	5V	
19	LaserB	CMOS, 0/5V, max 14 mA	FPK	**20**			Connected to pin 21
21			Connected to pin 20	**22**	LaserA	CMOS, 0/5V, max 14 mA	PWM, frequency or Q-Switch
23	GND	GND		**24**	ExtStop	CMOS, 0/5V	Input control signal
25	5V	5V	Output	**26**	Laser Gate	CMOS, 0/5V, max 14 mA	

[1)] requires hardware revision 1.8 or newer and firmware version 19 or newer

LP8_0...LP8_7 provide a parallel 8 bit output signal (e.g. for power control with IPG™/fiber lasers, waveform selection for SPI™ lasers and other).

LP8 Latch pin signals valid output at LP8_0..LP8_7 by submitting a latch pulse of software-controlled length.

MO can be used to enable main oscillator (e.g. for IPG™/fiber lasers or compatible), this signal is also visualised by the MO LED described above.

LaserA usage depends on software configuration and control, it is able to output a pulse-width modulated frequency (e.g. for controlling CO2 lasers), CW/continuously running frequency (e.g. for fiber lasers) or Q-Switch signal (e.g. for YAG lasers) in range 25 Hz..20 MHz.

LaserB can be used for emitting a FPK pulse (e.g. for YAG lasers).

AOut0 and AOut1 provide unipolar analogue output for controlling e.g. laser power or additional equipment or can be used for controlling power and simmer for SPI™ lasers.
PLEASE NOTE: output of 10V at AOut0 and AOut1 depends on the used power supply. So in case board is powered via USB, these outputs do not work, they require an external power supply via three-pin power connector described above.

ExtStart expects a CMOS-level input signal in respect to GND and can be used as external trigger signal to start operations when a HIGH-signal is detected at input pin.

ExtStop expects a CMOS-level input signal in respect to GND and can be used as external stop-signal in order to stop a running marking operation by using a HIGH-signal at input pin.

7.8 Digital Interface

This interface consist of different parts which belong together:

1. a 20 pin connector for connecting digital in- and output signals
2. two red jumpers to select opto-insulated or internal powered mode for the digital in- and outputs
3. 4 green and 4 red LEDs which signal the state of the digital outputs
4. 8 yellow LEDs which signal the state of the digital inputs (with hardware revisions prior to v1.7 these LEDs have been green)

The 20 pin connector provides 8 lines for input and 8 lines for output of digital signals that can work on CMOS level (non-insulated mode) or via opto-couplers (electrically insulated mode with external power supply) optionally. The operation mode depends on jumper settings described below. The connector is used as follows:

Upper Row Of Pins	Signal	Voltage	Remarks	Lower Row Of Pins	Signal	Voltage	Remarks
1	V_{ext}	5..24V	Input voltage to be used in opto-insulated mode only	**2**	GND_{ext}	GND	External ground
3	DOut0	CMOS, 0/5V or 0/V_{ext}	Default level: LOW [1)]	**4**	DIn0	CMOS, 0/5V or 0/V_{ext}	Encoder-input A1 for marking on-the-fly
5	DOut1	CMOS, 0/5V or 0/V_{ext}	Default level: LOW [1)]	**6**	DIn1	CMOS, 0/5V or 0/V_{ext}	Encoder-input B1 for marking on-the-fly
7	DOut2	CMOS, 0/5V or 0/V_{ext}	Default level: LOW [1)]	**8**	DIn2	CMOS, 0/5V or 0/V_{ext}	Encoder-input A2 for marking on-the-fly
9	DOut3	CMOS, 0/5V or 0/V_{ext}	Default level: LOW [1)]	**10**	DIn3	CMOS, 0/5V or 0/V_{ext}	Encoder-input B2 for marking on-the-fly
11	DOut4	CMOS, 0/5V or 0/V_{ext}	Default level: HIGH [1)]	**12**	DIn4	CMOS, 0/5V or 0/V_{ext}	
13	DOut5	CMOS, 0/5V or 0/V_{ext}	Default level: HIGH [1)]	**14**	DIn5	CMOS, 0/5V or 0/V_{ext}	
15	DOut6	CMOS, 0/5V or 0/V_{ext}	Default level: HIGH [1)]	**16**	DIn6	CMOS, 0/5V or 0/V_{ext}	
17	DOut7	CMOS, 0/5V or 0/V_{ext}	Default level: HIGH [1)]	**18**	DIn7	CMOS, 0/5V or 0/V_{ext}	
19	V	5V	Board output voltage, to be used only when not operating in insulated mode	**20**	GND	GND	Board-internal ground

[1)] Please note the wiring scheme and the resulting, inverted logic below: a level of LOW means, the output is pulled to GND and a load that is connected from V to this pin is turned on. An level of HIGH means, the output is pulled to V and a properly wired load if turned off.

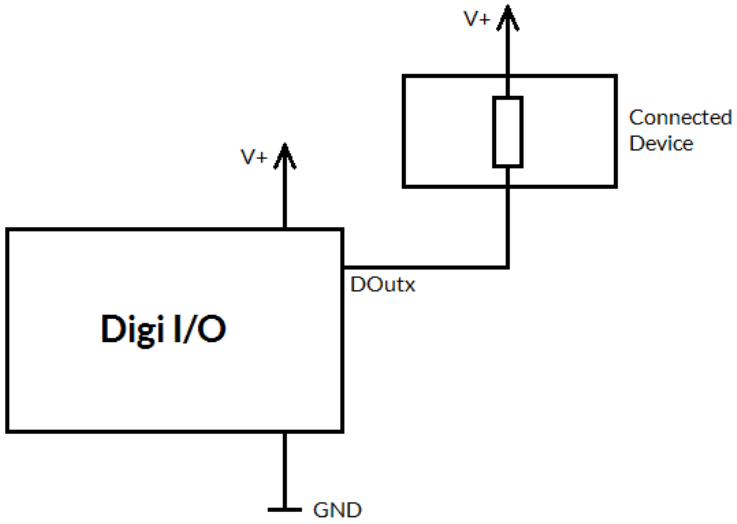
V_{ext} and GND_{ext} depend on opto-configuration as described below. In opto-insulated mode (opto-configuration jumpers not set) external power supply has to be connected to these inputs. Then DIn0..DIn7 and DOut0..DOut7 work in respect to this external power.



WARNING: When no opto-insulated mode is selected (opto-configuration jumpers are set), do NOT FEED ANY POWER into V_{ext}, this would cause damage to the E1803D board! In this case V_{ext} is equal to V (5V) of the board and GND_{ext} is connected to boards ground GND.

Maximum current for every output is 15 mA when internally powered (non-insulated mode), here it is recommended to use an external power supply.
Maximum current for outputs DOut0..DOut3 is 50 mA when externally powered (V_{ext} in insulated mode).

Signal output lines DOut0..DOut7 operate in open collector mode and have to be wired as follows:



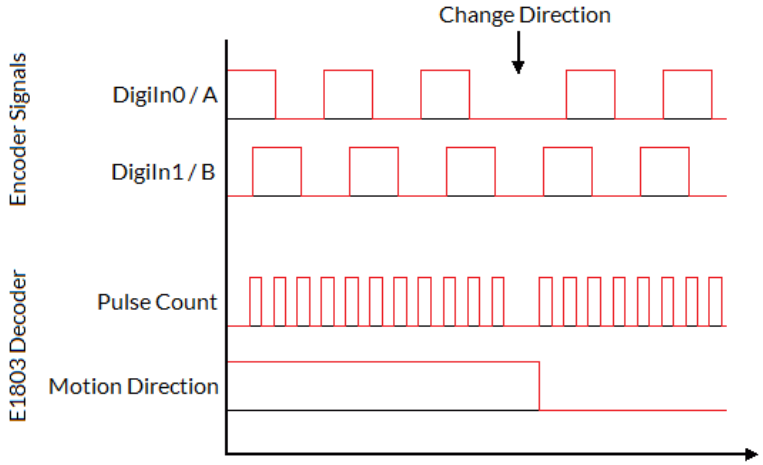
Here "DOutx" symbolises one of the digital outputs DOut..DOut7. V+ is either V (5V internal, non-insulated mode) or V_{ext} (up to 24V external, insulated mode). GND is either GND (non-insulated mode) or GND_{ext} (insulated mode). The internal resistor of the connected device is not allowed to have less than 490 Ohms in order to not exceed the given current limits.
DOut0..DOut3 provide LOW signal level by default, DOut4..DOut7 provide HIGH level by default. These levels are valid immediately on power-up of the card.

The DOut-lines make use of following logic:

Signal	LED	LED turned on on output level	Default output level on power-up
DOut0	red	HIGH	LOW
DOut1	red	HIGH	LOW
DOut2	red	HIGH	LOW
DOut3	red	HIGH	LOW
DOut4	green	LOW	HIGH
DOut5	green	LOW	HIGH
DOut6	green	LOW	HIGH
DOut7	green	LOW	HIGH

7.8.1 Marking On-The-Fly Signals

Digital inputs 0 and 1 (and optionally 2 and 3) can be used as position encoder signal inputs for marking on-the-fly applications. Here 90 degree phase-shifted input pulses are expected signalling motion direction and position change:



When these pulses are generated from a motion stage that moves the working piece, the resulting position information is used in marking on-the-fly mode to correct the marking positions accordingly. Resulting from that, marking will follow motion as far as available scanhead range and working area allows it.
The pulses generated out of the encoder signals have to be multiplied with a factor reflecting the resolution of the used encoder. To set up and adjust a marking on-the-fly-system properly, following steps have to be performed:

1. Connect encoder signals A and B to DigIn0 and DigIn1 and configure E1803D controller for encoder usage (either from within BeamConstruct or via programming interface as described below)
2. Mark a square without any encoder signals feed into the controller
3. When the square does not have exact size and/or is distorted, modify correction table and/or gain settings
4. When the square has correct size, mark it again but now with a slow motion (using encoder pulses)
5. When the square is damaged (means open on one side or compressed) the on-the-fly-factor has to be changed (set to a smaller or higher value)
6. Mark the same square again with a fast motion (using encoder pulses)
7. When the square is damaged (means open on one side or compressed) the on-the-fly-factor has to be changed (set to a smaller or higher value)

The on-the-fly-factor controls the strength of compensation and is the relation between speed of external device/encoder pulses and card-internal compensation calculation. When this factor is wrong, the marking results are distorted. For a square (as recommended to be used in calibration steps above) following results are imaginable:



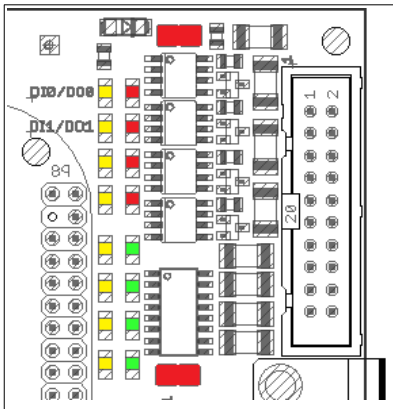
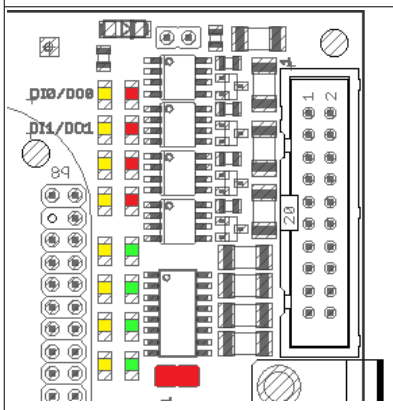
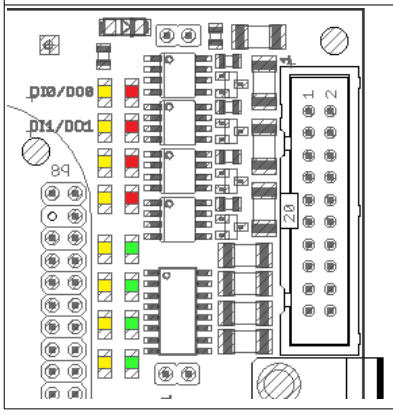
The left drawing shows an over-compensated system, here the internal compensation is too strong, the factor is too big. The right drawing shows an under-compensated set-up, here the factor is too small causing a too weak compensation. Only when marking result is really a square, the on-the-fly-factor is correct.

When tune-flag 2 is set, a second encoder can be used for 2D marking on-the-fly applications. In this mode digital inputs 0 and 1 (encoder inputs A1 and B1) correspond to X axis and on-the-fly factor for X direction. Additionally digital inputs 2 and 3 (encoder inputs A2 and B2) correspond to Y axis and on-the-fly factor for Y direction. Operation principle is the same as for 1D on-the-fly described above: the incremental values received from the encoders for X and Y are added to the current X and Y coordinates to be marked. Procedure for

adjusting the encoder factor is also the same, here it is recommended to perform this operation for X and Y movements separately and finally try both motion directions together.

7.8.2 Opto-Configuration Jumpers

Using these jumpers the operation mode for digital I/Os 0..7 can be chosen:

	Opto-mode 2, both jumpers set: not galvanically insulated, internally powered, using common ground (external ground GND_{ext} is connected with internal ground) ⚠ DO NOT CONNECT EXTERNAL POWER V_{ext}! Digital outputs: 5V (internal) Digital inputs: 5..24V in respect to GND and GND_{ext}
	Opto-mode 1, lower jumper set: not galvanically insulated, using common ground, external ground GND_{ext} is connected to internal ground, external power V_{ext} in range 5..24V has to be applied Digital outputs: 5..24V (V_{ext}) Digital inputs: 5..24V in respect to GND and GND_{ext}
	Opto-mode 0, no jumpers set: galvanically insulated (fully opto-coupled mode), external ground GND_{ext} is separated from internal ground, external power V_{ext} in range 5..24V has to be applied Digital outputs: 5..24V (V_{ext}) Digital inputs: 5..24V in respect to GND_{ext}

7.8.3 Output State LEDs

The green and red LEDs close to the connector signal the output state of the digital outputs. As shown in table above, four outputs have default state LOW (non inverted) and four have default state HIGH (inverted). The same is signalled by these green/red LEDs, they are on/off for output state HIGH/LOW. So please note: one of the output LEDs turned on does NOT necessarily mean the output is at HIGH level, this depends on the output and its corresponding default output level/output logic!

7.8.4 Input State LEDs

These 8 yellow LEDs show the state of corresponding 8 digital inputs. As long as a HIGH signal is detected on an input, the related LED is turned on.

7.9 Serial Interface

This is an 8 pin connector which provides access to UART0 RS232 and RS485 connection lines.
PLEASE NOTE: both, the RS232 and the RS485 interface are connected to the same serial interface internally! This means although there are two interface types available, only one logical serial line exists! Connecting two signal lines to RS232 and RS485 at the same time may damage the board irreversibly!

Upper Row Of Pins	Signal	Voltage	Remarks	Lower Row Of Pins	Signal	Voltage	Remarks
1	RX0	max +-28V	UART0 RS232	**2**	TX0	+-7V	UART0 RS232
3	GND			4			Do not connect!
5	RX0+	+5V	UART0 RS485	**6**	RX0-	-5V	UART0 RS485
7	TX0+	+5V		**8**	TX0-	-5V	

7.10 Extension Connectors

These connectors can be used to plug additional boards which provide new functions and additional interfaces. For a detailed description of available boards please refer below.

The two extension connectors on left hand side of the board can be used to place extension boards with additional peripheral interfaces. The extension connectors are designed to place/remove boards from time to time but they are not intended for constant hardware changes. So changing extension boards repeatedly and often e.g. as permanent part of a production process is not recommended.

PLEASE NOTE: when placing a new extension board
1.check correct orientation and position of the code pin which is closed in connector
2.place the pins of the extension boards onto the extension connectors exactly
3.move down the extension board by pressing on its extension connectors gently; DO NOT PRESS THE BOARD ITSELF BUT ONLY THE CONNECTORS!

PLEASE NOTE: When removing an extension board DO NOT pull on the extension connectors but hold both boards on their long side directly at the PCBs edges!

Due to of the large number of pins, it is easy to plug in an extension but more difficult to pull it out. So when removing an extension board, it is recommended to be very slow and to carefully pull each side up just a little bit to avoid bending of the pins as they exit.

8 Stand-Alone Operation

E1803D scanner controller cards can be operated in stand-alone mode. In this mode all marking data are stored on microSD-card and the board can operate without direct control of a host-PC that sends the data to be marked. Such stand-alone marking data can be created e.g. in BeamConstruct marking software.

The names of these stand-alone files have to be in format 8.3, means the filename has to consist of eight characters at max, followed by a file extension which consists of 3 characters. The base-stand-alone file comes with a file-extension .EPR. In case the stand-alone file contains dynamic data, a second file with the same filename but the extension .DAT is created (for details about dynamic stand-alone data please refer to section “8.1 Create Stand-Alone Data with BeamConstruct”). When a stand-alone-mode of type “ioselect” is used, the filename has to follow some specific rules too, here it typically has to be a number which corresponds to the selection done at the digital inputs (for details about the different stand-alone modes and their behaviour please refer to section “8.2 Stand-Alone Configuration Parameters”).

8.1 Create Stand-Alone Data with BeamConstruct

To use BeamConstruct for generation of stand-alone data for E1803D scanner controllers, the card has to be fully configured (including all scanner, laser and pen-parameters). Next the marking data to be stored on microSD-card have to be created. To generate stand-alone data, menu "Processing", submenu "Write Marking Data to File" or "Send Named Marking Data" has to be selected.

First one gives the possibility to write the data to microSD card when E1803D is switched off and the microSD card is plugged into host PC. Here it is recommended to use file extension ".EPR" for the file generated by BeamConstruct. Next it is also recommended to always let BeamConstruct write to microSD card directly because sometimes more than only one file is created. Direct write operation to BeamConstruct ensures all files are available on microSD and no data can be forgotten to be copied.

The second variant allows to download the stand-alone data to the controller while it is connected and running. Precondition for sending data to a running controller are:

- no mark operation is in progress (controller is idle)
- no stand-alone project is loaded (please refer to description of stand-alone modes and control commands below).
- a valid name is given in style 0:/filename.epr

This operation creates the .EPR-file and all additional files on microSD card of the running controller automatically.



PLEASE NOTE: such an .EPR-standalone file can NOT be converted back to vector data that could be edited in BeamConstruct! Creating these files is a one-way-conversion of your projects. Thus it is recommended to save these projects twice – once as normal .BEAMP-File which can be loaded and modified later and once as .EPR-file which has to be used on SD-card. This also means such .EPR-files are protected so that it is possible to give away own designs to some end-users which shall not be able to modify them.

E1803D controller supports all static data in stand-alone mode (like all kinds of static geometries, output signals, waiting for input commands, waiting for trigger, all laser- and scanner parameters as well as elements which set outputs directly). But it does not store the vector data using a possibly configured correction table! To get a valid correction for stand-alone operations, the related correction file has to be saved on microSD card and needs to be activated using parameter "corrtable0" in e1803.cfg configuration file (please refer to description above).

Next E1803D scanner card supports dynamic content when following conditions are met:

- a text element uses one of the laser vector font families "Roman", "Script" or "Times" and it makes use of an input element or
- a text element makes use of a TrueType font and it makes use of an input element; here any available TrueType font can be used and several hatch-patterns can be applied

  → for both types of text some limitations apply (only left to right orientation, no scaling/rotation/slant/mirroring is applied to the font and only the characters ' ', !, ", #, $,

`%, &, \, (, ), *, +, ,, -, ., /, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, :, ;, <, =, >, ?, @, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, [, \, ], ^, _, `, a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z, {, |, }` and ~ can be used, firmware version 3 or newer is required, BeamConstruct version 4.8 or newer is required); or

- a barcode element uses type "DataMatrix" or "QR" with option "Merge cells" disabled and it makes use of an input element
- any kind of hatch and combined hatches can be applied to a barcode element, not to a text element
- marking output is neither XY-flipped nor mirrored nor rotated or slanted
- when an input element of type "Serial Number" is used, serial number counting is done according to the settings of the related element
- when an input element of type "CSV File Data" is used, elements out of a CSV-table can be read and used for the dynamic element:
  the CSV-file itself needs to be placed on the SD-card and has to use the same name like the EPR file but with extension CSV (so when the stand-alone file is named "data.epr", the CSV-file needs to be named "data.csv"), resulting from that one EPR file can handle exactly one CSV file;
  supported parameters of the CSV input element are the column separator, the data column to read the data from and the "Endless loop" option (for details please refer to the manual of BeamConstruct);
  this feature requires a firmware version 13 or newer
- when any input element is used, the contents of the text/barcode can be changed via command "`cstxt`", in this case the automatic updating of the input element has no effect

When these conditions are met, a text or barcode can be modified during stand-alone operation either via stand-alone control commands as described below or via a Serial Number input element that is applied to it in BeamConstruct. Here all serial number, time, date and formatting functions of this input element are supported. To get a valid time in stand-alone mode, it needs to be set after boot-up via stand-alone control commands (as described below) or a SNTP time server has to be configured to retrieve current time from an external source (please refer to description e1803.cfg parameters above).

8.2 Stand-Alone Configuration Parameters

Within e1803.cfg configuration file of E1803D scanner controller one of the following stand-alone operation modes can be selected via the configuration parameter "`standalone`":

```
standalone=off
```

Stand-alone mode is fully disabled, the card acts as normal host-PC-controlled device and all .epr-files on the SD-card are ignored. Digital outputs are not toggled since no stand-alone operational states have to be signalised here (please refer next section).

```
standalone=dotmark
```

This is a special stand-alone marking mode where E1803D controller card acts as dot matrix marker. It is available in firmware versions 5 or earlier only. For details please refer section "9 Matrix Laser Dot Marking Mode" below.

```
standalone=auto
```

Stand-alone mode is enabled, a file specified by and additional parameter "autofile" is loaded and prepared for marking. Marking of this file is started only when an external trigger signal is detected. The file itself has to be specified via additional configuration parameter that gives the filename of the stand-alone file to be loaded. As an example a parameter: "`autofile=0:/myfile.epr`" would try to load the file "myfile.epr" from SD-card and prepare it for marking. In this mode the digital outputs are toggled as described in next section.

```
standalone=loop
```

This is the same like mode "auto" described above but using "loop" E1803D controller does NOT wait for an external trigger signal! So when no trigger points are set in stand-alone datafile itself, in this mode marking would be done in an infinite loop, repeating the given "`autofile`" again and again.

`standalone=haltedloop`

This is the same like mode "loop" described above, but marking does not start immediately. By default the controller is in state “halt” until the ExtStart input is set to HIGH level. Marking continues only as long this input stays at HIGH. When it goes back to LOW, marking is continued until the laser is turned off the next time and it is halted again. Next time ExtStart goes to HIGH, marking continues at the position where it was halted before. In this mode the timeout-parameter “`haltedlooptimeout`” can be used.

`standalone=iohaltedloop`

This mode is a combination out of "haltedloop" described above and “ioselect” described below (please refer there for usage details). In this mode a project can be selected via digital inputs but it is started immediately and marked in an endless loop as long as ExtStart input is HIGH (so the level at ExtStart is checked, not the rising edge of an applied signal). When a different project is selected by applying a different input pattern at DIn digital inputs, the current project is cancelled and the new one is started in a loop again.
This mode requires firmware version 2 or newer.
In this mode the timeout-parameter “`haltedlooptimeout`” can be used.

`standalone=ioselect`

This mode makes use of the digital interface (please refer above). Here it is possible to select one of 256 stand-alone marking jobs via a signal the digital inputs. The number that results out of the input pattern of the digital input lines specifies the filename of the marking job that has to be loaded from SD card:

Selected input(s)	**Stand-alone file loaded from SD-card**
All inputs set to LOW (not recommended to be used)	0.epr
DIn0 set to HIGH	1.epr
DIn1 set to HIGH	2.epr
DIn0 and DIn1 set to HIGH	3.epr
DIn2 set to HIGH	4.epr
DIn0 and DIn2 set to HIGH	5.epr
DIn1 and DIn2 set to HIGH	6.epr
DIn0, DIn1 and DIn2 set to HIGH	7.epr
DIn3 set to HIGH	8.epr
...	...
DIn4 set to HIGH	16.epr
...	...
DIn5 set to HIGH	32.epr
...	...
DIn6 set to HIGH	64.epr
...	...
DIn7 set to HIGH	128.epr
...	...
All inputs set to high	255.epr



PLEASE NOTE: 0.epr (no inputs set to HIGH) can be used but it is not recommended to do that. This value should be reserved for "no job active" to set the card into an inactive mode also in stand-alone operational mode. This may be necessary e.g. when new project data are downloaded to the controller without removing the SD-card.

Marking of a IO-selected job is started by external trigger signal (ExtStart input). When the input pattern at DIn0..DIn7 changes during marking, the currently running operation is continued and the other stand-alone job is loaded after marking operation has finished. In this mode the digital outputs are toggled as described in next section.

In stand-alone mode “ioselect” or “idxselect” .EPR-files are loaded from microSD card as soon as a new input pattern is detected at digital inputs or as soon as a new index is selected with command “clepr”. Depending on the size of the .EPR file and the speed of the microSD card, this may take a time that is too long for high-speed applications. Thus it is possible to operate such projects from controller's RAM completely: in e1803.cfg the numbers of the files to be loaded have to be specified with parameter “iobuff”, it can be used up to 20 times and expects the number of the file (so a line “iobuff=3” would be responsible for pre-loading file “0:/3.epr”). File “0.epr” can not be preloaded by this command.

Now these files are loaded into RAM and switching from one to an other is done much faster since toggling between them is done controller-internal and no more disk-operations are necessary for that.

PLEASE NOTE: when too much too large .EPR files are selected for preloading, this may exceed the available memory on card. This is signalled by the Error LED turned on and an appropriate message is stored in log buffer.

`standalone=idxselect`

This mode works exactly like the mode “ioselect” (described above), but it does not occupy any digital inputs. Instead of that, loading of a file can be done via an index number and the command `clepr`. There the number of the file to be loaded has to be given. For a full description of all functions and features of mode “idxselect”, please refer the “ioselect” description above.
This stand-alone mode requires a firmware version 8 or newer.

`iolatch=1`

This option can be enabled for one of the digital-input-controlled stand-alone modes “ioselect”, “idxselect” and “iohaltedloop”. When this option is set to 0 or when it does not exist in e1803.cfg, the digital input bitpattern at DIn0..DIn7 is used as new input value as soon as it is detected. When it is enabled, DIn7 is used as latch-bit. Then the digital input bitpattern at DIn0..DIn6 is used only when DIn7 is set to HIGH. So a proper method of selecting a digital input bitpattern with latch enabled would be:

- ensure DIn7 is at LOW
- apply the desired bitpattern at DIn0..DIn6
- wait for the maximum time the input bits may need to settle (depends on the external hardware and its capabilities)
- set DIn7 to HIGH
- wait until DOut0 goes to LOW
- wait until DOut1 goes back to HIGH (when a valid bitpattern was applied that corresponds to an existing EPR file) or until loading timeout has elapsed (when a bitpattern has been applied where no EPR file exists for
- set DIn7 to LOW

So as long as DIn7 is at low, state-changes at DIn0..DIn6 are ignored and the last detected bitpattern is used. In this mode only 127 different input bitpatterns are possible as DIn7 is used as latch bit.
This function is available in firmware versions 11 or newer.

8.3 Stand-Alone Control

The current stand-alone operational state is signalised via digital outputs:

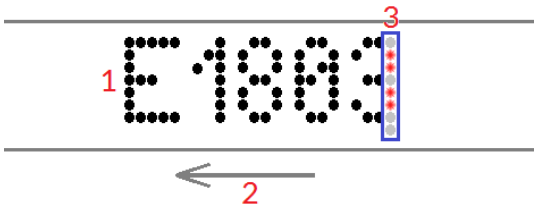
DOut0 – ready for marking – this output goes to HIGH as soon as a stand-alone job could be found on disk, was loaded successfully and is ready for marking. So external start signal should not be given until this output is HIGH. When a new stand-alone file is selected (e.g. via digital inputs in "`ioselect`"-mode) this output goes to LOW. It is switched back to HIGH only when the new file could be loaded successfully too.
This signal can be mapped to a different hardware output using configuration parameter `tunereadyout` and the related tune-flag.

DOut1 – marking active – as long as this output is HIGH, a marking operation is in progress. When a different stand-alone file is selected (e.g. via digital inputs) as long as this output is HIGH, marking is continued and the new file is NOT loaded. Once the current marking operation is completed, the output goes to LOW. After that the board continues with current marking data (when no new ones have been selected), or it tries to load new ones (when a new file was selected).
Please note: in firmware versions prior to v3 in stand-alone modes "`haltedloop`" and "`iohaltedloop`" this signal is not used since the user had full control over the marking process via the ExtStart input. So in case of these modes the logic is: when both, DOut0 and ExtStart are at HIGH, the controller is marking. Starting with firmware version 3 the DOut1 signal is used in same way as for all other stand-alone modes.
This signal can be mapped to a different hardware output using configuration parameter `tunemarkout` and the related tune-flag.

9 Matrix Laser Dot Marking Mode

Please note all functions described in this chapter are deprecated. They are available only for firmware version 5 or older. Before using these functions in new products, please contact HALaser Systems!

The E1803D controller card can be operated in a special mode where it does not make use of XY2-100 interface but acts as dot marker for matrix marking applications. Here a matrix laser, a dot peen or an other matrix printing device can be used. The dots are controlled via digital outputs turning the related dot on and off:



The controller works in stand-alone mode with no laser marking software connected to it. The objects to be marked have to be moved (2, from right to left in image above) and the movement information is feed into the controllers encoder inputs A1 and B1 of digital interface connector (for description please refer to section 7.8.1). Depending on the current position, the related dot-outputs are turned on and off marking one column on every distance step (3 symbolises the dot marking device in image above, red dots are lasers which are firing for the current column). This way the motion of the working piece forms up to two separate lines of text to be marked (1 in image).

9.1 Dot Mode Configuration Parameters

Within e1803.cfg configuration file of E1803D controller card the dot marking mode has to be configured by using different parameters:

Parameter	Description	Example
standalone	When set to "`dotmark`" this parameter enables the dot matrix marking mode. This is mandatory to use the following parameters, elsewhere they do not have any effect on the operation of the card.	`standalone=dotmark` Enables the dot matrix marker mode of E1803D
dotfont0	Specifies which font has to be used. The font size specifies how much of the max. 13 dots are used. All the available fonts are stored on microSD-card in sub-folder "fonts". Own fonts can be added to this, please contact HALaser Systems for details. This parameter is mandatory, for a single line of text it specifies the font for this line, when two lines of text have to be marked, it specifies the font for the upper line.	`dotfont0=0:/fonts/mono7.dfn` load the 7 pixels high, mono-spaced font "mono7.dfn" out of the "fonts"-folder of microSD-card
dotfont1	Specifies which font has to be used for the (optional) second line. This parameter is optional and has to be set only when two lines of text have to be marked below of each other. It specifies the font for the lower line. When the second line is used, parameter "dotfont1y" has to be set too.	`dotfont1=0:/fonts/straig6.dfn` load the 6 pixels high, mono-spaced font "mono6.dfn" out of the "fonts"-folder of microSD-card
dotfont1y	When two lines of text are marked this parameter specifies the y-offset of the second line (in unit dots). Using it vertical the position of the lower text line has to be set.	`dotfont1y=7` sets an Y-position of 7, means the upper border of the second (optional) text line will start at Dot7 output
dotdist	This parameter specifies the horizontal distance between two columns of dots. The value given here is in unit "increments" of connected encoder. When the value is positive, the encoder has to be	`dotdist=7500` starts a new column of dots whenever 7500 encoder increments have elapsed

	connected in a way where it counts to positive direction. In this case the text is marked from left to right, motion has to be done from right to left. When the value is negative, the encoder has to be connected in a way where it counts to negative direction. In this case the text is marked in reverse direction from right to left (mirrored in horizontal direction) and the motion of the working pieces need to have a suitable movement direction in order to have the expected results and no mirrored texts. The encoder counting direction has to fit to the configuration. When it works in wrong direction, only the first column of a text is marked, then nothing happens for a very long time. In this case it is necessary to exchange the two encoder lines connected to A1 and B1 in order to correct the counting direction.	
dottime	Specifies how long the dots have to be turned on at maximum. The value has to be given in unit nanoseconds and has a resolution of 500 nsec. When no dot-time is specified, turned on dots stay on until the motion has made enough progress to switch on the next row of dots. So in this case when the motion stops, the already turned on dots stay on endless.	`dottime=20000` turn off the dots latest after 20 usec

9.2 Dot Mode Hardware Interface

In dot matrix marking mode the laser connector is used to control the single dots. As soon as stand-alone mode “dotmark” is enabled, pinout of this connector is different:

Upper Row Of Pins	Signal	Voltage	Remarks	Lower Row Of Pins	Signal	Voltage	Remarks
1	Dot0	CMOS, 0/5V, max 8 mA		**2**	GND	GND	
3	Dot1	CMOS, 0/5V, max 8 mA		4			
5	Dot2	CMOS, 0/5V, max 8 mA		**6**	5V	5V	Output
7	Dot3	CMOS, 0/5V, max 8 mA		**8**	Dot10	CMOS, 0/5V, max 8 mA	
9	Dot4	CMOS, 0/5V, max 8 mA		10			
11	Dot5	CMOS, 0/5V, max 8 mA		**12**			Do not connect!
13	Dot6	CMOS, 0/5V, max 8 mA		**14**			Do not connect!
15	Dot7	CMOS, 0/5V, max 8 mA		**16**	ExtStart	CMOS, 0/5V	Input control signal
17	Dot8	CMOS, 0/5V, max 8 mA		**18**	5V	5V	
19	Dot9	CMOS, 0/5V, max 14 mA		**20**			Connected to pin 21
21			Connected to pin 20	**22**	Dot11	CMOS, 0/5V, max 14 mA	
23	GND	GND		**24**	ExtStop	CMOS, 0/5V	Input control signal
25	5V	5V	Output	**26**	Dot12	CMOS, 0/5V, max 14 mA	

Dot0 is always the uppermost dot. Usage of all other dots depends on the height of the used font. So when a font with a size of 8 is chosen, the dots Dot0..Dot7 are used.
The dot-outputs are switched to HIGH signal whenever a dot has to be marked. This behaviour can be changed by setting the "`tune`"-parameter to value 8 (for details please refer section 7.5 above), then they are inverted and are set to HIGH when they are off.
The ExtStart input is used to start marking of one line of predefined text.
Dots can be toggled with a maximum frequency of 2 MHz.

9.3 Dot Mode Control

The dot matrix marking mode is a stand-alone operation mode which can be controlled from outside easily. After proper configuration of the related parameters in e1803.cfg, data to be marked can be sent to the card via commands "`cdtl0`" and optional "`cdtl1`" (for a more detailed description of this command interface please refer to section 18.2 below). This command can be used to enqueue several texts in advance. On every rising edge on ExtStart input output of the next text in queue is started according to the current encoder position.

During operation the current dot mode stand-alone operational state is signalised via digital outputs:

DOut0 – ready for marking – this output goes to HIGH as soon as some text was received which can be marked in dot mode. So external start signal to should not be given until this output is HIGH. When no more data are available to be marked or when marking is currently in progress, this output goes to LOW.
When dot mark mode is configured to use one line of text only (parameter `dotfont1` not set), the controller becomes ready for marking as soon as at least one text is available (to be set with command "`cdtl0`").
When dot mark mode works using two lines of text (parameter `dotfont1` set), the controller always requires pairs of data, means it becomes ready for marking only when text for two lines is available (to be set with both commands "`cdtl0`" and "`cdtl1`").

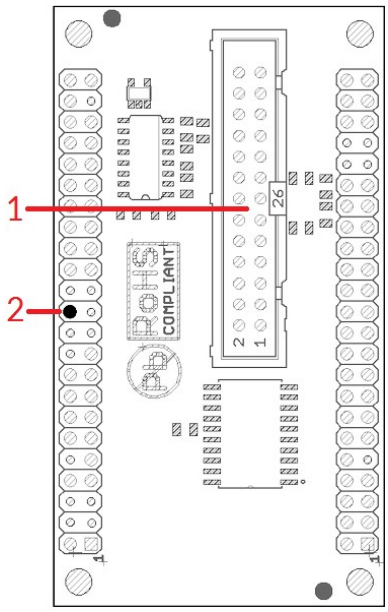
DOut1 – marking active – as long as this output is HIGH, a marking operation is in progress. During this time it is still possible to feed new marking data via commands "`cdtl0`" and "`cdtl1`", this does not influence the current operation.

10 Multi-IO Extension Board

This board is deprecated and should not be used any more.

The controller card can be extended by the Multi-IO board which utilitises the extension connector as described in section “7 Board And Connectors”.
To operate the controller with the Multi-IO Extension Board, firmware version 4 or newer is needed.

10.1 Board Connectors



The E1803D Multi-IO Extension Board provides following connectors and interfaces:

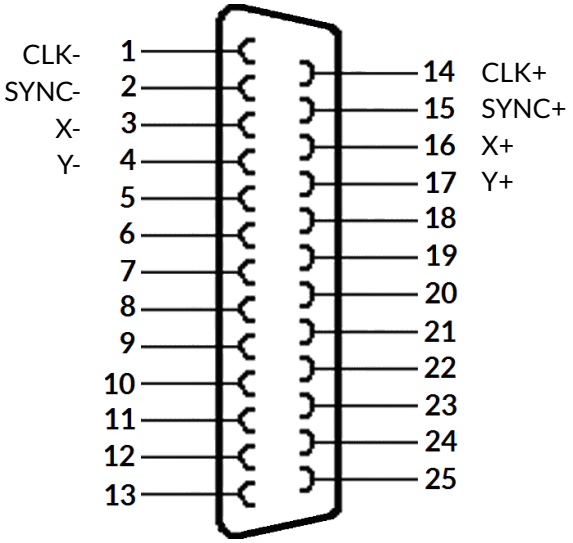
1. Multi-IO-connector offering different signals and interfaces including a secondary XY2-100(E)/XY2-200(E)/XY3-100 interface, analogue inputs, RS232/RS485 serial output
2. Code pin for proper placement of extension board on E1803D controller card

10.2 Multi-IO Interface

The 26 pin connector provides signals and inputs to be used for different purposes:

Upper Row Of Pins	Signal	Voltage	Remarks	Lower Row Of Pins	Signal	Voltage	Remarks
1	CLK-		Secondary head, XY2-100-compatible output signals	**2**	CLK+		Secondary head, XY2-100-compatible output signals
3	SYNC-			**4**	SYNC+		
5	X-			**6**	X+		
7	Y-			**8**	Y+		
9			Do not connect!	10			Do not connect!
11	GND_{AIn}	GND	12 bit analogue inputs	**12**	AIn0	0..5V	12 bit analogue inputs
13	AIn1	0..5V		**14**	AIn2	0..5V	
15			Do not connect!	16			Do not connect!
17			Do not connect!	18			Do not connect!
19	RX1	12V	UART1 RS232	**20**	TX1	12V	UART1 RS232
21	GND			**22**	GND		
23	RX1+	5V	UART1 RS485	**24**	RX1-	5V	UART1 RS485
25	TX1+	5V		**26**	TX1-	5V	

The first 8 pins of the 26 pin connector provide signals to be used to control up to two galvos of a scanhead. These signals are fully parallel to the ones from XY2-100 interface of E1803D main board and can be used for secondary head applications where two scanheads work in parallel. With a flat wire belt that makes use of these first 8 pins only, a direct connection with a standard D-SUB25 connector can be made:



The pins 11 to 14 offer three analogue measurement inputs AIn0 .. AIn2 which can be operated in respect to GND_{AIn} and work with a voltage range of 0..5V. This range is converted to a 12 bit digital value in range 1..4095 and can be read out of a connected application or can be used to perform different automated tasks.

PLEASE NOTE: under no circumstances apply a voltage higher than 5V to any of these inputs, this may damage the complete scanner controller card irreversibly!

The last 8 pins provide access to UART1 RS232 and RS485 connection lines.

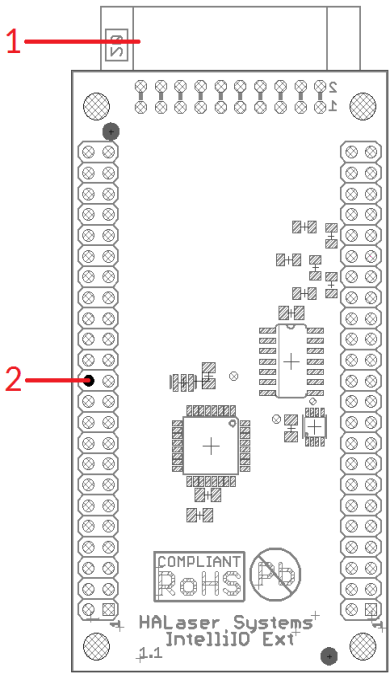
PLEASE NOTE: both, the RS232 and the RS485 interface are connected to the same serial interface internally! This means although there are two interface types available, only one logical serial line exists! Connecting two signal lines to RS232 and RS485 at the same time may damage the complete scanner controller card irreversibly!

11 Intelli-IO Extension Board v1

Please note: this extension is outdated and should not be used for new designs. Please refer to Intelli-IO Extension Board v2 below at "12 Intelli-IO Extension Board v2"!

The controller card can be extended by the Intelli-IO Extension board which utilises the extension connector as described in section "7 Board And Connectors".
To operate the controller with the Intelli-IO Extension Board, firmware version 5 or newer is needed.
This board offers different types of IO and also provides the possibility for software customisation according to customers special needs. Due to flexibility it can be operated in different modes and therefore used for different purposes.

11.1 Board Connectors



The E1803D Intelli-IO Extension Board provides following connectors and interfaces:

1. IO-connector offering different signals and interfaces as described below including digital inputs, digital outputs and analogue inputs
2. Code pin for proper placement of extension board on E1803D controller card

11.2 Intelli-IO Interface in IO mode

When operated in IO-mode, the 20 pin connector provides following signals and inputs:

Lower Row Of Pins	Signal	Voltage	Remarks	Upper Row Of Pins	Signal	Voltage	Remarks
1	DOut0	5V	Digital outputs, second 8 bit port	**2**	DOut1	5V	Digital outputs, second 8 bit port
3	DOut2	5V		**4**	DOut3	5V	
5	DOut4	5V		**6**	DOut5	5V	
7	DOut6	5V		**8**	DOut7	5V	
9	DIn0	5V	Digital inputs, second 8 bit port (6 bits used)	**10**	DIn1	5V	Digital inputs, second 8 bit port (6 bits used)
11	DIn2	5V		**12**	DIn3	5V	
13	DIn4	5V		**14**	DIn5	5V	
15	GND_{AIn}	GND	12 bit analogue inputs	**16**	AIn0	0..5V	12 bit analogue inputs
17	AIn1	0..5V		**18**	AIn2	0..5V	
19	5V	5V	Output	**20**	GND	GND	

The pins 1 to 8 provide 8 general purpose digital outputs. Maximum current to be pulled out of each output is 20 mA. It is recommended to supply power to E1803D controller via 3pin screw connector but not via USB when this extension board is used.
The pins 9 to 14 are digital general purpose input pins. They are low-active, means to signal a logical "1", they have to be pulled to ground (GND).

The pins 15 to 18 offer three analogue measurement inputs AIn0 .. AIn2 which can be operated in respect to GND_{AIn} and work with a voltage range of 0..5V. This range is converted to a 12 bit digital value in range 1..4095 and can be read out of a connected application or can be used to perform different automated tasks.

PLEASE NOTE: under no circumstances apply a voltage higher than 5V to any of these inputs, this may damage the complete scanner controller card irreversibly!

This extension board makes use of an own MCU (**Mic**rocontroller **U**nit) which handles all digital IOs and also has access to analogue inputs AIn0 and AIn1. This MCU can work fully parallel to the main scanner controller card and therefore is suitable for special, customised control applications. To get more information about such applications and to discuss your requirements, please contact HALaser Systems.

11.3 Intelli-IO Interface in motion mode

The hardware described within this section is designed to control motors. Motions caused by these motors may effect a person's health or may otherwise cause damage. Prior to installation and operation compliance with all relevant safety regulations including additional hardware-controlled safety measures has to be secured. The client shall solely be responsible to strictly comply with all applicable and relevant safety regulations regarding installation and operation of the system at any time.

When operated in motion-mode, the extension can be used to drive up to four stepper motor axes via step/direction signals. Here the 20 pin connector provides following signals and inputs:

Lower Row Of Pins	Signal	Voltage	Remarks	Upper Row Of Pins	Signal	Voltage	Remarks
1	Step0	5V	Stepper pulse output signals	**2**	Dir0	5V	Stepper motor direction output signals
3	Step1	5V		**4**	Dir1	5V	
5	Step2	5V		**6**	Dir2	5V	
7	Step3	5V		**8**	Dir3	5V	
9	Ref0	5V	Reference inputs	**10**	Ref1	5V	Reference inputs
11	Ref2	5V		**12**	Ref3	5V	
13	Do not connect!			14	Do not connect!		
15	GND_{AIn}	GND	12 bit analogue inputs	**16**	AIn0	0..5V	12 bit analogue inputs
17	AIn1	0..5V		**18**	AIn2	0..5V	
19	5V	5V		**20**	GND	GND	

The pins 1 to 8 provide the stepper motor control signals for axes 0..3 (step/direction signals to be used with a separate, external power driver). Maximum current to be pulled out of each output is 20 mA. It is recommended to supply power to E1803D controller via 3pin screw connector but not via USB when the Intelli-IO extension board is used.
The pins 9 to 12 are input pins for axes 0..3 to be used with the reference/homing position. They are low-active, means to signal a “switch was hit”, they have to be pulled to ground (GND).

The pins 15 to 18 offer three analogue measurement inputs AIn0 .. AIn2 which can be operated in respect to GND_{AIn} and work with a voltage range of 0..5V. This range is converted to a 12 bit digital value in range 1..4095 and can be read out of a connected application or can be used to perform different automated tasks.

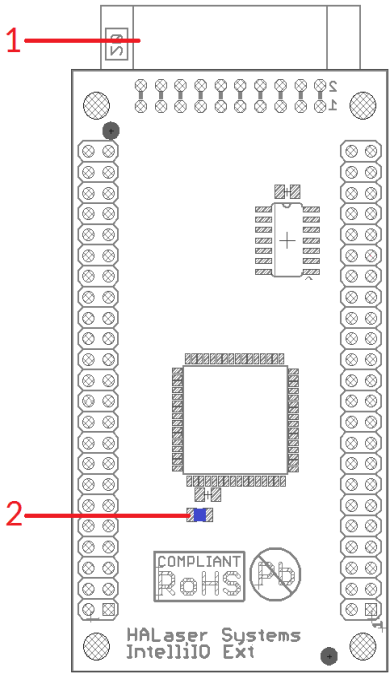
PLEASE NOTE: under no circumstances apply a voltage higher than 5V to any of these inputs, this may damage the complete scanner controller card irreversibly!

12 Intelli-IO Extension Board v2

The controller card can be extended by the Intelli-IO Extension board which utilises the extension connector as described in section “7 Board And Connectors”.
To operate the controller with the Intelli-IO Extension Board v2, firmware version 24 or newer is needed.
This board offers different types of IO and also provides the possibility for software customisation according to customers special needs. Due to flexibility it can be operated in different modes and therefore used for different purposes.
The v2 variant is fully backwards compatible to the board described at “11 Intelli-IO Extension Board v1” and provides followign enhancements:

- high-impedance analogue inputs
- all three analogue inputs are available at the Intelli-IO own MCU and can be used for custom operations
- up to 350 kHz stepper frequency in motion mode
- fully CRA-compliant

12.1 Board Connectors



The E1803D Intelli-IO Extension Board provides following connectors and interfaces:

1. IO-connector offering different signals and interfaces as described below including digital inputs, digital outputs and analogue inputs
2. State-LED to signal the activity and operation of the board

12.2 State LED Signals

The state LED is able to signal different operational states. During successful boot, following sequence is executed:

- LED blinking extremely fast
- LED is off
- LED blinking extremely fast
- LED blinking slowly and regularly

After this sequence, the firmware of the extension board has been loaded successfully and the board if cully operational. During an automated firmware update the sequence is different:

- LED blinking extremely fast (for a very short time)
- LED is permanently on while the A/E-LED in the E1803D blinks fast (this may need some time, at this stage in no case turn off the power until the A/E-LED on the main controller board is blinking regularly)
- LED is blinking fast while the A/E-LED in the E1803D blinks very
- LED blinking extremely fast
- LED blinking slowly and regularly

In both cases after the initial sequence the extension board should end up with a state LED blinking slowly and signalling everything is OK.

In case something has gone wrong, there are other signalling states possible:

- the state LED blinks with being on 10 times longer than being off: there is a general problem in loading the firmware, possibly the firmware used on the SD-card is not suitable for operating the Intelli-IO Extension v2
- after blinking extremely fast the state-LED stays on or off permanently: the firmware does not work, possibly the firmware used on the SD-card is not suitable for operating the Intelli-IO Extension v2

12.3 Intelli-IO Interface in IO mode

When operated in IO-mode, the 20 pin connector provides following signals and inputs:

Lower Row Of Pins	Signal	Voltage	Remarks	Upper Row Of Pins	Signal	Voltage	Remarks
1	DOut0	5V	Digital outputs, second 8 bit port	**2**	DOut1	5V	Digital outputs, second 8 bit port
3	DOut2	5V		**4**	DOut3	5V	
5	DOut4	5V		**6**	DOut5	5V	
7	DOut6	5V		**8**	DOut7	5V	
9	DIn0	5V	Digital inputs, second 8 bit port (6 bits used)	**10**	DIn1	5V	Digital inputs, second 8 bit port (6 bits used)
11	DIn2	5V		**12**	DIn3	5V	
13	DIn4	5V		**14**	DIn5	5V	
15	GND_{AIn}	GND	12 bit analogue inputs	**16**	AIn0	0..5V	12 bit analogue inputs
17	AIn1	0..5V		**18**	AIn2	0..5V	
19	5V	5V	Output	**20**	GND	GND	

The pins 1 to 8 provide 8 general purpose digital outputs. Maximum current to be pulled out of each output is 20 mA. It is recommended to supply power to E1803D controller via 3pin screw connector but not via USB when this extension board is used.
The pins 9 to 14 are digital general purpose input pins. They are low-active, means to signal a logical "1", they have to be pulled to ground (GND).

The pins 15 to 18 offer three analogue measurement inputs AIn0 .. AIn2 which can be operated in respect to GND_{AIn} and work with a voltage range of 0..5V. This range is converted to a 12 bit digital value in range 1..4095 and can be read out of a connected application or can be used to perform different automated tasks.
PLEASE NOTE: under no circumstances apply a voltage higher than 5V to any of these inputs, this may damage the complete scanner controller card irreversibly!

This extension board makes use of an own MCU (**Mic**rocontroller **U**nit) which handles all digital IOs and also has access to analogue inputs AIn0, AIn1 and AIn2. This MCU can work fully parallel to the main scanner controller card and therefore is suitable for special, customised control applications. To get more information about such applications and to discuss your requirements, please contact HALaser Systems.

12.4 Intelli-IO Interface in motion mode

The hardware described within this section is designed to control motors. Motions caused by these motors may effect a person's health or may otherwise cause damage. Prior to installation and operation compliance with all relevant safety regulations including additional hardware-controlled safety measures has to be secured. The client shall solely be responsible to strictly comply with all applicable and relevant safety regulations regarding installation and operation of the system at any time.

When operated in motion-mode, the extension can be used to drive up to four stepper motor axes via step/direction signals. Here the 20 pin connector provides following signals and inputs:

Lower Row Of Pins	Signal	Voltage	Remarks	Upper Row Of Pins	Signal	Voltage	Remarks
1	Step0	5V	Stepper pulse output signals	**2**	Dir0	5V	Stepper motor direction output signals
3	Step1	5V		**4**	Dir1	5V	
5	Step2	5V		**6**	Dir2	5V	
7	Step3	5V		**8**	Dir3	5V	
9	Ref0	5V	Reference inputs	**10**	Ref1	5V	Reference inputs
11	Ref2	5V		**12**	Ref3	5V	
13	Do not connect!			14	Do not connect!		
15	GND_{AIn}	GND	12 bit analogue inputs	**16**	AIn0	0..5V	12 bit analogue inputs
17	AIn1	0..5V		**18**	AIn2	0..5V	
19	5V	5V		**20**	GND	GND	

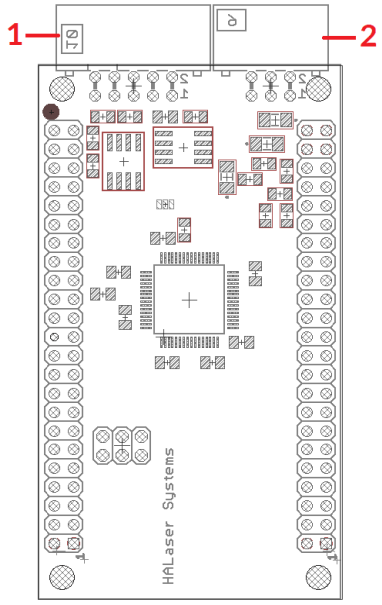
The pins 1 to 8 provide the stepper motor control signals for axes 0..3 (step/direction signals to be used with a separate, external power driver). Maximum current to be pulled out of each output is 20 mA. It is recommended to supply power to E1803D controller via 3pin screw connector but not via USB when the Intelli-IO extension board is used.
The pins 9 to 12 are input pins for axes 0..3 to be used with the reference/homing position. They are low-active, means to signal a “switch was hit”, they have to be pulled to ground (GND).

The pins 15 to 18 offer three analogue measurement inputs AIn0 .. AIn2 which can be operated in respect to GND_{AIn} and work with a voltage range of 0..5V. This range is converted to a 12 bit digital value in range 1..4095 and can be read out of a connected application or can be used to perform different automated tasks.
PLEASE NOTE: under no circumstances apply a voltage higher than 5V to any of these inputs, this may damage the complete scanner controller card irreversibly!

13 NX-02 Extension Board

While the controller already generates scanner signals for all types of parallel scanner protocols (such as XY2-100 or XY3-100), using the NX-02 Extension Board it is possible to drive scanheads that make use of serial scanner protocols (such as NX-02 and compatible). The controller card can be extended by the NX-02 board which utilitises the extension connector as described in section “7 Board And Connectors”.
To operate the controller with the NX-02 Extension Board, firmware version 17 or newer is needed.

13.1 Board Connectors



The E1803D NX-02 Extension Board provides following connectors and interfaces:

1. serial NX-02 protocol connector towards the scanhead
2. Analogue input signal connector

13.2 NX-02 Interface

The 10 pin connector provides the serial scanhead control signals:

Lower Row Of Pins	Signal	Remarks	Upper Row Of Pins	Signal	Remarks
1	DATA-	Connect with input of scanhead	**2**	DATA+	Connect with input of scanhead
3		Do not connect!	**4**	GND	
5			**6**	GND	
7			8	RET+	Connect with backchannel of scanhead
9	RET-	Connect with backchannel of scanhead	10		Do not connect!

Depending on the exact wiring of the used scanhead, it may be possible, an adaptor cable is needed. But for most NX-02 and compatible scanheads, a simple 1:1 connection from this 10-pin IDC connector to a D-SUB9-connector should be suitable (via direct flatbelt-cable connection). For details please refer to the description and pinout of the used scanhead.

13.3 Analogue Input Interface

This connector provides the following inputs:

Lower Row Of Pins	Signal	Voltage	Remarks	Upper Row Of Pins	Signal	Voltage	Remarks
1	AIn0	0..5V	12 bit analogue inputs	**2**	GND_{AIn}	GND	
3	AIn1	0..5V		**3**	GND_{AIn}	GND	
5	AIn2	0..5V		**4**	GND_{AIn}	GND	

The pins 1,3 and offer three analogue measurement inputs AIn0 .. AIn2 which can be operated in respect to GND_{AIn} and work with a voltage range of 0..5V. This range is converted to a 12 bit digital value in range 1..4095 and can be read out of a connected application or can be used to perform different automated tasks.



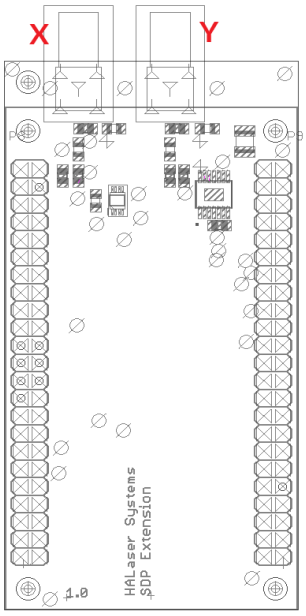
PLEASE NOTE: under no circumstances apply a voltage higher than 5V to any of these inputs, this may damage the complete scanner controller card irreversibly!

14 SDP Extension Board

While the controller already generates scanner signals for all types of parallel scanner protocols (such as XY2-100 or XY3-100), using the SDP Extension Board it is possible to drive scanheads that make use of the serial data protocols known from Newson™ scanners. This is a different interface and protocol than the serial NX-02 as it makes use of an own cable per axis and also provides power supply via the same, single cable. The controller card can be extended by the SDP board which utilitises the extension connector as described in section “7 Board And Connectors”.
To operate the controller with the SDP Extension Board, firmware version 20 or newer is needed.

PLEASE NOTE: to operate the SDP extension board, the E1803D scanner controller card needs to be supplied with 12V power via the screw terminak. Any other voltage may damage the connected SDP galvos! For further details please refer to section “7.3 Power” above!

14.1 Board Connectors



The E1803D SDP Extension Board provides following connectors and interfaces:

X – the screw-connector for the X axis (handles both, data and power supply)
Y – the screw-connector for the X axis (handles both, data and power supply)

PLEASE NOTE: to operate the SDP extension board, the E1803D scanner controller card needs to be supplied with 12V power via the screw terminak. Any other voltage may damage the connected SDP galvos! For further details please refer to section “7.3 Power” above!

15 E1803dock Extension Board

The E1803 dock is a full-size expansion board which resides on the very top of the controller card and acts as some kind of breakout-board. It does not occupy the extension connector but makes use of laser and digital interface – which itself are passed through and are available on top of that E1803dock. This board is available in different variants which are specialised for specific laser types each. When using a suitable E1803dock, connection between E1803D and laser is made simple and easy as only some simple 1:1 connections with standard cables are required. Thus E1803dock allows easy connection of the E1803D controller to lasers without the need to manufacture special cables. Following all available E1803Dock variants are described together with the E1803D signal names and where they can be found at the laser-specific connectors.



PLEASE NOTE: prior to using the E1803dock, please ensure the pinout of the connectors (as described below) really fit to your specific variant of your used laser. Elsewhere irreversible damage are possible at laser and/or scanner controller card.



PLEASE NOTE: when using the pass-through laser and/or digital interfaces in parallel to the laser-specific connectors, you need to ensure they are not used twice, out of both connectors. This may result in an undefined and unwanted behaviour otherwise, as they work parallel and can't be switched separately. When a parallel usage is intended, please double-check the current consumption of connected devices as the total current that can be provided by these lines does not double but is the same total value for two connections then. Exceeding the total maximum allowed current of a E1803D-signal may cause an irreversible damage to the controller otherwise.

15.1 E1803dock MOPA

This variant of the E1803dock can be used with most MOPA/fiber lasers that come with a LP8 digital interface for setting the power, such as

- IPG YLP with interface variant type G (without SPI serial interface and pin 12 alarm signal)
- IPG YLP with interface variant type E (without APD indexing)
- IPG YLP with interface variant type D, D1, B, B1
- MaxPhotonics MFP
- JPT YDFLP
- Raycus RFL-PMX
- Raycus RFL-PQB
- GZTech YFPN series

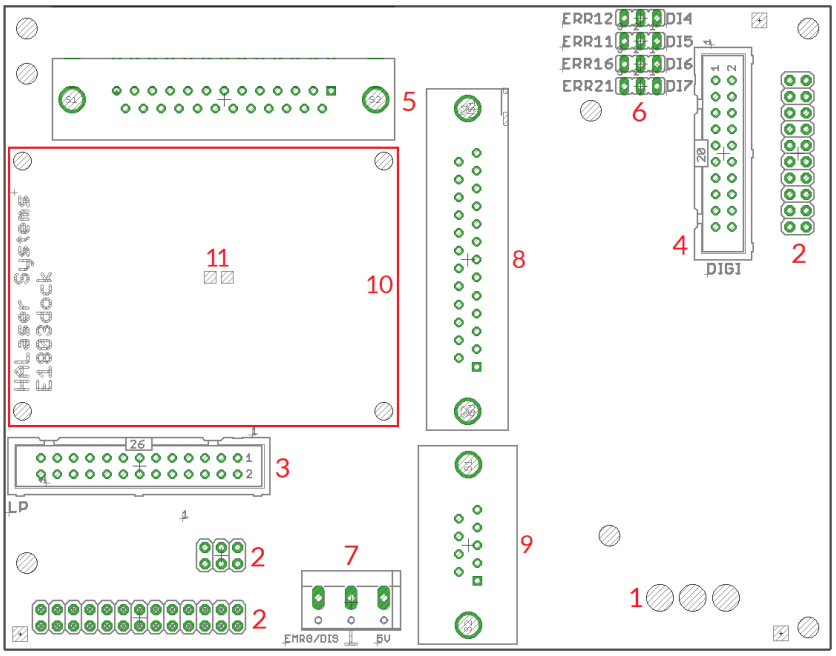
  … and compatible laser types.

Starting with hardware version 1.2, the E1803dock MOPA also supports laser types that expect an analogue signal for power control, such as

- IPG YLM
- Raycus RFL-QCW

  ...and compatible types.

Connection between E1803dock and laser is done mainly via a 1:1 D-SUB25 connection:

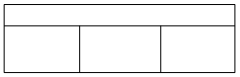


1. Screwdriver holes for access to the screw-terminal of the underlying E1803D
2. dock-connectors to underlying E1803D which connect with digital, laser and serial interface
3. Laser interface pass-through, for pinout please refer to section “7.7 Laser Signals”
4. Digital interface pass-through, for pinout please refer to section “7.8 Digital Interface” and to jumper description below
5. D-SUB25 connector for direct connection to laser. A connection can be established using a standard 1:1 D-SUB-cable. This connector provides the following pinout:

Pin	Description	Pin	Description
1	LP8_0 laser power signal, CMOS, max 8mA	14	GND
2	LP8_1 laser power signal, CMOS, max 8mA	15	Not connected
3	LP8_2 laser power signal, CMOS, max 8mA	16	DIn6 laser alarm pin 16, available at DIn5 only when jumper is set to „ERR16“ (refer to description below)
4	LP8_3 laser power signal, CMOS, max 8mA	17	Not connected
5	LP8_4 laser power signal, CMOS, max 8mA	18	MO main oscillator signal, CMOS, max 8mA
6	LP8_5 laser power signal, CMOS, max 8mA	19	LaserGate power amplifier signal, CMOS, max 14mA
7	LP8_6 laser power signal, CMOS, max 8mA	20	LaserA 5V pulse repetitive signal, CMOS, max 14mA
8	LP8_7 laser power signal, CMOS, max 8mA	21	DIn7 laser alarm pin 21, available at DIn7 only when jumper is set to „ERR21“ (refer to description below)
9	LP8 Latch laser power latch, CMOS, max 8mA	22	LaserB used to switch the pilot laser, CMOS, max 14mA
10	GND	23	Emergency input, connected to screw connector (7) and has to be pulled to HIGH for different laser types to enable operation
11	DIn5 laser alarm pin 11, available at DIn5 only when jumper is set to „ERR11“ (refer to description below)	24	Not connected
12	Not connected	25	Not connected
13	Not connected		

6. Jumper to configure laser alarm lines 12 (hardware version 1.2 and newer), 11, 16 and 21. When they are set to “ERRxx” position (jumper set to left), the related error signal is connected with DInx input as described above.
   When at least one of the jumpers is set to “ERRxx”-position, the digital interface needs to be configured to opto-mode 1 or opto-mode 2 (please refer to description in section “7.8.2 Opto-Configuration Jumpers” above). In this case the related digital input is no longer available.
   For jumpers that are set to position “DInx” (jumper set to right), the related digital input is available but the laser error can no longer be read.

7. Emergency input screw connector:



   EMRG GND 5V

   “EMRG” is connected to pin 23 of the MOPA D-SUB connector (5) and can be used to provide a HIGH-signal to the laser in order to enable it. Using this an emergency enable/off function can be implemented.
   “5V” is an output which provides 5V from the controller card. It can be used e.g. to apply a HIGH-signal to the EMRG-input, possibly via a relay contact. Please note: The “5V” pin of this screw connector is not allowed to be used for supplying power to any other peripherals.

8. D-SUB25 connector for direct connection to laser (available only in hardware variant 1.2 or newer). A connection can be established using a standard 1:1 D-SUB-cable. This connector provides the following pinout:

Pin	Description	Pin	Description
1	Not connected	14	Shield
2	Not connected	15	Not connected
3	Not connected	16	DIn6 laser alarm pin 16, available at DIn5 only when jumper is set to „ERR16“ (refer to description below)
4	Not connected	17	LaserGate power amplifier signal, CMOS, max 14mA
5	Not connected	18	Not connected
6	Not connected	19	DIn7 laser ready pin, available at DIn7 only when jumper is set to „RDY“ (refer to description below)
7	MO main oscillator signal, CMOS, max 8mA	20	Not connected
8	AOut0 0..4V analogue power control, max 15 mA; Please note: it has to be ensured on software side to not to apply more than 4V here. This is done in BeamConstruct automatically when laser type „IPG (MO, AOut0)“ is configured. Out of own applications when calling `E180X_ana_write()` a maximum value `a` of 26213 is allowed.	21	LaserB used to switch the pilot laser, CMOS, max 14mA
9	GND	22	Not connected
10	Not connected	23	Not connected
11	Not connected	24	Not connected
12	Not connected	25	Not connected
13	Not connected		

9. RS232 interface (available only in hardware variant 1.2 or newer), this is a pass-through of the RS232-signal of the on-board serial interface as described at “7.9 Serial Interface” which can be used for communication with the laser. The D-SUB9 connector provides the following pinout:

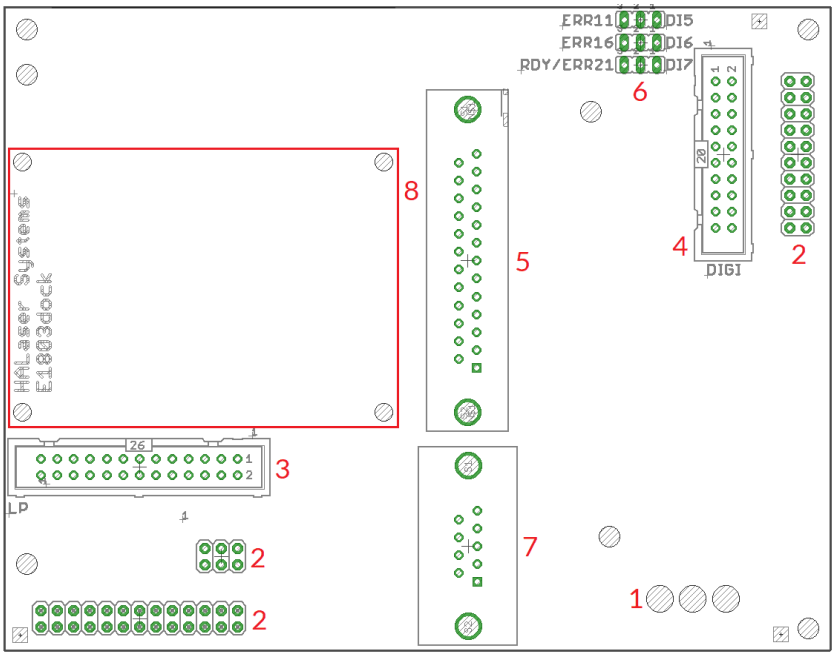
Pin	Description	Pin	Description
1	Not connected	6	Not connected
2	RX (to be connected with TX on laser side)	7	Not connected
3	TX (to be connected with RX on laser side)	8	Not connected
4	Not connected	9	Not connected
5	GND		

10. mounting holes for HALdrive XY3-100 to analogue converter, can be used for placing a HALdrive converter board
11. Pilot laser selection – starting with the hardware revision 1.8 of the E1803D scanner controller card and firmware version 19, the controller card provides an own, native output for controlling the pilot laser. So with these (or newer hard/software) versions, it is no longer necessary to use LaserB output for switching the pilot laser. The solder jumper (11) can be used to re-enable LaserB for use as pilot laser control pin.
    Following logic applies:
    - when the solder jumper is closed, LaserB or the pilot laser pin can be used for switching the pilot laser, but LaserB can’t be used for anything else except setting of the pulse width via serial protocol on LaserB, LP8_1 and LP8_2
    - when the solder jumper is open, LaserB can’t be used for switching the pilot laser but it can be used for any other operations such as FPK, PWM, …,but only via a separate IDC connection; when opened, it is completely disconnected from the laser interface connector

15.2 E1803dock YLM

This variant of the E1803dock can be used with analogue controlled MOPA/fiber lasers such as IPG YLM, Raycus RFL-QCW and compatible types. Starting with hardware version 1.2, the functionalities of the E1803dock YLM are contained in “15.1 E1803dock MOPA“ completely. **Thus the separate variant “YLM” is discontinued and no longer available.**

Connection between E1803dock and laser is done mainly via a 1:1 D-SUB25 connection (laser control signals) and a 1:1 D-SUB9 connection (RS232 communication):



1. Screwdriver holes for access to the screw-terminal of the underlying E1803D
2. dock-connectors to underlying E1803D which connect with digital, laser and serial interface
3. Laser interface pass-through, for pinout please refer to section “7.7 Laser Signals”
4. Digital interface pass-through, for pinout please refer to section “7.8 Digital Interface” and to jumper description below
5. D-SUB25 connector for direct connection to laser. A connection can be established using a standard 1:1 D-SUB-cable. This connector provides the following pinout:

Pin	Description	Pin	Description
1	Not connected	14	Shield
2	Not connected	15	Not connected
3	Not connected	16	DIn6 laser alarm pin 16, available at DIn5 only when jumper is set to „ERR16“ (refer to description below)
4	Not connected	17	LaserGate power amplifier signal, CMOS, max 14mA
5	Not connected	18	Not connected
6	Not connected	19	DIn7 laser ready pin, available at DIn7 only when jumper is set to „RDY“ (refer to description below)
7	MO main oscillator signal, CMOS, max 8mA	20	Not connected
8	AOut0 0..4V analogue power control, max 15 mA; Please note: it has to be ensured on software side to not to apply more than 4V here. This is done in BeamConstruct automatically when laser type „IPG (MO, AOut0)“ is configured. Out of own applications when calling `E180X_ana_write()` a maximum value `a` of 26213 is allowed.	21	LaserB used to switch the pilot laser, CMOS, max 14mA
9	GND	22	Not connected
10	Not connected	23	Not connected
11	Not connected	24	Not connected
12	Not connected	25	Not connected
13	Not connected		

6. Jumper to configure laser alarm and ready lines.
   The upper jumper is not used and always has to be set to “DI5”.
   When the middle jumper is set to “ERR16” position (jumper set to left), the laser error signal from pin 16 is connected with DIn6 input as described above.
   When the lower jumper is set to “RDY” position (jumper set to left), the laser ready signal from pin 19 is connected with DIn6 input as described above.
   When at least one of the jumpers is NOT set to “DIx” (right position), the digital interface needs to be configured to opto-mode 1 or opto-mode 2 (please refer to description in section “7.8.2 Opto-Configuration Jumpers” above). In this case the related digital input is no longer available.
   For jumpers that are set to position “DIx” (jumper set to right), the related digital input is available but the laser error/ready can no longer be read.

7. RS232 interface, this is a pass-through of the RS232-signal of the on-board serial interface as described at “7.9 Serial Interface” which is used for communication with the laser. The D-SUB9 connector provides the following pinout:

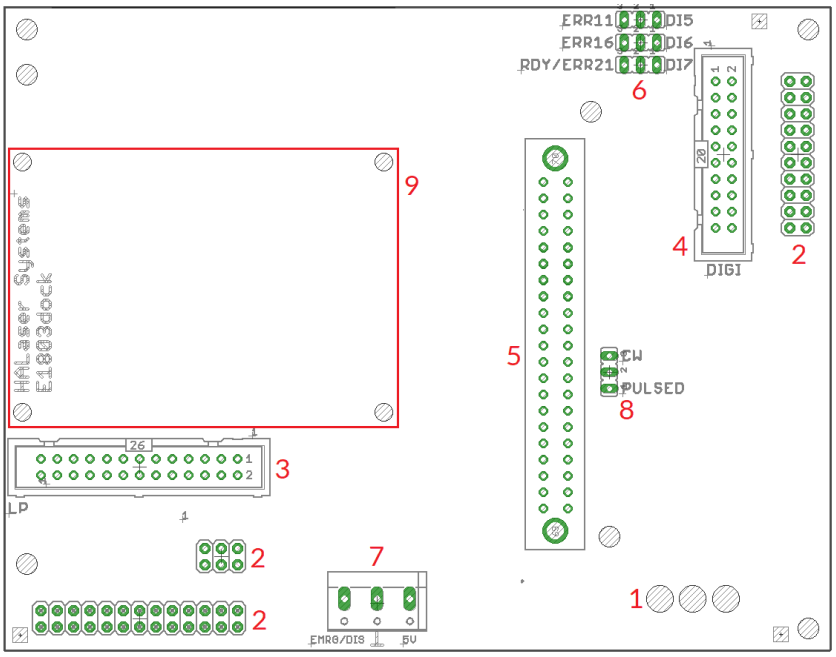
Pin	Description	Pin	Description
1	GND	6	Not connected
2	Not connected	7	Not connected
3	RX (to be connected with TX on laser side)	8	Not connected
4	TX (to be connected with RX on laser side)	9	Not connected
5	Not connected		

8. mounting holes for HALdrive XY3-100 to analogue converter, can be used for placing a HALdrive converter board

15.3 E1803dock SPI

This variant of the E1803dock can be used with SPI G4 fiber laser / TRUMPF™ TruPulse nano™ and compatible types that also make use of a compatible connector.
Starting with hardware revision 1.4, the pinout and functionality of this dock is available in E1803dock NCC (as described below). **Thus the separate variant "SPI" is discontinued and no longer available.**

Connection between E1803dock and laser is done mainly via a 1:1 68-pin SCSI-type connection:



1. Screwdriver holes for access to the screw-terminal of the underlying E1803D

2. dock-connectors to underlying E1803D which connect with digital, laser and serial interface

3. Laser interface pass-through, for pinout please refer to section "7.7 Laser Signals"

4. Digital interface pass-through, for pinout please refer to section "7.8 Digital Interface" and to jumper description below

5. 68pin Mini D Ribbon SCSI connector for direct connection to laser. A connection can be established using a standard 1:1 cable. This connector provides the following pinout:

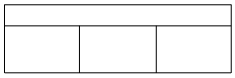
Pin	Description	Pin	Description
1	Not connected	35	Not connected
2	Not connected	36	GND
3	Not connected	37	GND
4	Not connected	38	Not connected
5	LaserGate power amplifier signal, CMOS, max 14mA	39	GND
6	LaserB used to switch the pilot laser, CMOS, max 14mA	40	GND
7	MO laser global enable signal, CMOS, max 8mA	41	GND
8	DIn6 laser temperature alarm pin 16, available at DIn5 only when jumper is set to „ERR16" (6, refer to description below)	42	GND

9	DIn7 laser general alarm pin, available at DIn7 only when jumper is set to „ERR21“ (6, refer to description below)	43	GND
10	DIn5 laser system fault pin 11, available at DIn5 only when jumper is set to „ERR11“ (6, refer to description below)	44	GND
11	Not connected	45	GND
12	Not connected	46	GND
13	LaserA PWM signal	47	GND
14	Not connected	48	GND
15	Disable input, connected to screw connector (7) and has to be pulled to LOW to enable operation	49	GND
16	Not connected	50	GND
17	LP8_0 laser power signal, CMOS, max 8mA	51	LP8_4 laser power signal, CMOS, max 8mA
18	LP8_1 laser power signal, CMOS, max 8mA	52	LP8_5 laser power signal, CMOS, max 8mA
19	LP8_2 laser power signal, CMOS, max 8mA	53	LP8_6 laser power signal, CMOS, max 8mA
20	LP8_3 laser power signal, CMOS, max 8mA	54	LP8_7 laser power signal, CMOS, max 8mA
21	Pulse/CW operation mode, can be set to LOW or HIGH signal via jumpers (8, refer to description below)	55	GND
22	Not connected	56	GND
23	LP8 Latch laser power latch, CMOS, max 8mA	57	connected
24	Not connected	58	GND
25	RS232 RX	59	GND
26	RS232 TX	60	GND
27	Not connected	61	Not connected
28	GND	62	Not connected
29	GND	63	Not connected
30	GND	64	AOut1 0..10V analogue simmer control, max 15 mA
31	GND	65	AOut0 0..10V analogue power control, max 15 mA
32	GND	66	Not connected
33	Not connected	67	Not connected
34	Not connected	68	Not connected

For details about the connector type used, here, please refer to the next section below

6. Jumper to configure laser alarms. When they are set to “ERRxx” position (jumper set to left), the related alarm/failure signal is connected with DInx input as described above.
   When at least one of the jumpers is set to “ERRxx”-position, the digital interface needs to be configured to opto-mode 1 or opto-mode 2 (please refer to description in section “7.8.2 Opto-Configuration Jumpers” above). In this case the related digital input is no longer available.
   For jumpers that are set to position “DInx” (jumper set to right), the related digital input is available but the laser error can no longer be read.

7. Disable input screw connector:



Here “DIS” is connected to pin 15 of the 68-pin connector and can be used to provide a HIGH-signal to

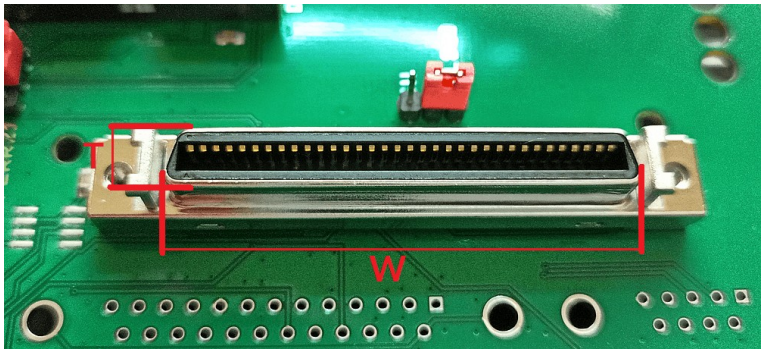
the laser in order to disable it. When the disable functionality is implemented an other way, it is also possible to directly connect the “GND” pin to “DIS”, then the laser is always enabled.
Please note. The “5V” pin of this screw connector can be used only for applying a disable-signal to “DIS” signal, it is not allowed to be used for supplying power to any other peripherals.

8. Jumper to configure pulsed/CW mode. When the jumper is set to “Pulsed” (plugged to lower position), pin 21 of the 68-pin laser connector is set to LOW and the pulses can be controlled via LaserA (PWM+/PWM-) signal. When set to “CW” (plugged to upper position), pin 21 is set to HIGH and the laser operates in continuous wave mode.

9. mounting holes for HALdrive XY3-100 to analogue converter, can be used for placing a HALdrive converter board

15.3.1 Connector to laser

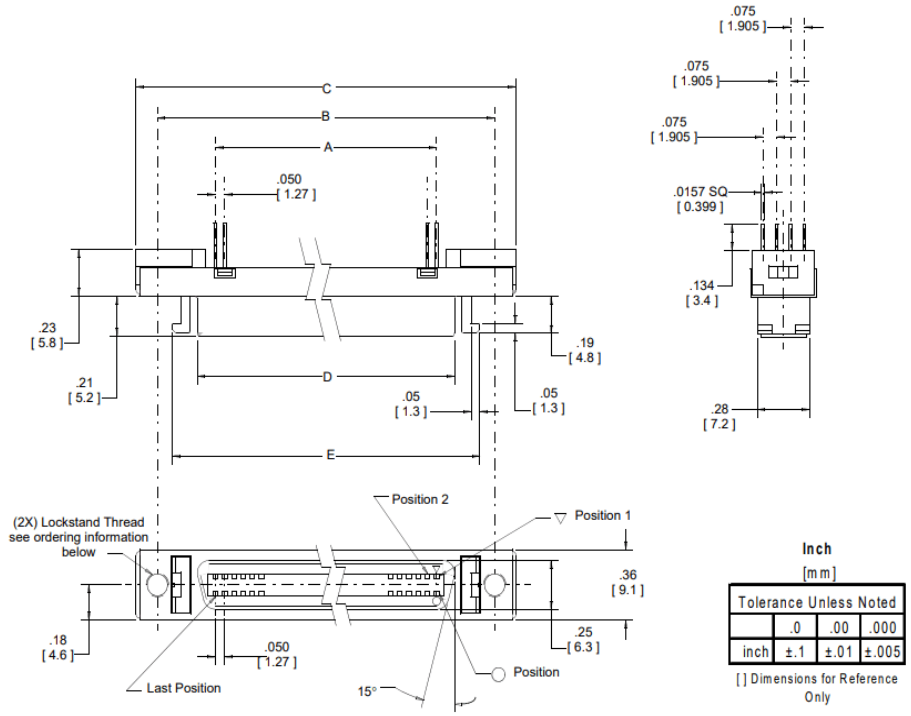
By default, SPI/Trumpf lasers are shipped with a cable which can be used to connect to the E1803dock SPI directly. However, it seems there are variants of this cable out there, which do not fit to the standard 68 pin SCSI connector of the E1803dock:



W=46,9 mm, T=6,3 mm

In this case a separate cable is needed which fits to both, the E1803dock SPI and the laser. For the specification of the connector on laser side, please refer to the manual of the laser itself.
On the E1803dock SPI a connector of type “10268-6212PC“ from 3M Electronics Solutions Division (or a compatible type) is used:

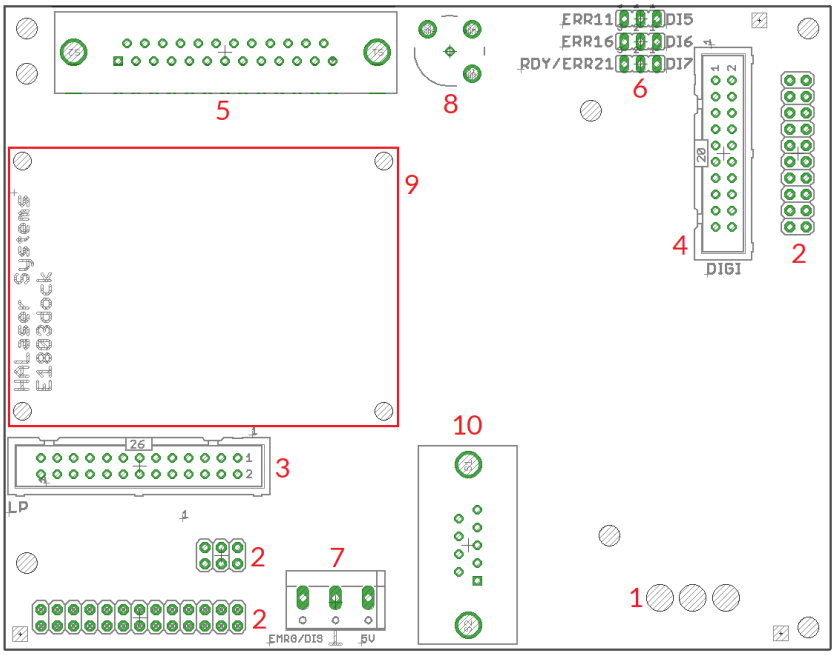
Contact Quantity	Dim ensions				
	A	B	C	D	E
68	1.650 [41.91]	2.28 [57.9]	2.51 [63.8]	1.85 [46.9]	2.12 [53.8]



Excerpt from the official datasheet of the connector,
for latest versions please contact 3M Electronics Solutions

15.4 E1803dock CO_2/YAG

This variant of the E1803dock can be used with all YAG and CO2 lasers which come with a compatible interface. For such lasers connection between E1803dock and laser is done mainly via a 1:1 D-SUB25 or BNC connection:



1. Screwdriver holes for access to the screw-terminal of the underlying E1803D
2. dock-connectors to underlying E1803D which connect with digital, laser and serial interface

3. Laser interface pass-through, for pinout please refer to section “7.7 Laser Signals”

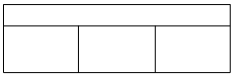
4. Digital interface pass-through, for pinout please refer to section “7.8 Digital Interface” and to jumper description below

5. D-SUB25 connector for direct connection to laser. A connection can be established using a standard 1:1 D-SUB-cable. This connector provides the following pinout:

Pin	Description	Pin	Description
1	Not connected	14	Not connected
2	Not connected	15	Not connected
3	GND	16	Not connected
4	Emergency input, connected to screw connector (7) and has to be pulled to HIGH for different laser types to enable operation	17	GND
5	GND	18	GND
6	GND	19	GND
7	Not connected	20	LaserB first pulse killer signal (FPK/QKILL for YAG) or pilot laser (for CO_2), CMOS, max 14mA
8	GND	21	AOut1 power control, analogue signal in range 0..10V, max 15mA
9	AOut0 frequency control, analogue signal in range 0..5V, max 15mA; this signal is hardware-divided by 2 comparing to the original AOut0	22	LaserA 5V output, CMOS, max 14mA
10	GND	23	MO used to switch the pilot laser, CMOS, max 8mA
11	GND	24	LaserGate non-inverted/high-active laser-on signal, CMOS, max 14mA
12	LaserGate inverted/low-active laser-on signal, CMOS, max 40mA	25	LaserA negative output of differential PWM signal (PWM-, also connected to shield of BNC-connector), +-5V, max 40mA
13	LaserA positive output of differential PWM signal (PWM+, also connected to core of BNC-connector), +-5V, max 40mA		

6. Jumpers are reserved for future use and all have to be set to “DIx” (right position)

7. Emergency input screw connector:



Here “EMRG” is connected to pin 23 of the D-SUB connector and can be used to provide a HIGH-signal to the laser in order to enable it. When emergency switch functionality is implemented an other way, it is also possible to directly connect the “5V” pin to “EMRG”.
“5V” is an output which provides 5V from the controller card. It can be used e.g. to apply a HIGH-signal to the EMRG-input, possibly via a relay contact. Please note. The “5V” pin of this screw connector is not allowed to be used for supplying power to any other peripherals.

8. BNC-connector for PWM/tickle-signal. It provides a differential +-5V LaserA signal which is also available as PWM+/PWM- at the D-SUB25 connector (pins 13 and 25)

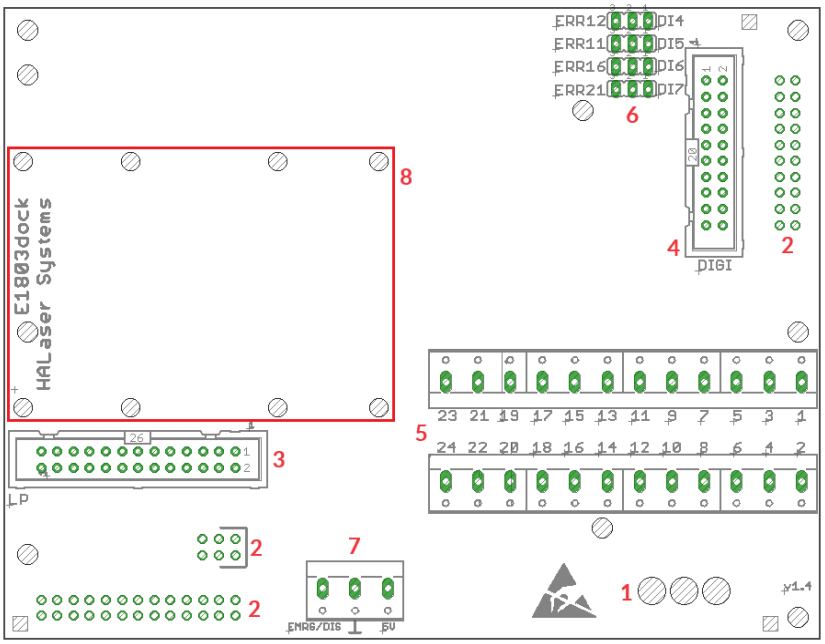
9. mounting holes for HALdrive XY3-100 to analogue converter, can be used for placing a HALdrive converter board

10. RS232 interface, this is a pass-through of the RS232-signal of the on-board serial interface as described at “7.9 Serial Interface”. The D-SUB9 connector provides the following pinout:

Pin	Description	Pin	Description
1	GND	6	Not connected
2	Not connected	7	Not connected
3	RX (to be connected with TX on laser side)	8	Not connected
4	TX (to be connected with RX on laser side)	9	Not connected
5	Not connected		

15.5 E1803dock NCC

While the majority of MOPA-lasers and many CO_2- or YAG-lasers come with a standard pin-out which allows to have a simple 1:1-connection, there are still different lasers available, which all have their own, non-common connectors and individual pin-outs. This is where the E1803dock NCC (**n**on-**c**ommon **c**onnector) can be used for. It works with all laser models that have not a common interface as described in sections about " E1803dock MOPA" and " E1803dock CO2/YAG" above. This is true e.g. for lasers such as SPI™ / Trumpf™ truePulse nano™, DAVI and others more. This E1803dock provides a screw-terminal where the laser-specific cable can be connected easily according to the signals and requirements of the laser:



1. Screwdriver holes for access to the screw-terminal of the underlying E1803D

2. dock-connectors to underlying E1803D which connect with digital, laser and serial interface

3. Laser interface pass-through, for pinout please refer to section "7.7 Laser Signals"

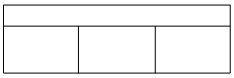
4. Digital interface pass-through, for pinout please refer to section "7.8 Digital Interface" and to jumper description below

5. Screw-terminal for connection of the laser-specific cable, here following signals can be found and need to be connected according to the requirements of the laser:

Pin	Description	Pin	Description
1	LaserGate, CMOS, max 14 mA	2	5V
3	PilotLaser, used to switch the pilot laser, CMOS, max 14 mA	4	AOut0 0..10 V analogue outout signal, max 8 mA
5	LaserB, FPK/second frequency, CMOS, max 14 mA	6	AOut1 0..10 V analogue outout signal, max 8 mA
7	MO, laser global enable signal, CMOS, max 14 mA	8	GND
9	DIn6, alarm pin 16, available only when jumper is set to “ERR16” (connector 6, refer to description below)	10	RS232 TX
11	DIn7, alarm pin 21, available only when jumper is set to “ERR21” (connector 6, refer to description below)	12	RS232 RX
13	DIn5, alarm pin 11, available only when jumper is set to “ERR11” (connector 6, refer to description below)	14	LP8 Latch laser power latch bit, CMOS, max 14 mA
15	LaserA PWM signal, CMOS, max 14 mA	16	LP8_7 laser power bit, CMOS, max 14 mA
17	EMRG emergency/enable input, connected to screw connector (7)	18	LP8_6 laser power bit, CMOS, max 14 mA
19	LP8_0 laser power bit, CMOS, max 14 mA	20	LP8_5 laser power bit, CMOS, max 14 mA
21	LP8_1 laser power bit, CMOS, max 14 mA	22	LP8_4 laser power bit, CMOS, max 14 mA
23	LP8_2 laser power bit, CMOS, max 14 mA	24	LP8_3 laser power bit, CMOS, max 14 mA

6. Jumper to configure laser alarms. When they are set to “ERRxx” position (jumper set to left), the related alarm/failure signal is routed to the screw-terminal and connected with DInx input as described above.
   When at least one of these jumpers is set to “ERRxx”-position, the digital interface needs to be configured to opto-mode 1 or opto-mode 2 (please refer to description in section “7.8.2 Opto-Configuration Jumpers” above). In this case the related digital input is no longer available.
   For jumpers that are set to position “DInx” (jumper set to right), the related digital input is available but the laser error can no longer be read.
   Jumper ERR12 is reserved for future use and has to be set to “DI4” (right position), setting it to the left position disables the related digital input.

7. Emergency input screw connector:

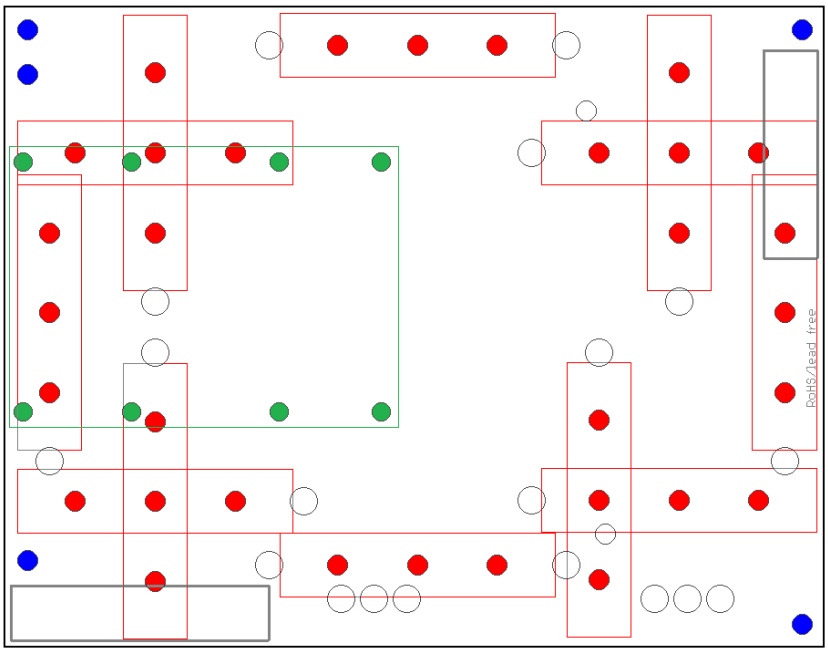


   Here “EMRG” is connected to pin 17 of the screw-terminal (4) and can be used to provide a HIGH-signal to the laser in order to enable it. When the emergency switch functionality is implemented an other way, it is also possible to directly connect the “5V” pin to “EMRG”.
   “5V” is an output which provides 5V from the controller card. It can be used e.g. to apply a HIGH-signal to the EMRG-input, possibly via a relay contact. Please note. The “5V” pin of this screw connector is not allowed to be used for supplying power to any other peripherals.

8. mounting holes for HALdrive XY3-100 to analogue converter, can be used for placing a HALdrive converter board

16 E1803base Mounting Kit

The E1803base extension is a mounting help for easy installation on DIN rails/C45 rails and other possibilities of mechanical integration into machines:



RED – mounting positions for DIN/C45 rail locks/DIN/C45 rail adapters (bottom side). Here pairs of locks can be mounted in one of 6 possible positions and one of two possible orientations (horizontal or vertical). Here locks of type Phoenix Contact 1201578 or similar can be used. With these locks the board then can be clamped on a DIN rail.

BLUE – mounting holes for the E1803D scanner controller card on top of the E1803base. Here hex-stands/distance bolts can be screwed in where the controller card is mounted on top.

GREEN – mounting holes for HALdrive XY3-100 to analogue converter, for placing a HALdrive converter board when E1803base is used as top-cover for E1803D

Mounting procedure for E1803base:

1. Identify suitable positions (RED) for two DIN/C45 rail locks and mount them on bottom side (two or three screws from top side into the lock on bottom)
2. Mount hex-stands or distance bolts in at least four of the given mounting holes (BLUE).
3. Mount E1803D on top of these hex-stands/distance bolts
4. Clamp the board on your DIN/C45 rail

Optionally and without the DIN/C45 rail clamps mounted, E1803base also can be used as top-cover for the E1803D controller. There the same (BLUE) holes can be used to mount the E1803base on top using hex-stands/distance bolts.

17 Quick Start into E1803D

Following a few steps are described that give users the possibility to quick start into usage of E1803D scanner controller. It makes use of BeamConstruct and USB connection. For this quick start manual it is assumed correct wiring of the controller is already done according to the description above. For more detailed information about BeamConstruct usage please also refer to quick start manual from https://.systems/download/manual_quickstart.pdf and to full user manual which is available at https://halaser.systems/download/manual.pdf.

To start with E1803D controller:

1. **SECURITY CHECK:** The following steps describe how to set up E1803D scanner controller card and how to control laser equipment with them. Thus all laser safety rules and regulations need to be respected, all required technical security mechanisms need to be available and active prior to starting with it.
2. Install latest software version from https://halaser.systems/download.php – for Windows this package contains all required drivers, for Linux no separate drivers are needed.
3. Connect E1803D controller via USB and apply +-12V .. +-24V power (depending on scanhead, as described above).
4. Now the Alive/Error-LED should light up and then start blinking after some time. When this does not happen, please turn power off, check if the microSD-card is placed correctly and then try again.
5. Evaluate the serial interface the controller is connected with – for Windows the Device Manager (can be found in Control Panel) will list a new COM-port (e.g. “COM3”); for Linux type “dmesg” in console to find out to which interface it was connected with (typically “/dev/ttyACM0”).
6. Start BeamConstruct laser marking software.
7. Go to menu “Project” → “Project Settings...”, then tab-pane “Scanner”.
8. Now you can select “E1803D” as scanner controller card. If is is not preselected, please choose the appropriate scanner controller in the related combo box.
9. Press the “Configure”-button to get into the settings dialogue for E1803D plug-in.
10. Enter the serial interface name in field “IP/Interface” (e.g. “COM3” or “/dev/ttyACM0”).
11. Leave everything with “OK”.
12. Draw some geometries as described in “BeamConstruct Quick Start Manual”.
13. **SECURITY CHECK:** Next the scanner controller card will be accessed for the first time. That means it is opened and initialised and all connected equipment may start working now. Thus it is very important to ensure all security regulations are met and nobody can be injured and no damage can be caused also in case laser output or other motion starts spontaneously and unexpectedly!
14. Press “F2” or go to menu “Process” → “Mark” to open the mark dialogue.
15. Start marking by pressing the yellow laser-button!

18 Command Interface

When E1803D scanner card is connected via USB and the USB-connection is NOT used for transmitting marking information, it can be used to send control commands to the card. Some of them are independent of the current operating mode and some of them can be used only in case the controller is operating in stand-alone mode.
Alternatively control commands can also be sent via Telnet using Ethernet connection. Here a Telnet-client has to connect to port 23 using the IP of the scanner controller. This Telnet client should work in passive mode.
So when E1803D scanner card is connected this way via Ethernet and the Ethernet-connection is NOT used for transmitting marking information, it can be used to send control commands to the card. Some of them are independent of the current operating mode and some of them can be used only in case the controller is operating in stand-alone mode.
Such a control command always consists of ASCII-text. An appropriate client has to connect to the serial port (COMx for Windows and /dev/ttyACMx for Linux where “x” is a number identifying the specific serial interface or TCP/IP port 23). As soon as the connection is established, commands can be sent to the card. All commands come with following structure:

```
cxxxx [parameter(s)]
```

The commands always start with character “c”. Next four characters identify the command itself. Depending on the command one or more optional or mandatory parameters may follow. The end of the command always has to be marked with a CR/LF (aka “enter”) and returns with an "OK" or with an error.

When communicating via RS485 serial interface in bus-mode (means when several controllers are connected via the same RS485-line, when `u0bypass` is set to 1 and when `busid` is set), every c-command has to be preceded by fixed three-bytes in format “bXX” where “XX” is a value in range 00..99 equal to the bus-identifier set with configuration parameter `busid`. So in busmode the structure of a d-command is
`bXXcxxxx [parameter(s)]`

18.1 General Commands

The following commands can be used in all scenarios, they do not depend on a specific operation mode of the card. Nevertheless it is recommended to not to send a command during card is marking to not to influence marking operation.

`cvers`

"**vers**ion" – return version information of controller card. This command returns a version string specifying version of hard- and firmware in style **`vFF-H`** where “**`FF`**” is the version of the firmware and “**`H`**” specifies the hadware revision of the controller.

`cecho <0/1>`

"**echo**" – when typing commands in a serial console communicating with the controller, all the typed characters are echoed, means they are sent back to the host so that a user can see what is typed. This is an unwanted behaviour when some kind of control software communicates with this interface. Using this command the serial echo mode can be turned off (parameter 0, only return values are sent back) or on (parameter 1, all data are sent back). When called with no parameters, the current echo mode value is returned.
Example: `cecho 0` – turn off echo mode

`cginp`

"**g**et **inp**ut" – get the current state of the digital inputs. The input state is returned as a decimal number representing the bitpattern at the inputs. So when e.g. a value "15" is returned, this means the lower four inputs of the digital interface are set to HIGH while the upper ones are at LOW level

`cgbsr`

"**g**et **b**oard **se**rial number" – returns the serial number of the card. This number is a unique, internal value that is used e.g. to identify a controller on host PC when more than one scanner card is used.

`cjsor <factor>`

"**j**ump **s**peed **o**verw**r**ite" - this command modifies the actual jump speed by using the given factor (in unit 1/100%). All operations make use of the changed jump speed until a factor of 10000 is set or until the controller is restarted. This is true for both, stand-alone applications where an .EPR-file is loaded from microSD-card and for host-controlled marking operations (via libe1803 or BeamConstruct).

`cmsor <factor>`

"**m**ark **s**peed **o**verw**r**ite" - this command modifies the actual mark speed by using the given factor (in unit 1/100%). All operations make use of the changed mark speed until a factor of 10000 is set or until the controller is restarted. This is true for both, stand-alone applications where an .EPR-file is loaded from microSD-card and for host-controlled marking operations (via libe1803 or BeamConstruct).

`cpwor <factor>`

"**p**o**w**er **o**verw**r**ite" - this command modifies the actual power by using the given factor (in unit 1/100). All operations make use of the changed power until a factor of 10000 is set or until the controller is restarted. This is true for both, stand-alone applications where an .EPR-file is loaded from microSD-card and for host-controlled marking operations (via libe1803 or BeamConstruct).
This command influences following methods of setting laser power:

- pulse width, here user has to ensure the resulting pulse width is smaller than the period of the related frequency, elsewhere the output will be a continuous signal
- LP8 laser port
- AOut0 and AOut1 analogue outputs

`cfror <factor>`

"**fr**equency **o**verw**r**ite" - this command modifies the actual frequency by using the given factor (in unit 1/100%). All operations make use of the changed power until a factor of 10000 is set or until the controller is restarted. This is true for both, stand-alone applications where an .EPR-file is loaded from microSD-card and for host-controlled marking operations (via libe1803 or BeamConstruct).
This command is not available for lasermodes CO_2 or YAG
This function requires firmware version 16 or newer.

`cpuor <factor>`

"**pu**lse-width **o**verw**r**ite" - this command modifies the actual pulse-width by using the given factor (in unit 1/100%). All operations make use of the changed pulse-width until a factor of 10000 is set or until the controller is restarted. This is true for both, stand-alone applications where an .EPR-file is loaded from microSD-card and for host-controlled marking operations (via libe1803 or BeamConstruct).
This command is not available for lasermodes CO_2 or YAG, there the pulse-width is changed via command `cpwor`.
This function requires firmware version 16 or newer.

`cgmtx`

"**g**et **m**atri**x**" - return the four elements of the 2x2 output matrix. The members of the matrix are returned as four integers with a factor of 1000. So returned values of "1200 0 0 1200" are equal to a matrix

1,2	0,0
0,0	1,2

Which itself defines a scale factor of 1,2 in both, X and Y direction of the output.
This function requires firmware version 11 or newer.

`csmtx <m11 m12 m21 m22>`

"**s**et **m**a**t**ri**x**" - set a 2x2 matrix which is used for the global output of the scanner card. This means, the values applied here influence the whole project which is currently be marked. Such a 2x2 matrix can be used to modify the X- and Y-scale, the rotation and the X- and Y-slant of the output. The matrix members m11, m12, m21 and m22 have to be given as integers which represent the matrix elements multiplied by 1000.
As soon as a custom matrix is set via this command, all matrix values out of a loaded EPR file are ignored and only the matrix-values currently set are used. This includes gain and rotation correction settings which may be set in such an EPR file.
Example: `csmtx 996 -87 87 996` - rotate the output by 5 degrees; these values are generated out of the unity matrix {1000, 0, 0, 1000} which was multiplied with the rotation matrix {cos(5)*1000, -sin(5)*1000, sin(5)*1000, cos(5)*1000}
This function requires firmware version 11 or newer.

`cswaf <factor>`

Sets a factor in unit bits per mm that is used with all commands that specify some distance or position (such as `cspof`). When this factor is set to a proper value, the given positions/distances can be specified in unit um instead of bits. When a factor of 1000 is specified, the functionality is reset back to the default behaviour where position values have to be given in unit bits.
This parameter requires a firmware version 18 or newer.

`cspof <x y z>`

"**s**et **p**osition **of**fset" – sets a n position offset for the complete output. This function expects three parameters for the offset to be set in X, Y and Z-direction. The values to be given here are signed 26 bit, means they need to be in range -33554431..33554432. To reset the offset for one or more directions, a value of 0 has to be set.
Please note: this offset is set prior to the matrix calculations which may be done when a custom output matrix is set using command `csmtx`. Resulting from that, the matrix calculation also applies to the offset defined here. So when the matrix e.g. defines a scale factor for the output, the offset values specified here are scaled by the same factor.
Example: `cspof -16777216 0 0` – shift the output by a quarter of the whole available working area to the left.
This function requires firmware version 16 or newer.

`cgana`

get **ana**logue input values – this command returns the values which have been read at analogue inputs AIn0..AIn2 recently. Reading of the analogue inputs is done cyclically and automatically with a low frequency. This command does not read the analogue values but returns the values which have been read at last cycle. So when this command is repeated too fast, it may return the same values.
This command returns the current values of all inputs AIn0, AIn1 and AIn2 all together.
To use this command, firmware version 4 and the Multi-IO Extension Board are needed.

18.2 Stand-Alone Control Commands

Following commands are useful in case scanner controller is operating in stand-alone mode where marking data are loaded from microSD-card using special EPR-fileformat.

`cstal <mode>`

"**s**et **st**and-**al**one" – set/change the stand-alone mode to be used. Using this command the stand-alone mode the controller is currently using, is changed. Here as parameter mode one of the following can be set which changes the stand-alone mode according to the description in section "8.2 Stand-Alone Configuration Parameters" above:

- auto
- ioselect
- idxselect
- loop
- haltedloop
- iohaltedloop

- off

Using this command stops a possibly running marking operation immediately, unloads all stand-alone data which may be loaded, sets the controller to idle mode and applies the new stand-alone operation mode.
This command requires firmware version 24 or newer.

`cstop`

"**stop**" – stop marking as fast as possible. A running marking operation is stopped and LaserGate is turned off.

`chalt <0/1>`

"**halt**" – halts or continues the processing and output of marking data. When given parameter is equal to 1, marking is stopped next time the laser is off but no vector data are flushed. On continue (parameter equal 0) controller continues processing at the point where halt occurred. When marking is stopped with `cstop` the halt-condition is cleared too, means on next transmission of new marking data they are processed without the need to explicitly continue operation.

`cstrt`

"**start**" – start marking operation. This command can be called only when no marking operation is running and when a valid project (.epr) file was loaded. In this case the currently loaded project is marked once.

`ctrig`

"**trig**ger" – send an external trigger signal by software. When scanner card is in state "marking" but waiting for an external trigger, this command releases this trigger. So behaviour is the same like a rising edge on the ExtStart input of the controller card.

`cstat`

"**stat**e" – return the current state of the card. This command returns one of the following texts identifying the operational state:

- `marking` – card is processing some marking data currently, means either actively outputting them or waiting for an external trigger to start marking
- `stand-alone` – controller is in stand-alone mode
- `idle` - card is waiting and not marking
- `waiting` - a project file was loaded, is ready for execution and waits for a trigger signal (either via ExtStart input or via command "`ctrig`")

`cscnc`

"set **CNC** data" – switch to a mode where G-Code process data are accepted via Telnet/serial interface. When this command is set, the G-Code reception mode stays active until a command "M2" (end of G-Code program) is detected. Only with this G-Code the controller returns to normal operation mode and again accepts native "c"- and "d"-commands.
This command can be used to download a CNC-program to the controller's RAM directly. Such a CNC-program then can be executed like any other, stand-alone program which has been loaded from the SD-card. So using of command "cscnc" requires stand-alone mode "auto" in order to store the received G-Code data in memory for further processing and to control execution of the G-Code data. After the G-Code data have been transferred and transmission has been ended with "M2", marking of these data can be started by applying an external trigger at ExtStart or by sending a trigger-event via command (e.g. "`ctrig`").
To successfully send G-Code data, some preconditions have to be met. For data transmission via Telnet:

- a G-Code line is limited to 255 characters maximum and always has to end with carriage return and/or line feed
- when transferring more than one line at the same time, the maximum packet size is 1460 bytes, at the end of such a packed a G-Code line has to end too and it is recommended to flush the full output buffer in order to invoke a data transmission over TCP/IP; this procedure is recommended in order to have an as fast as possible data transfer independent from the implementation a TCP/IP stack really uses

For data transmission via USB serial interface:

- a G-Code line is limited to 255 characters maximum and always has to end with carriage return and/or line feed

The structure and supported G-Code commands are described in section “19 Supported CNC G-Code Commands“ below.
This function requires firmware version 3 or newer.

`cxcnc`

“e**x**ecute **CNC** data” – switch to a mode where G-Code process data are accepted via Telnet/serial interface and executed more or less immediately. When this command is set, the G-Code reception mode stays active until a command “M2” (end of G-Code program) is detected. Only with this G-Code the controller returns to normal operation mode and again accepts native “c”- and “d”-commands.
This command can be used to send CNC-commands to the controller in order to let them be executed automatically like some direct control commands. Different to “`cscnc`” it is not related to stand-alone operations, the G-codes are executed like a stream of commands. So “`cxcnc`” provides an other possibility to send operation data to the controller. When enough data have provided to the controller, it starts marking automatically. In case of only very few CNC data that aren’t enough to let the controller start marking, this is done at the end of the CNC stream of data: after all the available G-Code data have been transferred and transmission has been ended with “M2”, marking of these data is started.
To successfully send G-Code data, some preconditions have to be met. For data transmission via Telnet:

- a G-Code line is limited to 255 characters maximum and always has to end with carriage return and/or line feed
- when transferring more than one line at the same time, the maximum packet size is 1460 bytes, at the end of such a packed a G-Code line has to end too and it is recommended to flush the full output buffer in order to invoke a data transmission over TCP/IP; this procedure is recommended in order to have an as fast as possible data transfer independent from the implementation a TCP/IP stack really uses

For data transmission via USB serial interface:

- a G-Code line is limited to 255 characters maximum and always has to end with carriage return and/or line feed

The structure and supported G-Code commands are described in section “19 Supported CNC G-Code Commands“ below.
This function requires firmware version 20 or newer.

`cgtin`

"**g**et **t**rigger **in**puts" – get the state of the external input signals. This command is not related to digital inputs of digital interface but provides information regarding signal state of external start and stop. It returns a value that specifies which of these input signals are currently HIGH:
0 – ExtStart and ExtStop are both LOW
2 – ExtStart is HIGH
4 – ExtStop is HIGH
6 – ExtStart and ExtStop are both HIGH

`cscor <idx>`

“**s**et **cor**rection” - specifies a new index for a previously loaded correction file (see description of configuration parameter `corrtable` in section “7.5 microSD-Card” above). The parameter `idx` can be a value in range 0..15 and needs to correspond to a previously loaded correction table. The newly set correction table applies to all vector data which are processed after this call. Thus it is recommended to use it only when marking operation was stopped – elsewhere it is not predictable how many vector data already have been pre-calculated with the previous correction table and starting with which vector data the new correction file is used.
When a `idx`-value is set which corresponds to no correction file data, no more correction is performed on vector data.
This command requires firmware version 6 or newer.

`cgcor`

"**g**et **cor**rection" - this command is the counterpart of `cscor` and displays the index number of the currently used correction file
This command requires firmware version 6 or newer.

`clepr <path>`
"**l**oad **epr**" – loads an EPR stand-alone file or CNC G-Code file from microSD card for outputting it on next marking operation. This command can be executed in stand-alone mode only.

When operating in stand-alone mode "idxselect", the command expects a number as parameter which specifies the index file to be loaded.
When operating in stand-alone mode "ioselect" or "iohaltedloop", the command is not supported.
When operating in any other stand-alone mode, the command expects the path to the file to be loaded as parameter. Since this is the only parameter, no quotes are allowed for the pathname. The pathname itself has to be in format
`0:/filename.epr`
or
`0:/filename.cnc`
where `0:/` specifies the microSD-card, `.epr` is the standard extension of E1803D stand alone marking data files (this name is a short-cut for "**E1803D Pr**ocessing Data") and `.cnc` is the extension which has to be used when an ASCII-G-code file is provided. EPR-files can be created out of BeamConstruct, CNC-files are text files containing valid G-Code commands as described in section "19 Supported CNC G-Code Commands"
During loading the ready-for-marking output signal is turned off and it is turned on only in case the file could be loaded successful (please refer to related section above).
Examples: `clepr 0:/test.epr` – loads a stand-alone file "test.epr" from microSD card

`cgepr`
"**g**et **epr**" – returns the name of the currently loaded stand-alone file or an error "no file specified" when no file is loaded.

`cdepr <path>`
"**d**elete **epr**" – deletes an EPR stand-alone file and all related, additional files from microSD card. This command can be executed in stand-alone mode only.

When operating in stand-alone mode "idxselect", the command expects a number as parameter which specifies the index file to be loaded.
When operating in stand-alone mode "ioselect" or "iohaltedloop", the command is not supported.
When operating in any other stand-alone mode, the command expects the path to the file to be delete as parameter. Since this is the only parameter, no quotes are allowed for the pathname. The pathname itself has to be in format
`0:/filename.epr`
where `0:/` specifies the microSD-card and `.epr` is the standard extension of E1701 stand-alone marking data files (this name is a shortcut for "**E**1803 **Pr**ocessing Data"). Deleting is done asynchronously, so the returned "OK" only verifies the command was accepted. Successful deletion of the file can be assumed after 0,5 seconds.
Examples: `cdepr 0:/test.epr` – deletes a stand-alone file "test.epr" from microSD card. When additional files `test.dat` (which may contain related dynamic data) and `test.ser` (which may hold serial number information) exist, they are deleted by this command too.
This command requires a firmware version 16 or newer.

`ctlxy <x> <y> <z>`
"**t**urn on **l**aser at given **XY**Z position" – lets the scanner move to a specific position and turns on the laser as long as ExtStart is HIGH
This command requires stand-alone mode "auto". It can be used to turn on the laser as long as the ExtStart input is at HIGH. So using this function some externally controlled drilling or cutting operations are possible. As parameter it expects the X, Y and Z coordinate positions to jump to (in unit 26 bits). The sequence of commands to use this function are as follows:

- unload the current project file ("e.g. by calling "`clepr 0:/notexist.epr`")

- call "ctlxy" together with the 26 bit XYZ position the scanner has to jump to → the parameters used for jumping are the ones out of the last EPR file executed
- the state-flag E180X_CSTATE_SAC_CTLXY is set
- the state-flag E180X_CSTATE_SAC_READY is set as soon as the scanner has arrived at the specified XYZ position, this happens together with the output DOut0:
- wait until DOut0 goes to HIGH → this output signals the specified XYZ position was reached and the controller is able to turn on the laser now
- pull ExtStart to HIGH → the laser is turned on within 1 usec at max using the laser parameters out of the last EPR file executed
- as long as ExtStart is HIGH, the laser is turned on and DOut1 is at HIGH and the state-flag E180X_CSTATE_SAC_MARKING is set
- pull ExtStart to LOW as soon as the laser has to be turned off → both DOut0 and DOut1 are pulled to low, the laser is turned off within 1 msec at max and all E180X_CSTATE_SAC_-flags are reset
- now the complete sequence has ended ad the ctlxy-command has elapsed. To turn on the laser via ExtStart again, a new command ctlxy has to be issued

Example:
`ctlxy 33554432 33554432 33554432`
Starts a ctlxy-sequence as described above with the scanner moved to the centre position.

This command requires firmware version 13 or newer.

`csbuf <idx> <path>`

"**s**et **buf**fer" - this command works similar to the configuration parameter "iobuff" and can be called in stand-alone modes "ioselect" and "idxselect". It can be used to preload EPR stand-alone data files into memory so that they can be accessed faster and without additional accesses of the SD card. The first parameter <idx> is mandatory and specifies the slot the EPR file has to be loaded to. Different to parameter "iobuff" where the slot number is given automatically and where the index value can be in range 1..255, here the slot number and the index number are always the same and are in range 1..20. So only the first 20 index values can be used together with this dynamic loading function. When only <idx> is given, the loaded file is 0:/<idx>.epr. Optionally also a file name <path> can be given in format 0:/file.epr. In this case the given file is loaded to the slot specified by <idx> and can be accessed with the related index number. After every call to csbuf the command cgbuf has to be executed repeatedly until it returns -1.

Examples:
`csbuf 3` – loads the file 0:/3.epr from SD-card and stores it in slot number 3 so that it can be accessed either via "clepr 3" (in stand-alone mode "idxselect") or via a digital input pattern at the digital interface representing a 3 (in stand-alone mode "ioselect)
`csbuf 5 0:/markme.epr` – loads the file 0:/markme.epr from SD-card and stores it in slot number 5 so that it can be accessed either via "clepr 5" (in stand-alone mode "idxselect") or via a digital input pattern at the digital interface representing a 5 (in stand-alone mode "ioselect)

This command requires firmware version 9 or newer.

`cgbuf`

"**g**et processed **buf**fer" - returns the index number of the buffer that is currently processed by a previous call to cgbuf. As long as this command returns a value greater than -1, no other calls to csbuf are allowed. The returned number specifies the index/slot number that is currently filled with data. When cgbuf returns -1, the loading operation has been finished.

This command requires firmware version 9 or newer.

`cstxt <"elementname"> <"text">`

"**s**et **t**e**xt**" – set a new text value to an element in currently loaded project. The parameters for this command both have to be given with quotes ("). Setting a text is possible only for dynamic elements like DataMatrix or QR barcodes or texts. Here "elementname" is the name of the element that has to be modified

(this is the same name like shown in element-tree of BeamConstruct) and the new text to be set. The "text" itself can be a format string as used within BeamConstruct when a serial number input element is involved
Example: `cstxt "Barcode 1" "Hello :-)"` - sets a new text "Hello :-)" for the element with name "Barcode 1"

`cgtxt <"elementname">`

"**g**et **t**e**xt**" – gets the currently used text value of an element in loaded project. The parameter for this command has to be given with quotes ("). Getting a text is possible only for dynamic elements like DataMatrix or QR barcodes or texts.

Example: `cgtxt "Barcode 1"` – gets the text from the element with name "Barcode 1"

`csser <"elementname"> <cnt>`

"**s**et **ser**ial number" – sets a new serial count value to an element in currently loaded project. The element name for this command has to be given with quotes ("). Setting a new count is possible only for dynamic elements like DataMatrix or QR barcodes or texts that have a serial number input element assigned. Setting the value has to be handled with care, here every value can be specified independent if it fits to possibly exiting beat count values.
Example: `csser "Text 2" 42` – set a new serial number count value 42 for element with the name "Text 2"

`cgser <"elementname">`

"**g**et **ser**ial number" – gets the current serial count value from an element in loaded project. The element name for this command has to be given with quotes ("). Getting the count is possible only for dynamic elements like DataMatrix or QR barcodes or texts that have a serial number input element assigned.

`ciser <"elementname">`

"**i**ncrement **ser**ial number" – increments the current serial count value of an element according to its serial number parameters. The element name for this command has to be given with quotes ("). Incrementing the count is possible only for dynamic elements like DataMatrix or QR barcodes or texts that have a serial number input element assigned. This function is more secure than forced setting of a new count value with "csser" since it can't violate the counting rules.

`cdser <"elementname">`

"**d**ecrement **ser**ial number" – decrements the current serial count value of an element according to its serial number parameters. The element name for this command has to be given with quotes ("). Decrementing the count is possible only for dynamic elements like DataMatrix or QR barcodes or texts that have a serial number input element assigned. This function is more secure than forced setting of a new count value with "csser" since it can't violate the counting rules.

`crser <"elementname">`

"**r**eset **ser**ial number" – resets the current serial count value of an element to its start-value (according to its serial number parameters). The element name for this command has to be given with quotes ("). Resetting the count is possible only for dynamic elements like DataMatrix or QR barcodes or texts that have a serial number input element assigned. This function is more secure than forced setting of a value with "csser" since it can't violate the predefined serial number parameters and uses the correct reset value.

`cstim <seconds>`

"**s**et **tim**e" – this command sets the system time to the value specified with the parameter. Here the number of seconds have to be specified that have elapsed since 01.01.1970 at 00:00:00. After sending this command the controller card operates at the given time. The time value is lost after next power cycle and has to be set again.

Example: `cstim 1420113600` – set the internal time of E1803D controller to 01.01.2015 12:00:00, here 1420113600 represents the number of seconds that have been elapsed between 01.01.1970 00:00:00 and 01.01.2015 12:00:00

`crtim`

"**r**etrieve **tim**e" – this command schedules time retrieval from an SNTP server asynchronously. It always returns with "OK" since the command is scheduled for execution during next working cycles. To use this command, controller has to be configured with IP, netmask, gateway and SNTP server IP correctly and needs to be able to access this SNTP server from its position in network. For details please refer to description of configuration parameters in section about microSD card above.

`cgtim`

"**g**et **tim**e" – returns the current time of the board in number of seconds that have elapsed since 01.01.1970 at 00:00:00. After powering up the board and before a valid time has been set, this value is undefined.

`cftim`

"**g**et **f**ormatted **tim**e" – returns the current time of the board as formatted string in style DD.MM.YYYY hh:mm:ss. After powering up the board and before a valid time has been set, this value is undefined.

`cstyr <year>`

"**s**et **t**ime **y**ea**r**" – sets the year of the current system time to the value given as parameter. This value has to be in range 1900..2038

`cstmo <month>`

"**s**et **t**ime **mo**nth" – sets the month of the current system time to the value given as parameter. This value has to be in range 1..12 according to the number of the month.

`cstdy <day>`

"**s**et **t**ime **d**a**y**" - sets the day of the current system time to the value given as parameter. This value has to be in range 1..28, 1..30 or 1..31 according to the length of the current month.

`csthr <hour>`

"**s**et **t**ime **h**our" – sets the hour of the current system time to the value given as parameter. This value has to be in range 0..23.

`cstmi <minute>`

"set time minute" – sets the minute of the current system time to the value given as parameter. This value has to be in range 0..59.

`cstsc <second>`

"**s**et **t**ime **se**cond" – sets the second of the current system time to the value given as parameter. This value has to be in range 0..59.

`cgsta`

"**g**et **s**erial s**ta**te" – this command applies only when working in stand-alone mode with dynamic serial number data that change on every mark operation. It returns information if the state of serial numbers has changed and is not yet saved (in this case "pending" is returned) or if they have been saved and therefore do not get lost when power is turned off now ("saved" is returned in this case).

`cssta`

"**s**ave **s**erial s**ta**te" - this command applies only when working in stand-alone mode with dynamic serial number data that change on every mark operation. When it is called, a command to save the current state of serial numbers is enqueued and will be processed as soon as controller is able to store these data. So when this command returns with "OK" that doesn't necessarily means the serial number states are saved now. The current save state still has to be checked by calling "cgsta" after "cssta" has been issued.

`cdtl0 <text>`

"**d**ot **t**ext **l**ine **0**" - this command is used in dot mode matrix marking mode only. It can be used to set a new line of text for the upper line to be marked in dot matrix mode. It can be called several times in advance to set some more texts. They are marked in the order they have been set on every marking cycle started by an ExtStart input signal. Stopping a marking operation by calling "cstop" or by applying a signal to ExtStop input deletes all previously set texts and empties the whole text buffer.

`cdtl1 <text>`

"**d**ot **t**ext **l**ine **1**" - this command is used in dot mode matrix marking mode only. It can be used to set a new line of text for the lower, optional line to be marked in dot matrix mode. It can be called several times in advance to set some more texts for the second line. When the controller card is configured to work with two lines of texts, it always expects pairs of texts to be set, means "cdtl0" and "cdtl1" needs to be used always together. They are marked in the order they have been set on every marking cycle started by an ExtStart input signal. Stopping a marking operation by calling "cstop" or by applying a signal to ExtStop input deletes all previously set texts and empties the whole text buffer.

`crrrr`

"**r**eboot" – perform a warm reboot of the hardware and restart the firmware. Reboot is done immediately, means this command does not return anything but connection to the board will be interrupted as soon as it has been sent.

18.3 Hardware Commands

These commands can be used to access hardware signals directly. When these hardware outputs are set or unset while a marking operation is running, they may have no effect as they may be overridden immediately. Thus it is recommended to execute them only when the controller card is idle and no other operations are in progress. But also in this case, when a hardware output is set to a specific state, any operation (especially marking cycle) that is executed afterwards, may override that specific state-changes. Following hardware-specific commands are supported:

`csout <value>`

"set output" – set the state of the digital outputs. The output to be set is specified as a decimal number representing the bitpattern. When no parameter is given, the behaviour is undefined.
Example: `csout 128` - set DOut7 at the digital interface to HIGH while all others stay at LOW

`cglog`

"**g**et **log**line" – returns a single logging line. This command has to be called repeatedly until an error is returned to get logging information from the controller. On each call of this function one logging line is returned. When "`cglog`" isn't used for a longer time it may be possible the internal log-buffer has overrun. In this case "`cglog`" will not return all log information.

`cslgt <value>`

"**s**et **L**aser**G**a**t**e" – set the state of the LaserGate output either to HIGH (value is set to 1) or to LOW (value is set to 0).
This command requires firmware version 15 or newer.

`cslmo <value>`
“**s**et **MO**” – set the state of the main oscillator output either to HIGH (value is set to 1) or to LOW (value is set to 0).
This command requires firmware version 15 or newer.

`cslp8 <value>`
“**s**et **LP8**” – set the state of the LP8 output port to the value given as parameter. Here value is allowed to be in range 0..255, the related bits of the LP8 output are set according to the bitpattern of the specified number.
This command requires firmware version 15 or newer.

`cmapz <mapping>`
Using this command the Z-axis hardware output can be used for other coordinates than the default Z. As mapping parameter the following values:

- `x` – the X-axis-coordinates are emitted at the Z-channel
- `-x` – inverted X-coordinates are emitted at the Z-channel
- `y` – the Y-axis-coordinates are emitted at the Z-channel
- `-y` – inverted Y-coordinates are emitted at the Z-channel
- `z` – the behaviour of the Z-output is set back to its default, regular Z-values are emitted when this command is given

In case of the first four mappings the Z-values are no longer available at a hardware output. When this command is called, the mapping of the output is changed immediately. A default mapping can be defined by using the configuration parameter “mapz” as described above in section “7.5 microSD-Card”
This command requires firmware version 24 or newer.

18.4 Mark Control Commands

The following section describes commands that can be used to send marking data (including vector data and laser/scanner parameters) to the controller. If possible, these commands should not be mixed with the commands described above but have a different structure:

- they always start with a character "d"
- the total length of one frame (means one command) is always 14 bytes
- they mustn't be terminated with CR/LF, the end of a frame is determined by its size of 14 bytes
- they contain binary, means not human-readable data and therefore can't be sent manually
- in bus-mode they are preceded by an additional identifier “bXX”

Different to the programming interface (“20.1 E1803D Easy Interface Functions”) mentioned below, this possibility to send control and marking data is completely independent from any host operating system and from any additional software or libraries. It gives the possibility to send marking data to the card right via some binary data which can be sent via Ethernet connection (Telnet) or USB serial interface.



PLEASE NOTE: when using Network/Telnet connection and when switching from a Mark Control Command ("d"-command) to a general command ("c"-command as described above) it is recommended to flush all output before sending a command of other type.

These commands always have the following structure:
`dCAAAABBBBEEEE`
`d` – marks starting point of a frame and identifies a mark control command with a fixed length of 14 bytes (including this character)
`C` – 8 bit value that specifies what command has to be executed
`AAAA` – 32 bit little-endian value, it's meaning and usage depends on "C"
`BBBB` – 32 bit little-endian value, it's meaning and usage depends on "C"
`EEEE` – 32 bit little-endian value, it's meaning and usage depends on "C"

When communicating via RS485 serial interface in bus-mode (means when several controllers are connected via the same RS485-line, when `u0bypass` is set to 1 and when `busid` is set), every d-command has to be

preceded by fixed three-bytes in format “bXX” where “XX” is a value in range 00..99 equal to the bus-identifier set with configuration parameter `busid`. So in busmode the structure of a d-command is `bXXdCAAAABBBBEEEE`.

It is recommended to collect commands before they are sent to the controller, especially in case Ethernet connection is used. In case of TCP/IP the used payload length of a TCP-frame is 1460 bytes which should be filled as much as possible in order to avoid additional data transfers. So when sending larger amounts of data to the controller, up to 104 command frames should be collected and then sent all together (104 * 14 = 1456 bytes which is close to 1460).

From time to time the controller sends back an answer to give back some state information. In case of Ethernet/Telnet connection this answer is not sent periodically but as response to a complete block of data sent to the card. Since the size of such a block is not specified and depends on the underlying TCP/IP implementation (in case of Ethernet connections), no predictions can be made after what amount of data a response frame is sent. Thus it is recommended to try to receive such a response frame every time some data have been transmitted until at least one frame was received. When host software is idle, it can try to read response frames permanently. To trigger transmission of a new response frame, "ping" control command `0x0A` can be used (for details please refer below).
In case of USB/serial connection this response is sent automatically after every 14 byte frame submitted, so it is necessary to always read them in order to avoid overrun of receive buffers.
Such a response frame gives back information about the current operational state of the card and comes in following structure:
`dRLLLLSSSSIIII`
`d` – marks starting point of a response frame with a fixed length of 14 bytes (including this character), this character can be used to re-synchronise
`R` – 8 bit value, currently always 0xFF; this value has to be checked for future compatibility, in case it is not 0xFF the frame has to be ignored!
`LLLL` – 32 bit little-endian value, here the amount of free command buffer space is returned; sending application has to ensure this buffer never overruns, so it is recommended to always leave a space of at least 200 commands (recommended: 1000), new commands should be sent only when there is more space than this left in this buffer
`SSSS` – 32 bit little-endian value, signalling operational state; this value can consist of following or-concatenated flags:

- `0x00000001` – card is currently marking
- `0x00000002` – the external start input is currently HIGH
- `0x00000004` – the external stop input is currently HIGH
- `0x00000008` – the external start input was set to HIGH after last response frame, this value is set only once for every rising edge on this input
- `0x00000010` – the external stop input was set to HIGH after last response frame, this value is set only once for every rising edge on this input
- `0x00000080` – the controller has received some data which may result in a marking operation; these data are currently processed but marking has not yet started
- `0x00000400` – card is active but currently waiting for an external trigger to continue operation

`IIII` - 32 bit little-endian value, lower 8 bit show the actual state of digital inputs

Currently following mark control commands (identified by the 8 bit hexadecimal value for position "`C`" in a frame) can be sent to the controller:

Jump to Position
Move to a given coordinate position using the current jump speed and with laser turned off
C = 0x00
AAAA = x-position to move to in range 0..67108863
BBBB = y-position to move to in range 0..67108863
EEEE = z-position to move to in range 0..67108863

Mark to Position

Move to a given coordinate position using the current mark speed and with laser turned on
C = 0x01
AAAA = x-position to move to in range 0..67108863
BBBB = y-position to move to in range 0..67108863
EEEE = z-position to move to in range 0..67108863

Start output

This command has to be called at the end of every marking sequence to ensure marking output really starts. This is important in case only a few vectors are sent to ensure marking is started but it is recommended to always use this command.
C = 0x02
AAAA = unused, set to 0
BBBB = unused, set to 0
EEEE = unused, set to 0

Wait for external trigger

Set a trigger point to current position of stream; emission of output data will stop until an external trigger signal is detected
C = 0x03
AAAA = unused, set to 0
BBBB = unused, set to 0
EEEE = unused, set to 0

Set speed values

Specify the speeds to be used during jump or mark movements (invoked by commands 0x00 and 0x01)
C = 0x04
AAAA = jumpspeed in unit bits per microsecond
BBBB = markspeed in unit bits per microsecond
EEEE = unused, set to 0

Set laser delays

Specify the delays to be used when laser is turned on or off
C = 0x05
AAAA = laser on delay in unit microseconds and in range -10000000..10000000
BBBB = laser off delay in unit microseconds and in range 0..10000000
EEEE = unused, set to 0

Set scanner delays

Specify the delays to be used before and after mark and within a polygon
C = 0x06
AAAA = jumpdelay in unit microseconds
BBBB = markdelay in unit microseconds
EEEE = in-polygondelay in unit microseconds

Stop marking

Tries to halt, continue or stop current output depending on the chosen option
C = 0x07
AAAA = stop option:

0 - tries to stop operation as fast as possible and rejects all data that still may be enqueued for execution

1 - marking is stopped next time the laser is off but no vector data are flushed, card is still active

2 - controller continues processing at the point where halt occurred (requires a previously called command 0x07 with stop option 1)

BBBB = unused, set to 0

EEEE = unused, set to 0

Set wobble parameters

Specify the wobble settings to be used for next marking operations

C = 0x08
AAAA = wobble amplitude in X-direction using unit bits and with maximum range of 0..10000000 bits
BBBB = wobble amplitude in Y-direction using unit bits and with maximum range of 0..10000000 bits
EEEE = wobble frequency in unit Hz*100 and in range 1..2500000

Set LP8 outputs

Set LP0..LP7 output pins on laser signal connector

C = 0x09
AAAA - bitpattern to be set on LP0..LP7 output pins, here only lower 8 bits are used.
BBBB = unused, set to 0
EEEE = unused, set to 0

Ping

This command can be used to let the controller send back a state-information. So it can be used to check if the card is still operating or not.



ATTENTION: this command should not be sent repeatedly and without any delay! This could cause E1803D scanner controller to stall because the massive data transfer has to be handled. So it is recommended to have a delay of at least 150 msec between every ping.

C = 0x0A
AAAA - unused, set to 0
BBBB = unused, set to 0
EEEE = unused, set to 0

Set digital outputs

Set DOut0..DOut7 output pins on digital interface connector

C = 0x0B
AAAA - bitpattern to be set on DOut0..DOut7 output pins, here only lower 8 bits are used.
BBBB = bitmask specifying which of the bits in AAAA have to be set or cleared, all these bits in AAAA are left unchanged, where the corresponding bit in BBBB is 0
EEEE = unused, set to 0

Set lasermode

Specify the laser mode the card has to operate with

C = 0x0C
AAAA - flags specifying the laser mode, here following values have to be or-concatenated to specify the behaviour of a laser:

- `0x40000000` - laser frequency on LaserA output is turned on immediately and together with laser gate signal, this flag can't be used together with `0x20000000`
- `0x20000000` - laser frequency on LaserA output is turned on after FPK time, this flag can't be used together with `0x40000000`
- `0x10000000` - laser supports FPK on LaserB output
- `0x08000000` - laser frequency has to be turned off and switched to standby-frequency
- `0x04000000` – a frequency can be emitted at LaserB permanently, the related frequency can be specified with command 0x15

Using these flags following laser types can be configured:

- CO_2-laser:
  `0x40000000 + 0x08000000`
- YAG-laser with FPK:
  `0x40000000 + 0x08000000 + 0x10000000` or
  `0x20000000 + 0x08000000 + 0x10000000`

- laser with continuously running frequency: `0x40000000`

BBBB = unused, set to 0
EEEE = unused, set to 0

Set marking on-the-fly parameters

Specify the parameters used for marking on-the-fly applications

C = 0x0D
AAAA = marking on-the-fly resolution in X-direction in unit bits per encoder increment
BBBB = marking on-the-fly resolution in Y-direction in unit bits per encoder increment
EEEE = unused, set to 0

Set laser frequency

Specify the frequency the laser has to be operated with during marks, usage of these parameters depends on the lasermode specified with command 0x0C

C = 0x0E
AAAA = frequency in unit Hz and in range 25..20000000 Hz
BBBB = pulse-width in unit microseconds and in range 1..65530 usec
EEEE = unused, set to 0

Set laser standby frequency

Specify the frequency the laser has to be operated with during jumps, usage of these parameters depends on the lasermode specified with command 0x0C

C = 0x0F
AAAA = frequency in unit Hz and in range 25..20000000 Hz
BBBB = pulse-width in unit microseconds and in range 1..65530 usec
EEEE = unused, set to 0

Set first pulse killer

Specify the pulse width of the FPK signal when laser is turned on, usage of these parameters depends on the lasermode specified with command 0x0C

C = 0x11
AAAA = FPK pulse width in unit microseconds*100
BBBB = the time the laser frequency has to be started after beginning of FPK using unit microseconds*2, this value is used only when lasermode flag 0x20000000 is set
EEEE = unused, set to 0

Switch MO-output

Turns the MO-output of laser interface connector on or off

C = 0x12
AAAA = turn MO output on when equal 1, turn it off when 0
BBBB = unused, set to 0
EEEE = unused, set to 0

Release external trigger

When card is waiting for an external trigger this command can be sent to release this external trigger by software and to continue execution without the need to receive a real external signal

C = 0x13
AAAA = unused, set to 0
BBBB = unused, set to 0
EEEE = unused, set to 0

Wait for external input signal

Stop execution until a defined input bitpattern is detected at configurable input pins DIn0..DIn7 of digital interface connector
C = 0x14
AAAA = a bitpattern specifying which signals LOW or HIGH have to be detected at digital input pins
BBBB = a bitpattern specifying which of the digital input pins have to be watched for a signal, these bits that are set to 0 are ignored while these bits, that are set to 1 have to get the state specified in previous parameter in order to let operation of card continue
EEEE = unused, set to 0

Set LaserB frequency

Specify the frequency LaserB output has to emit; this function can only be used when operating using a laser mode with flag `0x04000000` set (see command 0x0C above).
C = 0x15
AAAA = frequency in unit Hz and in range 25..20000000 Hz
BBBB = pulse-width in unit microseconds and in range 1..65530 usec
EEEE = unused, set to 0

Wait until on-the-fly-increments have been elapsed

This command adds some special kind of delay to the application. It can be used only when marking on-the-fly is enabled (by setting the on-the-fly factors), and halts laser marking not for a given time but for a given distance. Marking is continued only when the given number of increments has elapsed. When no or not enough increments are counted by the controller, operation only can be stopped.
This command requires firmware version 27 or newer.
C = 0x16
AAAA = positive or negative number of increments to wait for until operation continues; here it depends on the used counting direction of the encoder if the given distance-value has to be positive or negative, when sign of the number and counting direction of the encoder do not fit to each other, the controller will halt at this position for a very long time
BBBB = unused, set to 0
EEEE = unused, set to 0

Insert a delay into the stream of data

This command adds a delay and lets the controller wait for the given time of ticks until the next command is executed. One tick is equal to 0,5 usec.
This command requires firmware version 12 or newer.
C = 0x18
AAAA = number of ticks to wait for
BBBB = unused, set to 0
EEEE = unused, set to 0

Switch LP8-Latch-output

Turns the LP8-Latch-output of laser interface connector on or off
This command requires firmware version 12 or newer.
C = 0x19
AAAA = turn Latch output on when equal 1, turn it off when 0
BBBB = unused, set to 0
EEEE = unused, set to 0

Specify output for MIP-signal

Specify a single output pin of digital interface connector to be used for "Mark in progress"-signal, this output pin will be HIGH as long as a marking operation is in progress.
C = 0x2A
AAAA = the number (not a bitpattern!) of the digital output pin to be used for MIP-signal (in range 0..7)
BBBB = unused, set to 0
EEEE = unused, set to 0

Halt/continue current marking operation

Stops the current marking operation on very next appearance of a jump or continue a previously halted operation.
C = 0x2F
AAAA = 1 to halt marking and 0 to continue a halted operation
BBBB = unused, set to 0
EEEE = unused, set to 0

Specify output for WET-signal

Specify a single output pin of digital interface connector to be used for "Wait External Trigger"-signal, this output pin will be HIGH as controller is waiting for an external trigger.
C = 0x33
AAAA = the number (not a bitpattern!) of the digital output pin to be used for WET-signal (in range 0..7)
BBBB = unused, set to 0
EEEE = unused, set to 0

Set first row of 2x2 output matrix

Specify the elements m11 and m12 of a 2x2 output matrix which is applied to all coordinate values as soon as the second half is applied with command 0x41. This matrix can be used to scale, slant, rotate and mirror the input coordinates in respect to the output positions. For details please check out description of command 0x41 below
This command requires firmware version 3 or newer.
C=0x40
AAAA = the m11 part of the 2x2 matrix multiplied with 1000000
BBBB = the m12 part of the 2x2 matrix multiplied with 1000000
EEEE = unused, set to 0

Set second row of 2x2 output matrix

Specify the elements m21 and m22 of a 2x2 output matrix which is applied to all coordinate values together with the first row of matrix data which has to be set using command 0x40 in a preceding call. This matrix can be used to scale, slant, rotate and mirror the input coordinates in respect to the output positions. Assumed a matrix bases on a 4-elements array, it has following structure:

```
{m11, m12, m21, m22}
```

then these matrix values can be used and even combined with each other by multiplying them:

- rotation: `{cos(angle), -sin(angle), sin(angle), cos(angle)}`
- scaling: `{factorX, 0.0, 0.0, factorY}`
- slant X: `{1.0, 1.0/tan(angle), 0.0, 1.0}`
- slant Y: `{1.0, 0.0, 1.0/tan(angle), 1.0}`
- mirror X: `{-1.0, 0.0, 0.0, 1.0}`
- mirror Y: `{1.0, 0.0, 0.0, -1.0}`

This command requires firmware version 3 or newer.
C=0x41
AAAA = the m21 part of the 2x2 matrix multiplied with 1000000
BBBB = the m22 part of the 2x2 matrix multiplied with 1000000
EEEE = unused, set to 0

Download new firmware

Download a new firmware file to the controller and write it to the microSD-card so that it can be used after next reboot. The binary data of the new firmware have to be appended directly to this command. This command has to be used in a specific sequence in order to ensure the current firmware file is updated correctly:

- ensure the card is idle (state-flag SSSS is 0)
- send command 0x45 with length of firmware data and checksum
- send binary firmware data directly after this command
- wait until card state returns “active” (by repeatedly sending ping-commands), now in state flag SSSS bit 0x4000 (E180X_CSTATE_WRITING_DATA) is set
- wait until card state returns “idle” again (by repeatedly sending ping-commands), the flag 0x4000 no longer should be set in state flag SSSS
- check if an error occurred: when flags 0x8000 (E180X_CSTATE_WRITING_DATA_ERROR) is set in state flag SSSS, downloading or writing or checksum calculation failed and the original file was not replaced; for debugging in such a case the command “cglog” can be called repeatedly until the related error text was found; when this error flag is set it can be reset only by using command 0x45 again
- reboot the controller
- check if the version of the firmware has changed

This command requires firmware 4 or newer.
C = 0x45
AAAA = the length of the firmware file in bytes
BBBB = checksum for verification of the downloaded data, only when this checksum is correct, the old firmware file will be replaced; the checksum can be calculated using following function (C example code):

```
unsigned int crc32b(const char *buf,size_t len)
{
   int          k;
   unsigned int crc=0xFFFFFFFF;

   while (len--)
   {
      crc^=*buf++;
      for (k=0; k<8; k++)
       crc=crc&1 ? (crc>>1) ^ 0x82f63b78 : crc>>1;
   }
   return ~crc;
}
```

EEEE = specifies the file which has to be overwritten by the current data download:
- 0 – overwrite file 0:/version.txt when downloading of data was successful
- 1 – overwrite file 0:/e1803.fwi when downloading of data was successful
- 2 – overwrite file 0:/e1803.dat when downloading of data was successful
- 3 – overwrite file 0:/e1803.cfg when downloading of data was successful

Set Position Offset

Set a positive or negative offset which then applies to all following position commands submitted via command 0x00 or 0x01.
This offset is used until the controller is rebooted or until an other offset is set
This command requires firmware version 37 or newer.
C = 0x4D
AAAA = x-position offset in range -33554432..33554432
BBBB = y-position offset in range -33554432..33554432
EEEE = z-position offset in range -33554432..33554432

Set the digital outputs

Using this function the state of the digital interface outputs DOu0 to DOut7 can be changed by clearing and setting the related bits.
Additionally a delay value can be added which is applied after the change of the states of the outputs. This delay can be used to generate pulses. It lets the controller wait for the given time of ticks until the next command is executed. One tick is equal to 0,5 usec.
This command requires firmware version 22 or newer
C = 0x90
AAAA = clear bits; this value specifies which bits of the digital output have to be cleared. Here a value in range 0..255 is expected and all value-bits that are set, correspond to digital outputs that are cleared

BBBB = set bits; this value specifies which bits of the digital output have to be set. Here a value in range 0..255 is expected and all value-bits that are set, correspond to digital outputs that will be set
EEEE = number of ticks to wait for

Reset the board

This function performs a warm reboot of the hardware and restarts the firmware. Reboot is done immediately, means this command does not return anything but connection to the board will be interrupted as soon as it has been sent.
C = 0xFF
AAAA = unused, set to 0
BBBB = unused, set to 0
EEEE = unused, set to 0

19 Supported CNC G-Code Commands

Starting with firmware version 2 E1803D supports G-Codes stored in a file on microSD card. A related CNC file has to be placed at microSD card. The file extension decides how it is loaded and interpreted, all G-Code ASCII files need to end with ".CNC".

Starting with firmware version 3 E1803D supports G-codes sent to the controller via USB serial interface or Telnet. Such a data transfer has to be started with command "cscnc" or "cxcnc" (for details please refer to description above).

Starting with firmware version 20 the controller can also be configured to be in G-Code reception mode permanently by adding a parameter "`cxcnc`" to the e1803.cfg configuration file.

When one of these direct "`cxcnc`" control modes is active, a calling application can watch the execution state either via a digital output which is configured by using the configuration parameter `wetout`, or via a specific G-Code control protocol that is described in section 19.5 Control Protocol below.

To allow fast and efficient processing of a CNC file within E1803D, some points have to be noticed. So in order to improve loading performance it is recommended to:

- not to have lots of leading or trailing spaces
- not to make use of large comments
- have exactly one space between code and related parameter

Beside of that it is mandatory to

- have a space or CR/LF between two different codes (so e.g. "`G21 G90`" is valid but "`G21G90`" will result in an error)
- have no space within a code or within a parameter of a code (so e.g. "`G0 X-0.5 Y.75 Z10`" is valid but "`G 0 X-0 .5 Y. 75 Z 10`" is not and will result in an error)
- use a dot as separator in floating point variables (so e.g. "`T1 F6000.0`" is valid but "`T1 F6000,0`" is not and will result in an error)

Following the G-Code commands are described which are supported.

19.1 General G-Code Characters

Following codes and identifiers are supported by E1803D G-Code interpreter:

Code	Description	Example
%	Marks the begin of a G-Code file, this code is optional and does not have any effect	
;	Begin of a comment, the remaining line is ignored; in order to improve loading speed of a G-Code file comments and spaces at the end of a line should be removed	`G21 ; set unit to mm`
( )	Encapsulate a comment, all data within the brackets are ignored; in order to improve loading speed of a G-Code file comments and spaces at the end of a line should be removed	`G21 (set unit to mm) G90 (use absolute positioning)`
G	G-commands, please refer below for a description	`G1 X25.75 Y31 Z0.25`
M	M-commands, please refer below for a description	`M3`
T	T-commands, please refer below for a description	`T1 F3000`

19.2 Supported “G”-codes

Following “G” codes and identifiers are supported by E1803D G-Code interpreter:

Code	Description	Example
G0	Jump to a specified position using predefined unit mm or inch and with maximal speed or – when set with command M704 – with the jump speed that was defined before. The position to jump to is specified by two or three parameters X, Y and Z. This movement is done with the laser turned off and by taking laser and scanner delays into account.	`G0 X0 Y0 Z0`
G1	Move to a specified position using predefined unit mm or inch and with a default or – when set with command M704 – with the mark speed that was defined before. The position to move to is specified by two or three parameters X, Y and Z. This movement is done with the laser turned on and by taking laser and scanner delays into account.	`G1 X10 Y10.5 Z11.75`

G2	Adds a clockwise arc move with the laser turned on. An arc move starts at the current position and ends at the given XY coordinate, pivoting around a centre point offset given by the coordinates I and J or specified by the radius R. This command has two forms:
	
	IJ-form
	• I and J specifiy an X and Y offset, at least one of them is always required
	• mixing I or J with R will result in an undefined behaviour
	R-form
	• R specifies the radius, X and Y is mandatory
	• X or Y must be different from the current position
	• mixing I or J with R will result in an undefined behaviour
	
	Requires firmware version 20 or newer
		`G2 X12.5 Y30 I10 J10`
		
		`G2 X19 Y22.5 R 10`

G3	Same as G2 but performs counter-clockwise movements, for details please refer above.
	
	Requires firmware version 20 or newer
		`G3 X12.5 Y30 I10 J10`
		
		`G3 X19 Y22.5 R 10`

G4	When followed by a parameter „P“ execution is delayed by the given time (in unit seconds)	`G4 P0.002`
G17	Select the XY plane for marking operations. This means, given jump or movement coordinates in X and Y directions are applied to X and Y plane, optional Z coordinate is applied to remaining Z direction.	`G17`
G18	Select the ZX plane for marking operations. This means, given jump or movement coordinates in X and Y directions are applied to Z and X plane, optional Z coordinate is applied to remaining Y direction.	`G18`
G19	Select the YZ plane for marking operations. This means, given jump or movement coordinates in X and Y directions are applied to Y and Z plane, optional Z coordinate is applied to remaining X direction.	`G19`

Code	Description	Example
G20	Set measurement unit to inch, means all positions handed over e.g. with G0 or G1 will be followed by coordinates in inch. In reality this has no effect for E1803D since calculation of marking positions is done based on the given working area which has to be defined with command M709	`G20`
G21	Set measurement unit to mm, means all positions handed over e.g. with G0 or G1 will be followed by coordinates in mm. In reality this has no effect for E1803D since calculation of marking positions is done based on the given working area which has to be defined with command M709	`G21`
G70	Set measurement unit to inch, means all positions handed over e.g. with G0 or G1 will be followed by coordinates in inch. In reality this has no effect for E1803D since calculation of marking positions is done based on the given working area which has to be defined with command M709	`G70`
G71	Set measurement unit to mm, means all positions handed over e.g. with G0 or G1 will be followed by coordinates in mm. In reality this has no effect for E1803D since calculation of marking positions is done based on the given working area which has to be defined with command M709	`G71`
G90	Enable absolute positioning, means all positions handed over e.g. with G0 or G1 will be followed by absolute coordinates according to the used coordinate system.	`G90`
G91	Enable relative positioning, means all positions handed over e.g. with G0 or G1 will be followed by coordinates that are relative to the previously used position in used coordinate system.	`G90`

19.3 Supported “M”-codes

Following “M” codes and identifiers are supported by E1803D G-Code interpreter, here all codes in range 700..799 are specific to the E1803D and contain all laser-related parameters and values:

Code	Description	Example
M2	End of program. When this code is found, parsing of the CNC file is stopped and all following codes are ignored	`M2`
M3	Set laser on. This command does NOT turn on the laser but sets the internal state to „on“. This can be used to set e.g. the mark speed by a following command T1.	`M3`
M5	Set laser off. This command does NOT turn on or off the laser but sets its internal state to „off“. This can be used to set e.g. the jump speed by a following command T1.	`M5`
M700	Set the used laser type. This command is mandatory and has to be called prior to every laser-related command. As parameter it expects a decimal number which corresponds to the lasermode-types `E180X_LASERMODE_xxx` as described in section „20.1 E1803D Easy Interface Functions“	`M700 1073741831`

Code	Description	Example
M701	Set the frequency for the laser in unit Hz. This command is a place holder and has no effect when used with E1803D. Instead of that, the frequency has to be set lasertype-dependent via pulse length parameter of commands M715 (stand-by frequency) and M718 (mark or continuously running frequency)	`M701 25000`
M702	Set the laser power in unit 1/1000%. This command has to be used only in case of specific lasermodes. Resulting from the mode the effect is different: `E180X_LASERMODE_YAG` – used in case of an SPI-laser, the power-value is set at analogue output AOut0 `E180X_LASERMODE_MOPA` – used in case of an IPG-laser, the power-value is latched out at LP8 laserport	`M702 75555`
M703	Set laser-on-delay (at parameter A) and laser-off-delay (at parameter B) in unit usec.	`M703 A100 B200`
M704	Set jump-speed (at parameter A) and mark-speed (at parameter B) in unit mm/min. By default, when no value has been specified, a jump speed of 1000 mm/sec and a mark speed of 10 mm/sec is assumed.	`M704 A8936.592 B3000.00`
M705	Set jump-delay (at parameter A), mark-delay (at parameter B) and in-polygon-delay (at parameter C) in unit usec.	`M705 A500 B200 C2500`
M707	Set the working area left position (at parameter X), top position (at parameter Y) and depth position (at optional parameter Z) in unit specified with commands G70 or G71. Together with M709 this command specifies the valid working field, all positioning parameters handed over with e.g. G0 and G1 need to be located within this area. **PLEASE NOTE: this value should be set after the working area size is specified with command M709 as it makes use of the internal scale factor that is specified by M709!**	`M707 X110 Y100`
M708	Set jump-delay (at parameter A), mark-delay (at parameter B) and variable polygon-delay (at parameter C) in unit usec. Different to command M705 here the third parameter specifies a variable delay which applies only to polygons at maximum angle of 180°.	`M705 A500 B200 C2500`
M709	Set the working area width (at parameter X), height (at parameter Y) and depth (at optional parameter Z) in unit specified with commands G70 or G71. Together with M707 this command specifies the valid working field, all positioning parameters handed over with e.g. G0 and G1 need to be located within this area. By default, when no value has been specified, a working area size of 100x100x100 mm is assumed. **PLEASE NOTE: this value should be set as very first and prior to command M707 as it sets the internal scale factor that is used with all coordinate commands!**	`M709 X-55 Y55`
M710	IPG pulse length value in unit nsec. This command is for future use and currently does not have any effect.	`M710 10000`
M711	Set wobble distance in X-direction (at parameter X), Y-direction (at parameter Y) in unit specified by commands G70 or G71 and wobble frequency (at parameter C) in unit Hz. When all values are set to 0, wobble marking is disabled completely.	`M711 X2.5 Y2.5 C10000`

Code	Description	Example
M713	Set first pulse killer value (FPK) for YAG laser types using unit usec	`M713 10000`
M714	Set simmer value in unit 1/1000% for SPI laser types, the corresponding analogue voltage will be set at output AOut1	`M714 55000`
M715	Set standby-frequency (at parameter A) in unit Hz and standby-pulsewidth (at parameter B) in unit nsec.	`M715 A50000 B1000`
M717	Set waveform number for SPI laser types latched out at LP8 laser port.	`M717 17`
M718	Set laser frequency (at parameter A) in unit Hz and pulsewidth (at parameter B) in unit usec.	`M718 A50000 B10`
M719	Switch main oscillator on (1) or off (0). This command can be used together with SPI or IPG laser types prior to starting a mark operation to turn MO on or afterwards to turn it off. Required delays to turn on the MO are handled by E1803D internally.	`M719 1`
M720	This command is valid for CNC-operations only, that are invoked by the command "`cxcnc`". It flushes the currently collected CNC data and starts the related marking operation (if not already active). This command requires firmware version 20 or newer.	
M721	This command can be used to set custom default values for scanner movements. By default, X and Y axes are assumed to be centred within their working range and Z is assumed to be at the very bottom. At the very beginning, all movements with G0 or G1 commands will start at these positions causing a hard jump of the scanner when its default position is different. So using this command, an other default/starting position can be specified which fits to the default position of the scanner. This command has to be used after setting the working area scale value with M709, after setting the metric working area offset with M707 and prior the very first usage of G0/G1. This command requires firmware version 22 or newer.	`M721 Z25.0`

19.4 Supported "T"-codes

Following "T" codes and identifiers are supported by E1803D G-Code interpreter:

Code	Description	Example
T1	Set jump or mark speed to be used with commands G0 and G1 in unit mm/min. Here it depends on a previous M-command if this value specifies the mark-speed (M3) or the jump speed (M5).	`T1 F6000.0`

19.5 Control Protocol

While the G-Codes, its syntax and meaning are standardised, the communication protocol that is used to evaluate the current state of the CNC-like operation, is not defined. Here several different control standards and protocols exist that are not compatible to each other. So this controller makes use of a very common, slim and easy to interpret control protocol: the one which is known from the GRBL free software. Due to the different type of controller, there are some changes which are described here.

Following commands can be sent in between a stream of G-code data (different to GRBL they all have to be completed with a CR/LF in order to let the client software transmit the command to the controller properly):

- `?` - check the current state of processing. Here a string is returned which makes use of following syntax:

  `<STATE|WPos:x,y,z|Pn:0x00000000>`

  Here state can be “Idle” (when no marking operation is in progress), “Trig” when a marking process is active but the controller waiting for an external trigger signal at the ExtStart-input, “Hold” when a marking process is active but the execution was held by the “!” control command or “Run” when a marking operation is in progress;

  WPos is an optional value which returns the current/last position which was emitted to the scanhead at the X, Y and Z channels (using format 0.00);

  Pn is an optional value which returns the state of the digital inputs at the digital interface(s); this format is slightly different to the GRBL variant, here a hexadecimal number is given which represents the input pattern while GRBL lets one single character appear for each known input

- `!` - halt the current marking operation at the next suitable position, means the next time when the laser is turned off regularly; when this control command is used, the marking state changes from “Run” to “Hold”

- `~` - continue a marking operation which was held with the “!” control command

- `cstat` – similar to the `cstat`-command as described at “18.2 Stand-Alone Control Commands”, this command returns the current marking state as textual representation making use of the syntax of the `cstat`-command as described above

20 Programming Interfaces

The libe1803.dll/libe1803.so shared library provides an own programming interface that gives the possibility to access and control the E1803D scanner controller card.

Beside of that e1803inter.dll / libe1803inter.so come with some compatibility interfaces for different other scanner cards like Scanlab™ RTC4™ and SCAPS™ USC1/2. They can be used to access E1803D scanner card with existing software easily. To do that, the "e1803inter" shared library just has to be renamed to the library name of the original vendor (like RTC4DLL.DLL, libslrtc4.so or sc_optic.dll). In order to operate E1803D card with different than the default connection settings, recompilation of code to be reused is necessary. Here at the very beginning and before initialisation a call to `E180X_set_connection()` has to be added in order to specify the connection to the card. Apart from this single case, none of the different programming interfaces should be mixed.

Sources of e1803inter.dll/libe1803inter.so are open and available for free, they can be found in public GIT repository (available at https://sourceforge.net/p/oapc/code/ci/master/tree/).

20.1 E1803D Easy Interface Functions

The following functions belong to the native programming interface of E1803D scanner card and should be used preferential in order to get access to all features and full performance of the scanner card. Header files and additional data required for using this interface can be found online in our public GIT-repository at https://sourceforge.net/p/oapc/code/ci/master/tree/libe1803/. There at https://sourceforge.net/p/oapc/code/ci/master/tree/libe1803/libe1803_test/ also an example application can be found that demonstrates the usage of the API.

Functions of E1803D Easy Interface are either stream commands that are executed in the order they are called, or functions that are executed immediately.

The E1803D does NOT use the concept of two or more lists that have to be managed and switched by the calling application. Here all stream commands simply are sent to the card without the need to provide some additional management information. Output of data is started only when `E180X_execute()` is called or when a card-internal threshold is exceeded. This card-internal triggered output of data can be held back by calling function `E180X_set_trigger_point()` as very first so that marking starts only after an external trigger signal was detected by the card or when the trigger point was released out of software by calling `E180X_release_trigger_point()`.

E1803D Easy Interface uses unit “bits” as base for all units and parameters. Since E1803D card internally uses 26 bits resolution for a better accuracy and to minimize round-off errors, all calculations are done with these 26 bits. So the working area always has a size of 26 x 26 bits equal to 67108864 x 67108864. Independent from real resolution and output of hardware all calculations have to be done within this 26 bit range.

A typical flow of commands that makes use of the programming interface to output data at the scanner controller card looks like this:

- `E180X_set_connection()` → specifies how to connect with the scanner controller card but does not open the connection yet; this function returns an instance identifier which has to be used for all following function calls
- `E180X_load_correction()` → opens the connection to the scanner controller card (as specified in previous step) and optionally loads a correction table for the scanhead
- send scanner and laser configuration parameters
- send marking data via `E180X_jump_abs()` and `E180X_mark_abs()`
- `E180X_execute()` → ensure marking really starts, this function typically to be called once at the end of every complete marking cycle only to ensure the data are really flushed to the controller
- `E180X_get_card_state()` → call this function in a loop to ensure marking really has started by checking one of the flags `E180X_CSTATE_MARKING` or `E180X_CSTATE_PROCESSING` are set

- `E180X_get_card_state()` → call this function in a loop to ensure marking really has ended by checking both flags `E180X_CSTATE_MARKING` and `E180X_CSTATE_PROCESSING` are not set any more
- `E180X_close()` → close the connection to the controller card and release all related resources

A fully functional example application that shows how to use the programming interface is available in our public GIT-repository at https://sourceforge.net/p/oapc/code/ci/master/tree/libe1803/libe1803_test/.

E1803D Easy Interface provides following functions:

20.1.1 General functions

This section describes all general functions related to accessing the scanner controller card, starting and stopping operation of, checking the current operational state and other things more which are necessary to control flow of data and commands,

`unsigned char E180X_set_connection(const char *address)`

This function has to be called as very first. It is used to specify the IP address where the card is accessible at (in case of Ethernet connection) or the serial interface (in case of USB connection, "COMx" for Windows and "/dev/ttyACMx" for Linux where "x" is the number of its interface). By default IP 192.168.2.254 is used. This is the only function that has to be called in case of both, when compatibility functions and when the E1803D easy function interface is used.
It returns a card index number that has to be used with all following functions (this is true for Easy Interface and RTC-compatible functions).
PLEASE NOTE: calling this function does not open the connection to scanner controller card! This is done on first call to `E180X_load_correction()`!

Parameters:
`address` – a char-array containing the IP in xxx.yyy.zzz.aaa notation or the name of the serial interface (COMx or /dev/ttyACMx) to be used

Return: the board instance number or 0 in case of an error

`void E180X_set_password(const char n,const char *ethPwd)`

Sets a password that is used for Ethernet connection of E1803D card. The same password should be configured on E1803D configuration file e1803.cfg with parameter "`passwd`" to add an additional level of security to an Ethernet controlled card.

PLEASE NOTE: usage of this password does NOT provide enough security to control the card via networks that are accessible by a larger audience, publicly or via Internet! Also when this password is set, the card always should operate in secured, separated networks only!
Every card and every connection should use an own, unique password that can consist of up to 48 characters containing numbers, lower- and upper-case letters and punctuation marks. Due to compatibility reasons no language-specific special characters should be used.
When connected via USB serial interface, this password is ignored. In this case no authentication is done.

Parameters:
`ethPwd` – the password to be used to authorise at an E1803D card. To reset a local password for connecting to a card that doesn't has an Ethernet password configured, hand over an empty string "" here

`void E180X_close(unsigned char n)`

Closes the connection to a card and releases all related resources. After this function was called, no more commands can be sent to the card until `E180X_set_connection()` and `E180X_load_correction()`/`n_load_correction_file()`/`load_correction_file()`/`ScSCIInitInterface()` is called again.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`

int E180X_set_debug_logfile(const unsigned char n,const char *path,const unsigned char flags)

This function can be used during development to check an own application regarding called commands and their parameters. It lets libe1803 write all function calls into a logfile so that it is possible to evaluate the real order of commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`path` – full path to the file which has to be used as debug log file
`flags` – a bunch of OR-concatenated flags which specify what function calls have to be written into or filtered from the log output; when 0x00 is specified here, the log file is kept quite small. When 0x01 is set, all motion-related function calls are added too, when 0x02 is set, all calls which check the state of the card are added to the log file.

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

int E180X_set_sync(const unsigned char n,const unsigned int flags,const unsigned int value)

This function sends a synchronisation `value` to the controller. As soon as marking reaches the related position in stream, the value returned by function `E180X_get_sync()` changes to the value given here.
This command delays execution of the data by 0,5 usec, so it should not be used excessively. A value of 0xFFFFFFFF disables this function.
This function requires firmware version 3 or newer.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – currently unused, set to 0 for future compatibility
`value` – the value to be used as sync-identifier, here on every call a different value should be handed over in order to differentiate what is returned by `E180X_get_sync()`.

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

unsigned int E180X_get_sync(const unsigned char n)

Returns a sync-identifier as set by `E180X_set_sync()` as soon as the related position in stream was reached.
This function requires firmware version 3 or newer.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`

Return: the last sync-identifier which was identified and processed in stream of commands or 0xFFFFFFFF when function is not used/turned off

int32_t E180X_execute(const uint8_t n, const uint32_t flags)

Starts execution of all previously sent commands in case card is not already outputting these data. This function should be called typically once as soon as all vector data have been sent to the controller. It should be called in every sequence of commands, even when the controller is already marking to terminate the sequence of vectors which have been submitted. This call does not necessarily start the marking operation as the controller is free to decide to do this at an earlier point in time but it ensures a proper transmission of all data from control-PC to the controller. To exactly define when marking has to start, function `E180X_set_trigger_point()` should be used.
When parameter `flags` is set to 0, the function works asynchronously, means after flushing the marking data it returns immediately. In this case, the calling application has to check for the marking states “marking” and “idle” to find out if marking is completed (by using function `E180X_get_card_state()`).

When parameter `flags` is set to `E180X_COMMAND_FLAG_SYNC`, the function blocks, until marking has completed or has been stopped. In this case no further calls to `E180X_get_card_state()` are necessary, as the card state can be assumed to be "idle" when the function returns.
In general and independent from this function, marking is finished only when STOP (ExtStop signal input or `E180X_stop_execution()`) is invoked or when the internal buffer is empty. When internal buffer runs empty because subsequent data are not sent fast enough, an additional call to `E180X_execute()` is necessary in order to output the remaining data.
This is not a stream command since it controls the already sent stream of commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_stop_execution(unsigned char n)`

Stops the currently running execution as fast as possible and drops all data and commands that still may be queued. Calling this function also would drop all laser and scanner parameters that are already sent to the controller but not yet processed. Thus after calling this function it may be necessary to set scanner and laser parameters again in order to ensure they are used for following operations.
This is not a stream command since it controls the current stream of commands.
PLEASE NOTE: this function should not be called on the off-chance "to be sure nothing is running". The command works asynchronously and causes a state-change which can have some unwanted side-effects when it is used without a specific reason. So calling the function should be done only when the controller is really in state marking/running, and after calling it, no other functions have to be used until the state (`E180X_get_card_state()`) has changed back to idle. As the stop is done as fast as possible, the point, at what the stream is stopped, is undefined. Means any function that has been called between the last call to `E180X_execute()` and `E180X_stop_execution()` may not have an effect and probably needs to be repeated.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_halt_execution(unsigned char n,unsigned char halt)`

Halts or continues the processing and output of marking data. On `halt=1` marking is stopped next time the laser is turned off. Different to a full stop, no vector data are dropped. On continue (`halt=0`) controller continues processing at the point where halt occurred. When marking is stopped with `E180X_stop_execution()` the halt-condition is cleared too, means on next transmission of new marking data they are processed without the need to explicitly continue last operation.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`halt` – 1 to halt operation next time the laser is off, 0 to continue a previously halted operation

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_set_trigger_point(unsigned char n)`

Specifies a point in data stream where execution has to stop until an external trigger signal (mark start) or a manual release of this trigger point is detected. This expects a rising edge on ExtStart input or calling of function `E180X_release_trigger_point()`.
This is a stream-command, means it is executed at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_release_trigger_point(unsigned char n)`

This function should be called only when a previous call to `E180X_set_trigger_point()` was done. It acts like an external trigger signal, releases the waiting condition and lets the controller start processing. So this function provides some kind of software-simulated external start-signal.



ATTENTION: this command will not arrive at the controller when there is no more space left on it, means when all controller-internal buffers are filled. So after a call to `E180X_set_trigger_point()` and during sending of commands and data to the controller, application has to ensure there is some space left in controller's buffers. This can be done by calling `E180X_get_free_space()` with flag `E180X_FREE_SPACE_PRIMARY` for checking the available space in primary buffer. It is recommended to leave space for at least 10000 elements in primary buffer in order to let a call to `E180X_release_trigger_point()` work properly.
When the buffers already have been filled completely, this function will no longer work and marking can be started only by applying the ExtStart hardware signal.
This is not a stream-command, it is applied to controller immediately.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`unsigned int E180X_get_startstop_state(unsigned char n)`

This function returns a bit pattern that informs about state of the start and stop input pins.
This is not a stream command since it returns the current state immediately. Here "current state" means the last known state. When the state changes during this call, it may be possible the previous, no longer actual state is given back since transmission of data from controller to host is done asynchronously and independent from a call to this function.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`

Return: a bit pattern specifying the current state:

- bit 0 and 1 (0x00000003) specify if the start input was set after last call of this function, when these bits are set, a rising edge has been detected at this input; calling this function resets the internal state of these bits, means when it is called again and when no new rising edge has been detected meanwhile, these bits will be 0
- bit 2 and 3 (0x0000000C) specify if the stop input was set after last call of this function, when they are set, a rising edge has been detected at this input; calling this function resets the internal state of these bits, means when it is called again and when no new rising edge has been detected at top input meanwhile, these bits will be low
- bit 12 (0x00001000) this bit signals the start input is low, as long as this bit is set no start input signal is detected

`int E180X_get_card_state(const unsigned char n,unsigned int *state)`

This function returns a bit pattern that informs about cards current operational state. Here "current state" means the last known state. When the state changes during this call, it may be possible the previous, no longer actual state is given back since transmission of data from controller to host is done asynchronously and independent from a call to this function.
The card-states are enqueued internally in order to not to lose a "busy"-state which may be available for a very short time only in case of very small and fast marking cycles. So every state change caused by the calling application results in on state change returned by this function. This means for every marking cycle the application has to wait for two state changes: first wait until this function signals "busy" (`E180X_CSTATE_MARKING|E180X_CSTATE_PROCESSING`), next wait until this function signals "ready" (0).
During transfer of vector data and scanner/laser parameters this function should be called as rarely as possible: every call of `E180X_get_card_state()` performs a fully cycle of transmission and receiving of data to and

from the controller. Dependent on the current transmission state this may result in submission of a small block of data which does not uses the full available bandwidth. On excessive use of this function this can slow down the whole transfer of data.
This is not a stream command, it returns the current state immediately.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`state` – pointer to a variable where the card state has to be written to: a bit pattern of or-concatenated constants specifying the current state:

- `E180X_CSTATE_MARKING` – card is currently marking
- `E180X_CSTATE_PROCESSING` – card has received some data that are enqueued for marking
- `E180X_CSTATE_WAS_START_PRESSED` – the ExtStart input was triggered, this flag is cleared after it has bean read and is set again only when ExtStart was triggered again
- `E180X_CSTATE_WAS_STOP_PRESSED` – the ExtStop input was triggered, this flag is cleared after it has bean read and is set again only when ExtStop was triggered again
- `E180X_CSTATE_FILE_WRITE_ERROR` – this flag is returned only in case stand-alone data are written to the microSD card and in case an file error occurs during this procedure. As writing an EPR file is done as asynchronous stream, errors during this procedure are not announced by the functions which are called but only by this error state. For more information about writing of stand alone data please refer to section “20.1.8 Writing of stand-alone data”
- `E180X_CSTATE_WAIT_EXTTRIGGER` – the controller is in state “marking” but is not yet processing any data as it is waiting for an external trigger
- `E180X_CSTATE_HALTED` – the controller is in state “marking” but is not yet processing any data as it is currently halted by function `E180X_halt_execution()`
- `E180X_CSTATE_WAIT_INPUT` – the controller is in state “marking” but is not yet processing any data as it is waiting for a specific input pattern at the digital inputs
- `E180X_CSTATE_SAC_READY` – this flag applies only to stand-alone modes; it is similar to output DOut0 and signals the controller has loaded a stand-alone file and is ready for marking
- `E180X_CSTATE_SAC_MARKING` – this flag applies only to stand-alone modes; it is similar to output DOut1 and signals the controller is marking a loaded EPR file
- `E180X_CSTATE_SAC_CTLXY` – this flag applies only to stand-alone modes; it signals a “`ctlxy`” command was received and the related mode is active

When the function returns an error code instead of `E180X_OK`, this value is undefined and can't be used.

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_delay(unsigned char n, double delay)`

Pause marking for the given time/wait for execution of the next command in stream for the given time. This is a stream-command, means it is executed at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`delay` – time to wait until marking continues in unit usec, smallest possible value is 0,500 usec

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_get_free_space(unsigned char n,int buffer)`

This function returns the space (in unit "commands") that is free in one of the buffers of E1803. Here parameter `buffer` decides which buffer has to be checked.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`buffer` – expects a constant which decides what buffer has to be checked, it has to be set to one of the following values:

- `E180X_FREE_SPACE_PRIMARY` – return size of the primary buffer; it can be used to avoid memory on host system is filled which may happen when vector data are sent to the controller while it's internal buffers are already full. In this case these data would have been stored on host side consuming some memory there. Using this function this problem can be avoided by sending commands only in case this function returns a value that is (much) larger than 0.
  The primary buffer that can be checked by using this value is one of two available buffers on E1803D controller. The primary one has a size of 1 million and is used to feed the secondary buffer (with a size of 20 million). So when this function returns 1000000, this does not mean the buffer is empty and no vector data currently processed – they still may be stored in secondary buffer. So to check the operational state of the controller, only function `E180X_get_card_state()` can be used.
  This buffer has also to be checked when function `E180X_release_trigger_point()` is used in order to ensure the command can arrive at the controller. For a detailed description please refer to explanation of `E180X_release_trigger_point()` above.
- `E180X_FREE_SPACE_SECONDARY` – return size of the secondary buffer; this one is filled by data from primary buffer and contains raw commands (like single micro vectors that concatenate to a full vector during output).

Return: -1 in case the function failed or the amount of free space in primary buffer.

`void E180X_get_version(unsigned char n, unsigned short *hwVersion, unsigned short *fwVersion)`

Get the hardware and software version of the used board. It is recommended to call this function after successful connect always and to check if used hardware and firmware version is at least a version that is known to work with own software.
This is not a stream command, it is executed immediately and independent from all other commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`hwVersion` – pointer to a variable where the hardware revision/version number is written into
`fwVersion` – pointer to a variable where the revision/version number of the firmware running on the board is written into

`int E180X_get_library_version()`

Returns an integer value which is an identifier specifying the version of this shared library. In decimal notation this identifier uses format "Mmmrrr" where "M" is the major version, "m" the minor version number and "r" the release count. The bigger the whole returned number is, the newer the library is.

`int E180X_get_serial_number(const unsigned char n,char *serial,const int length)`
Reads the serial number of the used board and returns it as 7 bit ASCII data.
This is not a stream command, it is executed immediately and independent from all other commands.
This function requires a firmware version 12 or newer.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`serial` – pointer to a char-array where the serial number has to be stored into, this memory area needs to have a size of at least 40 bytes
`length` – available length of the memory area where `serial` points to

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

int E180X_ana_read(const unsigned char n,const unsigned int flags,unsigned short *a)

Read a value from one of the analogue inputs. Here the parameter `flags` decides which output has to be accessed in which way:

- `E180X_COMMAND_FLAG_ANA_AIN0` – read data from analogue input AIn0
- `E180X_COMMAND_FLAG_ANA_AIN1` – read data from analogue input AIn1
- `E180X_COMMAND_FLAG_ANA_AIN2` – read data from analogue input AIn2

The value returned in `a` is always a 16 bit value in range 0..65535 independent from the real resolution of the hardware.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – flags specifying when the command has to be executed and which analogue output has to be set
`a` – value read from the analogue input, independent on the real resolution of the hardware, here always a 16 bit value is returned, means a value of 65535 would correspond to full input voltage of 5V
Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

This function requires firmware version 4 or newer and the Multi-IO expansion board or the Intelli-IO expansion board.

20.1.2 Laser and scanner related functions

This section describes all functions which are related to the scanhead and laser control and therefore have influence on the signals at the laser interface and the XY2-100 connector. The related interfaces are described in section „7.7 Laser Signals“ and section „7.6 Scanner Signals“.

int E180X_load_correction(unsigned char n, const char* filename, unsigned char tableNum)

Opens connection to the card and loads a correction file to be used during vector data output. In case a previously loaded correction table has to be flushed and no other correction has to be used, parameter "`filename`" needs to be empty.
This function has to be called for first time on initialisation and before any vector data are sent to the board. It is mandatory to call this function at least once since it establishes connection to E1803D card. So when no correction file has to be used this function still has to be called but with an empty filename “”.
This function supports different correction table file formats directly and without previous conversion:

- BeamConstruct .bco high resolution files
- Scanlab .ctb and .ct5 files
- SCAPS .ucf files
- Raylase .gcd files
- Rofin .fcr files
- CTI/GSI .xml files
- Sunny .txt 5x5 point correction files
- Han’s .crt files

This is not a stream-command, means its data may be applied immediately and independent from current stream state.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`filename` – the full path to the correction file to be loaded from file system, when "" is specified here, a previously used correction file is flushed and no/neutral correction is used as long as no other correction table is given
`tableNum` – the 0-based correction table number these data have to be loaded for; it is possible to download up to 16 different correction tables and to switch between them during operation using function `E180X_switch_correction()`

Return: `E180X_OK` or an `E180X_ERROR_`- or RTC-compatible return code in case of an error

int E180X_switch_correction(unsigned char n, unsigned char tableNum)

Switches between up to 16 correction tables on the fly. When a table-number is given where no file was downloaded before using function `E180X_load_correction()`, no correction is performed on all following vector data.
This is a stream-command, means the new correction is applied to vector data sent to the card after this command but NOT to already sent but not yet processed data. Thus on-the-fly switching between correction tables is possible.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`tableNum` – the 0-based table number of the correction that has to be used for all following vector data

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

int E180X_set_xy_correction(const unsigned char n,const unsigned int flags const double gainX, const double gainY,const double rot,const int offsetX,const int offsetY,const double slantX,const double slantY)

Sets size correction factor and offset for X and Y direction of working area as well as a rotation.
This function will overwrite all corrections specified with `E180X_set_matrix()`.
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – an optional set of OR-concatenated flags which specify the correction/modification of the output further. `E180X_COMMAND_FLAG_XYCORR_FLIPXY` exchanges the X and Y coordinates and therefore flips the output, `E180X_COMMAND_FLAG_XYCORR_MIRRORX` mirrors the X-coordinates, `E180X_COMMAND_FLAG_XYCORR_MIRRORY` mirrors the output in Y-direction. If none of these additional corrections is required, set `flags` to 0.
`gainX` – scale factor in x-direction, 1.0 means no scaling
`gainY` – scale factor in y-direction, 1.0 means no scaling
`rot` – rotation of whole working area in unit degrees
`offsetX` – offset in x-direction in unit bits, 0 means no offset
`offsetY` – offset in y-direction in unit bits, 0 means no offset
`slantX` – trapezoidal correction along X-axis in range -45..45°
`slantY` – trapezoidal correction along Y-axis in range -45..45°

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

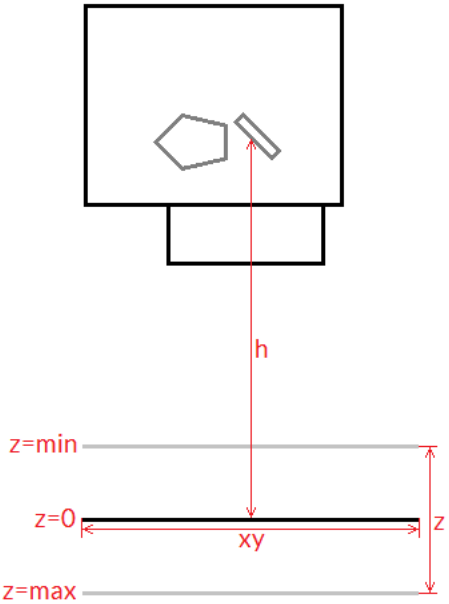
int E180X_set_z_correction(const unsigned char n,const int flags,const double gainZ,const int offsetZ,const unsigned int h,const double xy_to_z_ratio)

Set additional Z correction parameters.
This function may be used to adjust the Z-axis and in cases where third axis is used with a large Z working range for an additional Z-focus-correction. Here additional deviation occurs when no F-Theta lens is used caused by the fact that the beam is always sent from the centre of the scanhead – which causes some kind of projection resulting in larger or smaller X and Y positions depending on the real Z height. This deviation can be adjusted by an internal Z-focus-correction.
Beside of that, the Z-position can be adjusted via a gain factor and by using an offset, shifting it into a specific direction
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands and changed values apply only to these vector data and coordinates, which are sent after calling this function.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`

flags – optionally set the flag E180X_COMMAND_FLAG_ZCORR_MIRRORZ to invert the orientation of the Z-axis, for normal scanhead working in correct direction, this value can be left at 0
gainZ – scales the Z axis by the given factor, when set to 1.0, no scaling is applied to the Z-coordinates
offsetZ – shifts the Z-position into the given direction (using unit bits), when set to 0, no offset is applied to the Z-coordinates
h – the vertical height from last mirror of the scanhead to the working area (Z-position 0 of working area) in unit bits, when set to 0, the Z-focus-correction is disabled
xy_to_z_ratio – factor specifying the ratio between maximal horizontal working area size and maximal vertical movement size. As an example: when the working area has a size of 100 x 100 mm and the Z-axis has a maximum movement range of -20 mm .. 20 mm, the ratio to be set is 2,5 (100 mm horizontal divided by 40 mm vertical); when this value is set to a ratio equal or smaller than 0.0, the Z-focus-correction is disabled

For more details about the Z-focus correction applied by setting the parameters h and xy_to_z_ratio, please refer to the image below:



Here "h" is the height from the position where the beam hits the last mirror to the position of the working area at z=0 position (in unit bits). "xy" is the width of the working area to be used together with the "z" range from "z=min" to "z=max" to calculate the xy_to_z_ratio. All working area parameters like its width "xy" and the "z"-range are expected to be the theoretical maximum of the full range, not the – possibly smaller – range used in a specific setup.

Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_set_speeds(unsigned char n, double jumpspeed,double markspeed)

Set scanner speed values to be used for all following vector data and until not replaced by other speed values.
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands. So values set here apply only to these vector data that are sent after this command.

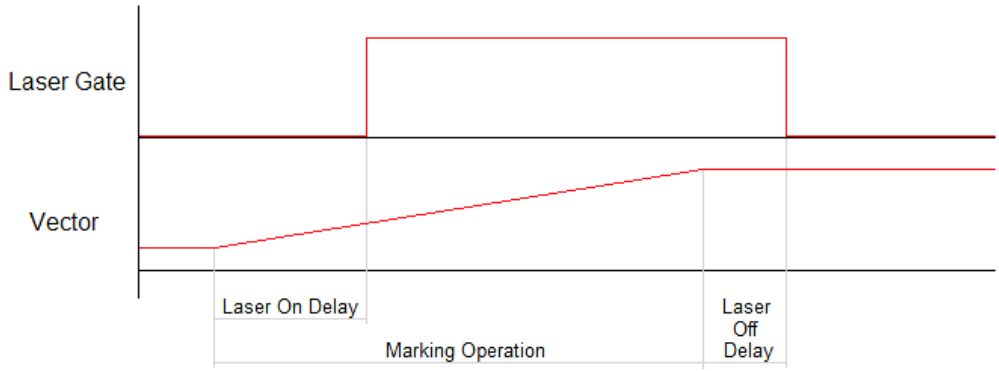
Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
jumpspeed – scanner movement speed during jumps (movements when laser is off) in unit bits/msec and range 1..4294960000
markspeed – scanner speed during mark (movements when laser is on) in unit bits/msec and range 1..4294960000

Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_set_laser_delays(unsigned char n,double ondelay,double offdelay)

Set laser delay values to be used for all following vector data and until not replaced by other delay values.
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands. So values set here apply only to these vector data that are sent after this command.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`ondelay` – laser on delay in unit microseconds, can be a negative or a positive value
`offdelay` – laser off delay in unit microseconds, must be a positive value



Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_set_scanner_delays(const unsigned char n,const unsigned int flags,const double jumpdelay,const double markdelay,const double polydelay)`
Set scanner delays in unit microseconds. Smallest possible value and resolution is 0.5 microseconds.
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands. So values set here apply only to these vector data that are sent after this command.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – here some flags can be set which add some further functional specifications and features to this function. At the moment following flags are supported and can be OR-concatenated with each other:
- `E180X_COMMAND_FLAG_SCANNER_VAR_POLYDELAY` – when this flag is set, the value set via `polydelay` is not applied statically to every point within a polygon, but it is set dynamically depending on the angle between two lines; no angle (a straight line) results in no delay while an 180 degree angle results in a full delay as set by value `polydelay`; this flag requires firmware version 2 or newer

`jumpdelay` – the jump delay value in unit microseconds
`markdelay` – the mark delay value in unit microseconds
`polydelay` – the in-polygon delay value in unit microseconds

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_set_scanner_mode(const unsigned char n, const unsigned int mode)`
Sets the operation mode for the scanner, this function influences the communication mode with the scanhead.
This function requires firmware version 10 or newer.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`mode` – one of the following values can be set:
- `E180X_SCANNERMODE_XY2_100` – standard and default mode, output is normal XY2-100 with 16 bit resolution and normal 100 kHz output cycle clock
- `E180X_SCANNERMODE_XY2_200` – output is XY2-200 with 16 bit resolution and 200 kHz output cycle clock

- `E180X_SCANNERMODE_XY2_100E` – output is extended XY2-100E with 18 bit resolution and normal 100 kHz output cycle clock
- `E180X_SCANNERMODE_XY2_200E` – output is extended XY2-200E with 18 bit resolution and 200 kHz output cycle clock
- `E180X_SCANNERMODE_SL` – requires the " NX-02 Extension Board" and enables output of scanner control data using a two-wire serial protocol with 100 kHz output cycle clock; when this mode is enabled, the connector of the SL Extension Board is used while the main D-SUB25 connector is turned off

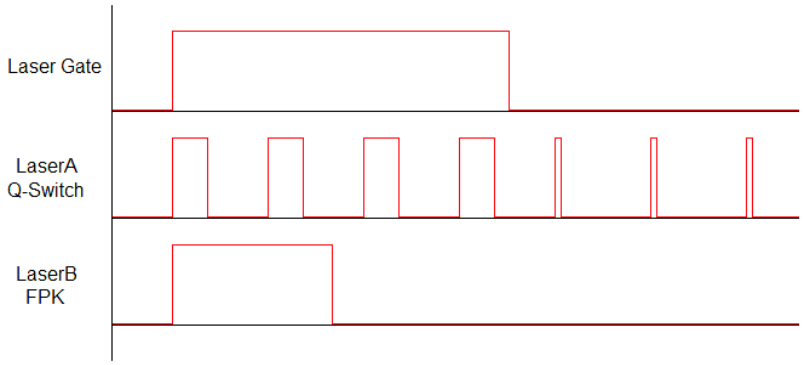
Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_set_laser_mode(unsigned char n, unsigned int mode)`

Sets the laser mode to be used for all following operations, this value influences the signals emitted at the connectors of the card. This function has to be called prior to setting any other laser parameters (like frequency, standby-frequency, power).
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands.

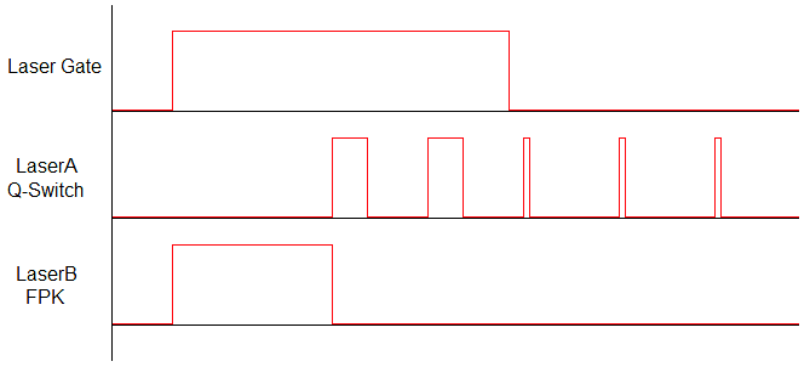
Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`mode` – the laser mode, here one of the following values is possible:

- `E180X_LASERMODE_CO2` – for controlling CO_2 lasers, this mode supports stand-by frequency at LaserA output (to be set with function `E180X_set_standby()`) and PWM-modulated frequencies during marking and for power control (to be set with function `E180X_set_laser_timing()`)
- `E180X_LASERMODE_YAG1` – for controlling YAG lasers, this mode supports stand-by and Q-Switch frequency at LaserA output (to be set with function `E180X_set_standby()`) and a first pulse killer signal at output LaserB that is issued on beginning of a mark together with the Q-Switch frequency (to be set with function `E180X_set_fpk()`):



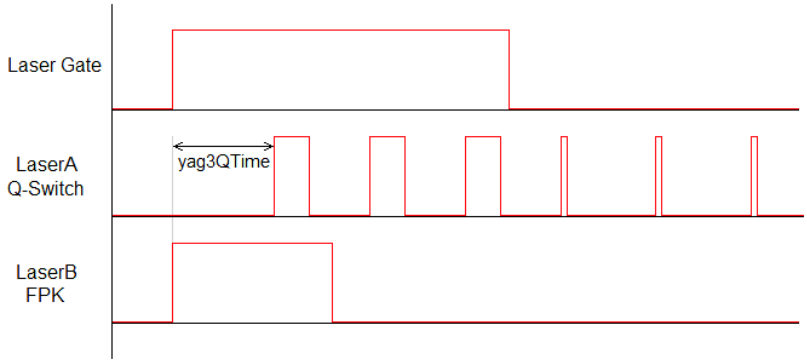
Here Q-Switch signal is started together with laser gate and FPK pulse. At end of mark when laser gate is turned off stand-by frequency is emitted at LaserA.

- `E180X_LASERMODE_YAG2` - for controlling YAG lasers, this mode supports stand-by and Q-Switch frequency at LaserA output (to be set with function `E180X_set_standby()`) and a first pulse killer signal at output LaserB that is issued on beginning followed by Q-Switch frequency that starts when FPK pulse has finished:



Here FPK and laser gate are started together. Q-Switch signal is started at end of FPK pulse. At end of mark when laser gate is turned off, stand-by frequency and pulse-width is emitted at LaserA instead of Q-Switch frequency.

- `E180X_LASERMODE_YAG3`– for controlling YAG lasers, this mode supports stand-by and Q-Switch frequency at LaserA output (to be set with function `E180X_set_standby()`) and a first pulse killer signal at output LaserB that is issued on beginning followed by Q-Switch frequency that starts after a freely configurable time period "yag3QTime":



Here FPK and laser gate are started together. Q-Switch signal is started after yag3QTime has elapsed according to the beginning of FPK pulse. This time value can be set using function `E180X_set_fpk()`. At end of mark when laser gate is turned off, stand-by frequency and pulse-width is emitted at LaserA instead of Q-Switch frequency.

- `E180X_LASERMODE_CRF` – for controlling lasers that require a continuously running frequency (like fiber-lasers), this frequency is emitted at LaserA output and can be set and changed by calling function `E180X_set_standby()`.
- `E180X_LASERMODE_DFREQ` – for controlling special lasers that require two frequencies, the second, continuously running frequency is emitted at LaserB output and can be set with function `E180X_set_laserb()`
- `E180X_LASERMODE_MOPA` – for fiber lasers which are driven by a main oscillator and power amplifier and that are power-controlled via LP8 digital port and latch bit

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_set_laser(const unsigned char n,const unsigned int flags,const char on)`

Switches the laser on or off independent fro many mark or jump commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – handling flags specifying the behaviour of this command, `E180X_COMMAND_FLAG_STREAM` to use it as stream command, `E180X_COMMAND_FLAG_DIRECT` to execute it immediately and independent on current stream and execution state; in case `E180X_COMMAND_FLAG_STREAM` is used, please ensure this function call is followed by other stream commands, elsewhere the laser is turned off for security reasons as soon as no more data are available to process in order to not to let the laser fire while the card is waiting
`on` – set to 1 to turn the laser on or to 0 to turn it off

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_set_wobble(unsigned char n,unsigned int x,unsigned int y,double freq)`

This function gives the possibility to not to let the laser beam follow the given path directly but to rotate around the specified path and lasers current position. Depending on chosen wobble-parameters and marking speed this results either in a thick or a sinusoidal line. This call sets wobble parameters to be used for all following vector data and until not replaced by other wobble values or by 0 which disables wobble mode. This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`x` – wobble amplitude in x direction in units bits and range 1..10000000
`y` – wobble amplitude in y direction in units bits and range 1..10000000
freq – wobble frequency in Hz in range 1..25000

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_jump_abs(unsigned char n,int x,int y,int z)`

Perform a jump (movement with laser turned off) to the given position. This causes a galvo movement from current position to the one specified by this functions parameters using the jump speed and taking the jump delay into account:



When laser was turned on before this function is called, laser is turned off at the beginning with a delay specified by laser off delay (please refer to description of `E180X_mark_abs()` for a diagram showing laser off delay too).
This is a stream-command, means it is executed at a point in stream that is relative to the other stream commands. So values set here apply only to these vector data that are sent after this command.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`x` – the x-coordinate in unit bits the scanner has to jump to (in range -33554431..33554432)
`y` – the y-coordinate in unit bits the scanner has to jump to (in range -33554431..33554432)
`z` – the z-coordinate in unit bits the scanner has to jump to (in range -33554431..33554432, requires a hardware that is equipped with Z- channel)

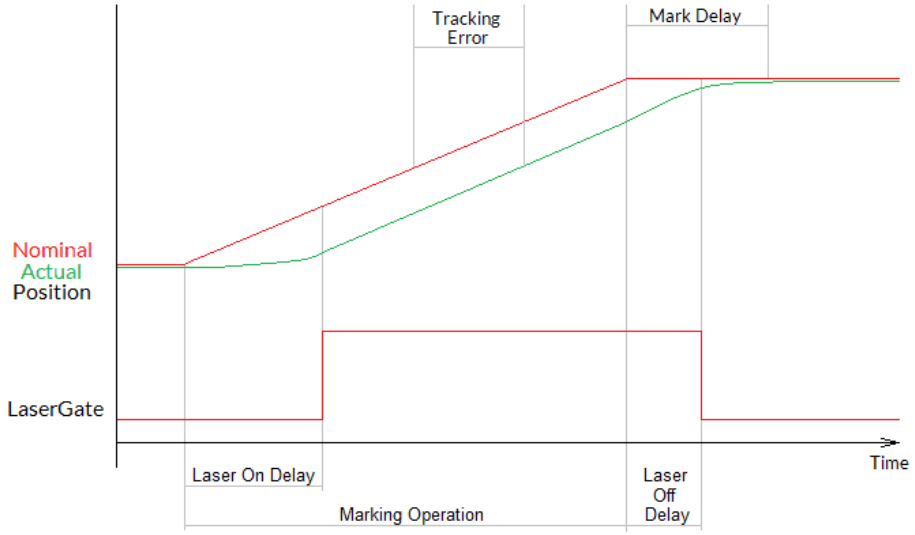
Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_jump_abs_list(const unsigned char n,int *x,int *y,int *z,const uint32_t length)`

This function is similar to `E1803X_jump_abs()` but it burst-transmits an array of jump positions to be used. For the parameters `x`, `y` and `z` same-length arrays are expected which contain the related coordinates. The additional parameter `length` specifies the number of indices the array consists of.
This function requires firmware version 22 or newer and software version 8.4 or newer.

`int E180X_mark_abs(unsigned char n,int x,int y,int z)`

Perform a mark (movement with laser turned on) to the given position. This causes a galvo movement from current position to the one specified by this functions parameters using the mark speed and taking the mark delay into account. When laser was turned off before this function is called, laser is turned on at the beginning with a delay specified by laser on delay:



This is a stream-command, means it is executed at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`x` – the x-coordinate in unit bits the scanner has to move to (in range -33554431..33554432)
`y` – the y-coordinate in unit bits the scanner has to move to (in range -33554431..33554432)
`z` – the z-coordinate in unit bits the scanner has to move to (in range -33554431..33554432, requires a hardware that is equipped with Z- channel)

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_mark_abs_list(const unsigned char n,int *x,int *y,int *z,const uint32_t length)`

This function is similar to `E1803X_mark_abs()` but it burst-transmits an array of mark positions to be used. For the parameters `x`, `y` and `z` same-length arrays are expected which contain the related coordinates. The additional parameter `length` specifies the number of indices the array consists of.
This function requires firmware version 22 or newer and software version 8.4 or newer.

`int E180X_get_pos(const unsigned char n,int *x,int *y,int *z)`

This function returns the last position of the scanner:

- when an XY3-100 scanhead is connected, that provides its actual position, these values are returned
- when no such scanhead is connected, the last nominal position sent to the head are returned

The function needs the controller to be in idle-mode, means no marking operation is allowed to run in order to get the position information.
Please note: when a correction file is set, and/or a matrix is set and/or an offset is set and/or any other function is used which modifies the position data, the coordinates returned here are **not** the values which have been sent with the last call to `E180X_jump_abs()` or `E180X_mark_abs()` or `E180X_set_pos()` as they have been

processed and modified by these correction functions. So while the jump/mark functions set position data according to the desired coordinate system, the values returned by `E180X_get_abs()` are the real-world coordinates at the hardware.
This function requires firmware version 16 or newer.

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_set_pos(unsigned char n,int x,int y,int z,unsigned char laserOn)`

Perform a raw, immediate movement to the given position.



HANDLE WITH CARE! This function causes galvo movement to the given position immediately, without respect to any mark or jump speed values, without micro-vectorisation or intermediate steps! This means it can result in a very heavy movement for the galvos and in worst case it may cause some damage! Since the resulting movement speed may be way too high for the used galvos, they may overshoot and need some time until the desired position is reached. So this function is mainly intended to be used for very small position changes in respect to the galvos current position.
This is a stream-command, means it is executed at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`x` – the x-coordinate in unit bits the scanner has to jump to (in range -33554431..33554432)
`y` – the y-coordinate in unit bits the scanner has to jump to (in range -33554431..33554432)
`z` – the z-coordinate in unit bits the scanner has to jump to (in range -33554431..33554432, requires a hardware that is equipped with Z- channel)
`laserOn` – specifies if the movement has to be done with laser turned on (1) or off (0)

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_set_pixelmode(const unsigned char n,const unsigned int mode,const double powerThres,const unsigned int res)`

Set the operational mode for `E180X_mark_pixelline()`. This function influences the behaviour when marking a pixel line. This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`mode` – pixel marking mode, this parameter can be set to:

- 0 – default mode, while marking a pixel line the controller tries to perform jumps when power is below of the given threshold `powerThres` to save marking time
- `E180X_PIXELMODE_NO_JUMPS` – no jumps are performed, the given power threshold is ignored and the full pixel line is done with marking speed; this mode is slower but can result in more accurate and more exact images
- `E180X_PIXELMODE_JUMP_N_SHOOT` – marking of the line is no longer done with a continuous movement but with a sequence "jump to position → shoot → jump to next position → shoot → jump to next position → shoot..."; here the shoot-time is equal to the laser-off-delay minus laser-on-delay as set with function `E180X_set_laser_delays()`
- `E180X_PIXELMODE_HW_POWER_CONTROL` – when this flag is set, the controller card takes care about setting the power for the pixels. This works only when a lasermode is chosen where the scanner card supports native power control. When this flag is set, a power-callback, handed over together with a call to `E180X_mark_pixelline()` is ignored.
- `E180X_PIXELMODE_GATE_POWER_CONTROL` – this is a special bitmap marking mode where no real power control is supported. When this flag is set, the LaserGate output is toggled depending on the required output power. Since this output supports only states LOW and HIGH, this bitmap marking mode results in black and white images only
- `E180X_PIXELMODE_JUMP_LEAVE_POWER` – during bitmap marking, when no flag `E180X_PIXELMODE_NO_JUMPS` is set, below of a specific power threshold a jump is performed. By default, prior to such a jump, the laser power is set to 0 to handle faultily lasers

that have spurious emissions even when LaserGate is at LOW. For laser types, which do not suffer from such emissions, this flag can be set. It leaves the last power value active also during jumps, which saves some marking time. So this flag can be used for speed-optimising bitmap-marking.

`powerThres` – this value is used only in default mode, when the marking power for some pixels is below of the given value (in unit percent), a jump is performed to save marking time, during this jump the laser is off and no marking is done

`res` – reserved, set always to 0

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_mark_pixelline(const unsigned char n,int x,int y,int z,const int pixWidth,const int pixHeight,const int pixDepth,unsigned int pixNum,const double *pixels,E180X_power_callback power_callback,void *userData)`

This function can be used to mark a single line of a bitmap image. Here horizontal, vertical and even 3D bitmap lines (going into depth) can be marked. Direction and orientation of the line to be marked can be chosen freely. A full image can be created by concatenating several lines. Power control during marking of such a bitmap line is not limited to some specific power outputs, it can be fully customised via a callback function.

Parameters:

`n` – the 1-based board instance number as returned by `E180X_set_connection()`

`x, y, z` – the starting coordinates of the line in unit bits

`pixWidth` – the width of a single pixel (in unit bits), when this is set to a value greater or smaller than 0 while all the others are equal 0, a horizontal line is drawn; the sign of the value specifies the marking direction

`pixHeight` – the height of a single pixel (in unit bits), when this is set to a value greater or smaller than 0 while all the others are equal 0, a vertical line is drawn; the sign of the value specifies the marking direction

`pixDepth` – the depth of a single pixel (in unit bits, requires a 3D-capable scanhead), when this is set to a value greater or smaller than 0 while all the others are equal 0, line goes into depth; the sign of the value specifies the marking direction

`pixNum` – the number of pixel data contained in the array of intensity values handed over with the following parameter

`pixels` – an array of double-values with a length equal the number of pixels specified with `pixNum` and with an allowed range of 0.0..100.0 specifying the intensity; every entry of this array is equal to one pixel of the bitmap, so a greyscale-pixelline with brightness values in range 0..255 has to be converted to values in range 0.0..100.0

`power_callback` – this is a callback function of type

```
int (*E180X_power_callback)(unsigned char n, double power, void *userData)
```

which is used to set the power for every pixel. There these `E180X_`-functions have to be called that belong to the used laser type and set the power values according to it's hardware capabilities. Within the power callback function only stream commands are allowed to be called. It is not possible to use external devices that are not synchronous to E1803D command stream. The power callback has to return with `E180X_OK` when setting of power was successful. In case of an error the appropriate error code has to be returned, the pixel marking function will be cancelled in such a case too and does not finish marking of the line. Parameter `n` is the 1-based board instance number specifying the board the power has to be changed for, power is the power to be set in unit percent and `userData` are some free to use, custom data that can be handed over on call to `E180X_mark_pixelline()`.

`userData` – here some custom data can be handed over which are forwarded on and handed over at every call of the power-callback

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_set_matrix(unsigned char n, double m11, double m12, double m21, double m22)`

Specify a 2x2 matrix that contains scaling and rotation corrections for the output. When a given matrix element parameter has a value smaller or equal -10000000 it is ignored and the previous/default value is kept at this position in matrix.
This function will overwrite all corrections specified with `E180X_set_xy_correction()`.
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`m11` – first matrix element in first row
`m12` – second matrix element in first row
`m21` – first matrix element in second row
`m22` – second matrix element in second row

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`unsigned int E180X_get_head_state(const unsigned char n,const unsigned int flags)`

Returns head status information in case the connected scanhead provides such data via STATUS signal of XY2-100 interface. When the head does not provide such information or returns invalid data or a proprietary data format, the function returns 0xFFFFFFFF. Otherwise the returned value can be AND-concatenated with `HEAD_STATE_MASK` to find out what kind of head is connected: a resulting value of `HEAD_STATE_2D_HEAD` identifies a 2D scanhead, `HEAD_STATE_3D_HEAD` a 3D scanhead. Depending on this, the returned value contains the following state information:

Bit	`HEAD_STATE_2D_HEAD`	2D Head Remarks	`HEAD_STATE_3D_HEAD`	3D Head Remarks
19 / C2	0	Identification bit	0	Identification bit
18 / C1	1	Identification bit	0	Identification bit
17 / C0	1	Identification bit	1	Identification bit
16 / S15	Power state		X servo ready	
15 / S14	Temperature state		X temperature state	
14 / S13	In-field		X tracking error	
13 / S12	X-position ACK		0	
12 / S11	Y-position ACK		Y servo ready	
11 / S10	1		Y temperature state	
10 / S9	0		Y tracking error	
9 / S8	1		0	
8 / S7	Power state		Z servo ready	
7 / S6	Temperature state		Z temperature state	
6 / S5	In-field		Z tracking error	
5 / S4	X-position ACK		0	
4 / S3	Y-position ACK		X channel parity error	
3 / S2	1		Y channel parity error	
2 / S1	0		Z channel parity error	
1 / S0	1		CLK channel error	
0 / Par	x	Always 0	Parity bit (even)	

The exact usage of these fields depends on the used head, so for further details please refer to the related scanhead manual.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – when flag E180X_COMMAND_FLAG_HEAD_STATE_RAW is set here, the state-information from the head are returned as they are received. When this flag is not used, the returned data are checked and filtered – only in case they fit to the bit patterns shown above, the received data are returned, elsewhere the error information 0xFFFFFFFF is given back.

Return: the received (filtered or raw) XY2-100 state data as received from the head or 0xFFFFFFFF in case of an error

int E180X_set_laser_timing(unsigned char n, double frequency, double pulse)

Set the frequency and pulse-width to be used during marking at LaserA output of laser connector.
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
frequency – emitted frequency in unit Hz and in range 25..20000000 Hz
pulse – pulse width in usec, this value has to be smaller than period length that results out of frequency

Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_set_standby(const unsigned char n,const double frequency,const double pulse, const bool force)

Set the frequency and pulse-width to be used during jumps, as stand-by frequency or as continuously running frequency at LaserA output of laser connector.
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
frequency – emitted frequency in unit Hz and in range 25..20000000 Hz. When a value of 0 is given, the frequency at LaserA output is turned off at end of mark.
pulse – pulse width in usec, this value has to be smaller than period length that results out of frequency
force – when set to true, the new stand-by frequency is not applied the next time the laser is turned off, but immediately

Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_set_laserb(const unsigned char n,const double frequency,const double pulse)

Set the frequency and pulse-width to be used at LaserB output of laser connector. To use LaserB as second frequency output, a laser mode with flag E180X_LASERMODE_DFREQ has to be configured.
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
frequency – emitted frequency in unit Hz and in range 25..20000000 Hz
pulse – pulse width in usec, this value has to be smaller than period length that results out of frequency

Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_set_fpk(unsigned char n, double fpk, double yag3QTime)

Set the parameters for first pulse killer signal that is emitted via laser connector whenever the laser is turned on; this applies to YAG-modes only and is emitted as one single pulse at LaserB output.
This is a stream-command, means its parameters are applied at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`fpk` – the length of the first pulse killer signal in usec
`yag3QTime` – the length of the first pulse killer signal in usec, this value is used only when laser mode `E180X_LASERMODE_YAG3` is set, elsewhere it is ignored

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_lp8_write(const unsigned char n,const unsigned int flags,const unsigned char value)`

Sets the LP8_0..LP8_7 outputs of 8 bit laser port of laser interface connector without touching the related latch output. Total execution time of this command is 1 usec.
Depending on the value of parameter flags this is either a stream-command (means it is executed at a point in stream that is relative to the other stream commands) or a direct command (means it is executed immediately on calling).

Parameters:
`n` – the 1-based board instance number as returned by E180X_set_connection()
`flags` – handling flags specifying the behaviour of this command, `E180X_COMMAND_FLAG_STREAM` to use it as stream command, `E180X_COMMAND_FLAG_DIRECT` to execute it immediately and independent on current stream and execution state
`value` – the 8 bit value to be set at LP8 port
Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_lp8_write_latch(unsigned char n, unsigned char on, double delay1,unsigned char value, double delay2,double delay3)`

Sets the LP8 8 bit laser port of laser interface connector with freely definable delays and toggles the related latch output automatically; calling this function causes the following sequence of commands:

- turn latch bit on/off
- wait for `delay1` usecs
- set LP8
- wait for `delay2` usecs
- turn latch bit off/on
- wait for `delay3` usecs

The whole execution time of this sequence is 1.5 usecs for setting LP8 outputs and toggling latch plus `delay1` plus `delay2` plus `delay3`.
This is a stream-command, means it is executed at a point in stream that is relative to the other stream commands.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`on` – specifies if the latch bit has to be set to HIGH (on=1) or LOW (on=0) on first step, on second step it will toggle to value `!=on`
`delay1` – delay to be issued after setting/clearing the latch bit for the first time
`value` – the 8 bit value to be set at LP8 port
`delay2` – delay to be issued after setting LP8 output and before clearing/setting the latch bit
`delay3` – delay to be issued after clearing/setting the latch bit for the second time

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

int E180X_lp8_write_mo(const unsigned char n, const unsigned int flags, const unsigned char on)

Sets the main oscillator output MO of laser interface connector to be used with e.g. fiber lasers. Depending on the value of parameter `flags` this is either a stream-command (means it is executed at a point in stream that is relative to the other stream commands) or a direct command (means it is executed immediately on calling).

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – handling flags specifying the behaviour of this command, `E180X_COMMAND_FLAG_STREAM` to use it as stream command, `E180X_COMMAND_FLAG_DIRECT` to execute it immediately and independent on current stream and execution state
`on` – the state the MO output has to be switched to; PLEASE NOTE: the main oscillator depends on the current internal state of the laser. Thus turning it on is always possible but turning off the MO is possible only when the controller is not yet handling the laser-off delay, means it is not possible as long as the laser is turned on. In such a case this command is ignored.

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

int E180X_lp8_write_pilot(const unsigned char n, const unsigned int flags, const unsigned char on)

Sets the pilot laser output pin of the laser interface connector.
Depending on the value of parameter `flags` this is either a stream-command (means it is executed at a point in stream that is relative to the other stream commands) or a direct command (means it is executed immediately on calling).
This function requires hardware revision 1.8 or newer and firmware version 19 or newer.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – handling flags specifying the behaviour of this command, `E180X_COMMAND_FLAG_STREAM` to use it as stream command, `E180X_COMMAND_FLAG_DIRECT` to execute it immediately and independent on current stream and execution state
`on` – the state the pilot laser output has to be switched to; PLEASE NOTE: the pilot laser signal may affect the internal state of the laser. Thus turning it on may prevent other functions of the laser being able to be used. For details about such dependencies, please check the manual of the used laser.

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

int E180X_ana_write(const unsigned char n,const unsigned int flags,const unsigned short a)

Write a value to one of the analogue outputs. Here the parameter `flags` decides which output has to be accessed in which way:

- `E180X_COMMAND_FLAG_ANA_AOUT0` – send data to analogue output AOut0
- `E180X_COMMAND_FLAG_ANA_AOUT1` – send data to analogue output AOut1

One of these flags can be combined with `E180X_COMMAND_FLAG_STREAM` to use it as stream command or with `E180X_COMMAND_FLAG_DIRECT` to execute it immediately and independent on current stream and execution state. Parameter `a` needs to be always a 16 bit value independent from the real resolution of the hardware.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – flags specifying when the command has to be executed and which analogue output has to be set
`a` – value to be set at the analogue output, independent on the real resolution of the hardware, here always a 16 bit value has to be given, means a value of 65535 would correspond to full output voltage of 10V
Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

20.1.3 Digital interface functions

The following section describes all functions which can be used to describe data at the digital interface as described in section „6.8 Digital Interface“ and section „6.8.1 Marking On-The-Fly Signals“.

`int E180X_digi_write(const unsigned char n,const unsigned int flags,const unsigned int value,const unsigned int mask)`

Sets the 8 bit digital output port.

Depending on the value of parameter `flags` this is either a stream-command (means it is executed at a point in stream that is relative to the other stream commands) or a direct command (means it is executed immediately on calling).

Parameters:

`n` – the 1-based board instance number as returned by `E180X_set_connection()`

`flags` – handling flags specifying the behaviour of this command, `E180X_COMMAND_FLAG_STREAM` to use it as stream command, `E180X_COMMAND_FLAG_DIRECT` to execute it immediately and independent on current stream and execution state

`mask` – specifies which of the bits in "value" have to be used for setting and clearing output data, only these bits that are set to 1 in `mask` are changed according to the given `value`

`value` – the 8 bit value to be set at digital out port

Return: `E180X_OK` or an `E180X_ERROR_` return code in case of an error

`int E180X_digi_pulse(const unsigned char n, const unsigned int flags, const unsigned int in_value, const unsigned int mask, const unsigned int pulses, const double delayOn, const double delayOff)`

Send a sequence of pulses to the 8 bit digital interface. When the controller works with a firmware version 5 or later, this operation causes nearly no data transmission load.

This command is available as stream-command only (means it is executed at a point in stream that is relative to the other stream commands).

Parameters:

`n` – the 1-based board instance number as returned by `E180X_set_connection()`

`flags` – currently only `E180X_COMMAND_FLAG_STREAM` is supported here

`mask` – specifies which of the bits in "value" have to be used for setting and clearing output data, only these bits that are set to 1 in `mask` are changed according to the given `value`

`value` – the 8 bit value to be set at digital out port

`pulses` – specifies how often the output has to be set/cleared

`delayOn` – the delay (in unit usec) which has to be issued every time after setting the output, the minimal resolution of this value is 0,5 usec

`delayOff` – the delay (in unit usec) which has to be issued every time after clearing the output, the minimal resolution of this value is 0,5 usec

Return: `E180X_OK` or an `E180X_ERROR_` return code in case of an error

`int E180X_digi_read(const unsigned char n,const unsigned char flags,unsigned int *value)`

Reads the 8 bit digital input port.

This is not a stream-command, means it is executed immediately and returns current state of the digital inputs. When parameter `flags` is set to 0, the state of the digital inputs is requested actively which results in a separate data transmission to the controller card. On excessive use of this command, that may slow down communication with the controller dramatically. Alternatively `flags` can be set to `E180X_COMMAND_FLAG_PASSIVE` which does not cause such a request. Instead of this the last known state of the digital inputs is returned by this function based on the last regular feedback from the controller or based on the last call to this function with this passive-flag not set. So when this flags is used, the returned `value` may be several hundred milliseconds old.

When marking on the fly is enabled using function `E180X_digi_set_motf()`, digital inputs 0 and 1 (and optionally also digital inputs 2 and 3 in case of 2D marking on-the-fly) are used for MOTF-encoder and

therefore not available as standard inputs. In such a case state of these bits is undefined and does not reflect the current input state caused by the external encoder.

Parameters:
`n` - the 1-based board instance number as returned by `E180X_set_connection()`
`value` – pointer to a variable where the current digital input state has to be written into.
When the function returns an error code instead of `E180X_OK`, this value is undefined and can't be used.

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_digi_wait(unsigned char n,unsigned long value,unsigned long mask)`

Stop execution and output of data until the given bitpattern was detected at digital inputs of digital interface connector. Here parameter `mask` specifies which of the bits at the input have to be checked, they have to be set to 1. These bits within `mask` that need to be ignored have to be set to 0. Parameter `value` itself defines the states of the bits that has to be detected at the input to continue processing of data. All bits of value that correspond to bits of mask, that are 0, are ignored.

Parameters:
`n` - the 1-based board instance number as returned by `E180X_set_connection()`
`value` – the expected bitpattern at digital input
`mask` – specifies which of the input bits and value bits have to be used for comparison

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_digi_set_motf(const unsigned char n,const unsigned int flags,const double motfX,const double motfY)`

Disables or enables marking on-the-fly functionality and specifies factors for X- and Y-direction. When this function is called with values for `motfX` or `motfY` greater than 0, marking on-the-fly is enabled and digital inputs 0 and 1 of the digital interface are no longer available as general purpose inputs. Now they are used as decoder inputs for a 90 degree phase shifted encoder signal for marking on-the-fly functions. When both parameters `motfX` and `motfY` are set to 0, marking on-the-fly is disabled and inputs 0 and 1 no longer work as encoder inputs.
When tune flag "2" is set (for 2D marking on-the-fly, please refer to description of e1803.cfg parameters above), the two factors for X and Y are assigned to separate encoder inputs. Here factor for X applies to values received on digital inputs 0 and 1 and factor for Y applies to values received on digital inputs 2 and 3.
Depending on value of parameter `flags` this is or is not a stream-command, means it switches states of digital inputs 0 and 1 (plus optionally 2 and 3) and marking on-the-fly functionality at the related position in stream or immediately.
Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – when flag `E180X_COMMAND_FLAG_DIRECT` is set, the new MOTF-factors are applied immediately, when flag `E180X_COMMAND_FLAG_STREAM` is used instead, the command acts as stream-command and sets the new MOTF-factors as soon as this command is due in current stream of command.
`motfX` – marking on-the-fly factor for X-direction in unit bits per encoder increment
`motfY` – marking on-the-fly factor for Y-direction in unit bits per encoder increment
Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_digi_set_motf_sim(unsigned char n, double motfX, double motfY)`

Disables or enables simulated marking on-the-fly functionality and specifies factors for X- and Y-direction. When this function is called with values for `motfX` or `motfY` greater than 0, simulated marking on-the-fly is enabled and internal 100 kHz signal generator is used to create static marking on-the-fly pulses in positive direction. A possibly enabled on-the-fly operation using external signals on digital inputs 0 and 1 of digital interface connector is disabled. When both parameters `motfX` and `motfY` are set to 0, marking on-the-fly is disabled completely.
This is not a stream-command, means it enables simulated marking on-the-fly functionality immediately.
Parameters:

n – the 1-based board instance number as returned by E180X_set_connection()
motfX – marking on-the-fly factor for X-direction in unit bits suitable for to be simulated movement-speed on 100 kHz encoder counting frequency
motfY – marking on-the-fly factor for Y-direction in unit bits suitable for to be simulated movement speed on 100 kHz encoder counting frequency
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_digi_wait_motf(const unsigned char n, const unsigned int flags, const double dist)

Halts the current marking operation for a given distance of the on-the-fly encoder. Different to E180X_delay() this function does not use a time to wait until marking is continued but a distance specified by parameter dist and measured by the connected encoder. To use this function marking on-the-fly has to be enabled by calling E180X_digi_set_motf() or E180X_digi_set_motf_sim() before.

This command is useful for applications where several vector data have to be marked which in total are larger than the available working area (e.g. marking long texts on a cable). For this the vector data to be marked have to be concatenated in suitable pieces where each is smaller than the available working area. Then these pieces can be marked consecutively with following sequence of commands:

1. E180X_set_trigger_point() (used only once at the very beginning to define the starting point and to initialise internal MOTF counters). This trigger point later has to be released either by applying an ExtStart signal or by calling function E180X_release_trigger_point()
2. E180X_digi_wait_motf() to wait for the beginning of the first piece of vector data to be marked, the given distance is equal to the distance from the starting point in 1)
3. E180X_jump_abs()/E180X_mark_abs() for vector data of character to be marked; here one piece of the whole set of vector data has to be sent to the controller
4. E180X_digi_wait_motf() to wait for the distance until next set of vector data; here the distance between the starting points of two pieces of vector data has to be given
5. continue at 3) until all pieces of vector data have been sent

This is a stream-command, means it is executed at a point in stream that is relative to the other stream commands.
Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – specifies how the distance value is handed over, with E180X_COMMAND_FLAG_MOTF_WAIT_INCS a value in unit “encoder increments” is expected, with E180X_COMMAND_FLAG_MOTF_WAIT_BITS a distance in unit “bits” is expected. In second case the X-on-the-fly factor of a preceding call to E180X_digi_set_motf() or E180X_digi_set_motf_sim() is used.
dist – the distance to wait for until marking has to be completed, the unit of this value is specified with preceding parameter flags
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_digi_set_motf_powerctl(const unsigned char n,const unsigned int flags,const double motfSpeed,const double lowValue, const double highValue)

Perform a marking on-the-fly speed-dependent power adjustment. This function uses the currently set power as well as the handed over nominal speed to calculate the real power based on the real marking on the fly speed. So to use this functionalities, following sequence of commands is necessary:
- set the on-the-fly factors/enable marking on the fly by calling E180X_digi_set_motf()
- set the current power, here dependent on the used laser type the appropriate function has to be called
- call E180X_digi_set_motf_powerctl() to define automatic power adjustment parameters
This function requires firmware version 7 or newer.
Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – specifies the method of power control/the power control output which has to be used. Here exactly one of the following flags can be used:
- E180X_COMMAND_FLAG_ANA_AOUT0 – to use analogue output A0 for power control
- E180X_COMMAND_FLAG_ANA_AOUT1 – to use analogue output A01for power control

- E180X_COMMAND_FLAG_FREQ_LASERA – this is not really a power control method but enables the option to change the frequency of LaserA output while the pulse-pause-ratio of the related waveform is kept constant
motfSpeed – nominal speed (in unit bits/sec) which corresponds to the current nominal power set in previous call, when the real, measured marking on-the-fly speed rises above this value, the related power output is adjusted to also ensure a higher power output, when the actual, measured MOTF-speed becomes lower than the nominal motfSpeed, power is adjusted to lower values
lowValue – lower clipping value, the automatically adjusted power will never become smaller than this
highValue – upper clipping value, the automatically adjusted power will never become bigger than this
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_digi_set_mip_output(unsigned char n,unsigned int value,unsigned int flags)

This function can be used to specify which of the digital outputs has to be used for signalling "marking in progress". When value is set to 0xFFFFFFFF, this function is disabled and scanner controller card does not provide this signal automatically. When the number of the digital output (in range 0..7) is given as value, the related digital output pin is used for "mark in progress" signal.



PLEASE NOTE: here the number (means the count) of one specific output pin has to be given, not a bitpattern specifying one or more pins!
During operation the selected "mark in progress" pin is HIGH as long as the scanner is moving and/or the laser is on and/or a delay is processed and when marking parameter are processed between these operations. It becomes LOW as soon as no more marking data are available and current operation is stopped or when scanner is waiting for an external trigger signal (ExtStart).
This is not a stream-command, when it is called it is applied to current configuration immediately.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
value – the number of the digital output to be used for this signal
flags - currently unused, set always to 0 for compatibility
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_digi_set_wet_output(const unsigned char n,const unsigned int value,const unsigned int flags)

This function can be used to specify which of the digital outputs has to be used for signalling "waiting for external trigger". When value is set to 0xFFFFFFFF, this function is disabled and scanner controller card does not provide this signal automatically. When the number of the digital output (in range 0..7) is given as value, the related digital output pin is used for "waiting for external trigger" signal.



PLEASE NOTE: here the number (means the count) of one specific output pin has to be given, not a bitpattern specifying one or more pins!
During operation the selected "waiting for external trigger" pin is HIGH as long as the controller is waiting for an external trigger to be applied at ExtStart input. It becomes LOW as soon as this signal has been detected or when current operation is stopped.
This is not a stream-command, when it is called, it is applied to current configuration immediately.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
value – the number of the digital output to be used for this signal
flags – currently unused, set always to 0 for compatibility

Return: E180X_OK or an E180X_ERROR_-return code in case of an error

20.1.4 Serial interface functions

Following functions are described which can be used to access the serial interface as described in section “7.9 Serial Interface”

int E180X_uart_write(const unsigned char n,const unsigned int flags,const char *sendData,const unsigned int in_length,unsigned int *sentLength)

Send data to RS232/RS485 serial interface using the serial interface parameters which are configured in e1803.cfg configuration file. This command is executed depending on the given flags, means the data are sent to the serial interface independent from the context the function has to be called within or in context of the current stream.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – here pairs of flags a) and b) have to be OR-concatenated with each other to specify how to transmit the data exactly:

a) `E180X_COMMAND_FLAG_UART1` – this flag specifies which UART interface has to be used for transmitting data, at the moment only one UART interface exists, so only this flag can be set and has always to be set

b) `E180X_COMMAND_FLAG_STREAM` – when this flag is combined with the flag from a), data are enqueued to the current stream and transmitted depending on its position within that stream

b) `E180X_COMMAND_FLAG_DIRECT` – when this flag is combined with the flag from a), data are transmitted immediately (means as fast as possible and not within the regular stream of data)

b) `E180X_COMMAND_FLAG_ASYNC` – when this flag is combined with the flag from a), data are transmitted asynchronously (means as soon as the next package of data is sent to the controller which either is done on `E180X_execute()` or when enough other data are handed over, on arrival at the controller these data are sent to the UART interface immediately and not within the regular stream of data); in other words, this is a specific variant of `E180X_COMMAND_FLAG_DIRECT` where data are transmitted immediately but the controller doe not wait for the response to arrive

`sendData` – pointer to byte-array which contains the data which have to be sent
`in_length` – length of the data in `sendData`
`sentLength` – pointer to a variable where the amount of data is returned which really has been sent by this function; when a different return code than `E180X_OK` is given back, this value is undefined.
Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

int E180X_uart_read(const unsigned char n,const unsigned int flags,char *recvData,const unsigned int maxLength,unsigned int *receivedLength)

Receive data from RS232/RS485 serial interface using the serial interface parameters which are configured in e1803.cfg configuration file. This command is executed immediately, means it checks for data arrived at serial interface independent from the context the function has to be called within.

Parameters:
`n` – the 1-based board instance number as returned by `E180X_set_connection()`
`flags` – here pairs of flags a) and b) have to be OR-concatenated with each other to specify how to receive the data exactly:

a) `E180X_COMMAND_FLAG_UART1` – this flag specifies which UART interface has to be used for receiving of data, at the moment only one UART interface exists, so only this flag can be set
b) `E180X_COMMAND_FLAG_DIRECT` – when this flag is combined with the flag from a), data are tried to be read immediately, this means the function does not return until some data could be read from the serial interface or until a timeout occurred
b) `E180X_COMMAND_FLAG_ASYNC` – when this flag is combined with the flag from a), data are received asynchronously, this means when no data are available on call of the function, it returns immediately with `E180X_OK` and setting `receivedLength` to 0. In such a case the function has to be called later again in order to receive some data. As soon as some data have arrived, `receivedLength` specifies the size of these data on return of the function

`recvData` – pointer to byte-array where received data have to be stored into, this buffer should have a size of at least 513 bytes
`maxLength` – maximum number of bytes the buffer specified by `recvData` is able to store

receivedLength – pointer to a variable where the amount of data is returned which really has been received by this function; when a different return code than E180X_OK is given back, this value is undefined.
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

20.1.5 Intelli-IO extension functions (IO-mode)

Following functions require the Intelli-IO Extension Board as described in section “11.2 Intelli-IO Interface in IO mode”. They can be used to access the digital IOs of this board. The analogue inputs of the Intelli-IO board are not directly subject to this extension and therefore can be read by general function E180X_ana_read() as described above. All functions described here require firmware version 5 or newer.

int32_t E180X_motion_digi_write(const uint8_t n, const uint32_t flags, const uint32_t in_value, const uint32_t mask)

Sets the second 8 bit digital output port which is located on the Intelli-IO extension.
Depending on the value of parameter flags this is either a stream-command (means it is executed at a point in stream that is relative to the other stream commands) or a direct command (means it is executed immediately on calling).
Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – handling flags specifying the behaviour of this command, E180X_COMMAND_FLAG_STREAM to use it as stream command, E180X_COMMAND_FLAG_DIRECT to execute it immediately and independent on current stream and execution state
mask – specifies which of the bits in "value" have to be used for setting and clearing output data, only these bits that are set to 1 in mask are changed according to the given value; this parameter requires a firmware version 12 or newer, elsewhere it is ignored and the function assumes a mask-value of 0xFFFFFFFF which results in an output bitpattern which is exactly the given value with all bits set/cleared
value – the 8 bit value to be set at digital out port

Return: E180X_OK or an E180X_ERROR_ return code in case of an error

int32_t E180X_motion_digi_read(const uint8_t n, const uint32_t flags, uint32_t *value)

Reads the second 8 bit digital input port which is located on the Intelli-IO extension. Since this extension board provides only six input bits, the upper two bits always will be 0.
This is not a stream-command, means it is executed immediately and returns current state of the digital inputs. When parameter flags is set to 0, the state of the digital inputs is requested actively which results in a separate data transmission to the controller card. On excessive use of this command, that may slow down communication with the controller dramatically. Alternatively flags can be set to E180X_COMMAND_FLAG_PASSIVE which does not cause such a request. Instead of this the last known state of the digital inputs is returned by this function based on the last regular feedback from the controller or based on the last call to this function with this passive-flag not set. So when this flags is used, the returned value may be several hundred milliseconds old.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
value – pointer to a variable where the current digital input state has to be written into.
When the function returns an error code instead of E180X_OK, this value is undefined and can't be used.

Return: E180X_OK or an E180X_ERROR_-return code in case of an error

20.1.6 Intelli-IO extension functions (motion mode)

Following functions require a Intelli-IO Extension Board as described in section “11.3 Intelli-IO Interface in motion mode” and can be used to control motion operations. Here command options that set a value or start a

motion are always available in two options: as stream (flag E180X_COMMAND_FLAG_STREAM is set) or as direct command (flag E180X_COMMAND_FLAG_DIRECT is set). Stream-commands are always executed in the order they are sent to the controller and the controller always waits until one motion has been finished before any other command in this stream is executed. Direct commands are executed immediately, here it is up to the user to wait until a motion operation has ended. This wait-operation is the same as for every other scanner operation: first one has to wait until operation started, next one has to wait until operation has ended. It is mandatory to always wait for both state changes.
All the functions described here require at least firmware version 6.

int E180X_motion_set_steps(const unsigned char n, const unsigned int flags,const double steps)

Set the factor which defines the relation between steps (increments) of the used stepper motor and the distance that it travels. This value needs to be specified prior to all other operations in order to allow correct calculation of all distances and speeds as expected by the other functions as described below. For the E1803 motion extension no default value exists, so if no factor is set, motion operations are done with an undefined, random value which may lead to unexpected results.
The E1803 motion extension programming interface always makes use of real distances (mostly in unit mm) and does not expect the calling application to do the conversion from increments to mm.
Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – command flags specifying the type of function call (E180X_COMMAND_FLAG_STREAM or E180X_COMMAND_FLAG_DIRECT) and for which axes the given values have to be applied (E180X_COMMAND_FLAG_AXIS_0, E180X_COMMAND_FLAG_AXIS_1, E180X_COMMAND_FLAG_AXIS_2, E180X_COMMAND_FLAG_AXIS_3)
steps – factor which defines relation between stepper motor steps and travel distance (in unit increments/mm for longitudinal movements or increments/degree for rotational movements)
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_set_pulsewidth(const unsigned char n, const unsigned char width)

Set the width of the setp-pulses to the given value in unit usec. By default this value is set to 10 usec which should fit for most applications. Setting it to smaller typically values does not make any sense, setting larger values may be necessary in case of some slo stepper motor drivers. In this case please note: larger pulse with values result in a smaller maximum step frequency as these large pulses may overlap each other in such cases and result in a permanent HIGH-signal instead of a step-frequency.
This function requires firmware version 11 or newer.
Parameters
n – the 1-based board instance number as returned by E180X_set_connection()
width – the new pulsewidth to be used in range 10..200 usec
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_set_limits(const unsigned char n,const unsigned int flags,const double llimit,const double hlimit,const unsigned double in_slimit)

Set motion limits for axis operations. When any follow-up command tries to set values beyond these limits, these values are clipped to the allowed range set with this function.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – command flags specifying the type of function call (E180X_COMMAND_FLAG_STREAM or E180X_COMMAND_FLAG_DIRECT) and for which axes the given values have to be applied (E180X_COMMAND_FLAG_AXIS_0, E180X_COMMAND_FLAG_AXIS_1, E180X_COMMAND_FLAG_AXIS_2, E180X_COMMAND_FLAG_AXIS_3)
llimit – lower motion limit (in unit mm or degrees)
hlimit – upper motion limit (in unit mm or degrees)
in_slimit – speed limit (in unit mm/sec or degrees/sec)
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_set_accel(const unsigned char n,const unsigned int flags,const double accel)

Set the acceleration to be used for start and stop for all motion operations and for the specified axes.

Parameters:

`n` – the 1-based board instance number as returned by `E180X_set_connection()`

`flags` – command flags specifying the type of function call (`E180X_COMMAND_FLAG_STREAM` or `E180X_COMMAND_FLAG_DIRECT`) and for which axes the given values have to be applied (`E180X_COMMAND_FLAG_AXIS_0`, `E180X_COMMAND_FLAG_AXIS_1`, `E180X_COMMAND_FLAG_AXIS_2`, `E180X_COMMAND_FLAG_AXIS_3`)

`accel` – acceleration (in unit mm/sec^2 or degrees/sec^2)

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

int E180X_motion_set_speed(const unsigned char n,const unsigned int flags,double speed)

Set the speed for the next motion operations and for the specified axes. Since all motions are combined movements where all axes start and stop at the same time, the speed value given here is some kind of recommendation which may not be used at the next motion operation. Here following rules apply:

- a speed value given here is never exceeded
- when only one axis is moved by a motion operation `E180X_motion_move_abs()` or `E180X_motion_move_rel()` at the same time, the speed value given here is used for this movement
- when more than one axis is moved by a motion operation `E180X_motion_move_abs()` or `E180X_motion_move_rel()` at the same time, the controller calculates speeds for all axes which ensure they all start and stop their movements at the same time

Parameters:

`n` – the 1-based board instance number as returned by `E180X_set_connection()`

`flags` – command flags specifying the type of function call (`E180X_COMMAND_FLAG_STREAM` or `E180X_COMMAND_FLAG_DIRECT`) and for which axes the given values have to be applied (`E180X_COMMAND_FLAG_AXIS_0`, `E180X_COMMAND_FLAG_AXIS_1`, `E180X_COMMAND_FLAG_AXIS_2`, `E180X_COMMAND_FLAG_AXIS_3`)

`speed` – motion speed (in unit mm/sec or degrees/sec)

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

int E180X_motion_move_abs(const unsigned char n,const unsigned int flags,const double pos0,const double pos1,const double pos2,const double pos3)

Start a motion operation to the given absolute positions using at maximum the speeds specified with `E180X_motion_set_speed()`.

Parameters:

`n` – the 1-based board instance number as returned by `E180X_set_connection()`

`flags` – command flags specifying the type of function call (`E180X_COMMAND_FLAG_STREAM` or `E180X_COMMAND_FLAG_DIRECT`) and for which axes the given values have to be applied (`E180X_COMMAND_FLAG_AXIS_0`, `E180X_COMMAND_FLAG_AXIS_1`, `E180X_COMMAND_FLAG_AXIS_2`, `E180X_COMMAND_FLAG_AXIS_3`); the additional flag `E180X_COMMAND_FLAG_DONOTWAIT` can be set to perform a movement parallel to other operations, for details please refer to description of function `E180X_motion_stream_wait()` below

`pos0` – absolute motion position for axis 0 (in unit mm or degrees); this value is used only when flag `E180X_COMMAND_FLAG_AXIS_0` is set

`pos1` – absolute motion position for axis 1 (in unit mm or degrees); this value is used only when flag `E180X_COMMAND_FLAG_AXIS_1` is set

`pos2` – absolute motion position for axis 2 (in unit mm or degrees); this value is used only when flag `E180X_COMMAND_FLAG_AXIS_2` is set

`pos3` – absolute motion position for axis 3 (in unit mm or degrees); this value is used only when flag `E180X_COMMAND_FLAG_AXIS_3` is set

Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_move_rel(const unsigned char n,const unsigned int flags,const double pos0,const double pos1,const double pos2,const double pos3)

Start a motion operation which changes the current axis position by the value specified here and by using at maximum the speeds specified with E180X_motion_set_speed().
Please note: in case of a direct operation it is mandatory to wait for the end of all previous motion operations before this function is called. Elsewhere the real current axis position is not known and a relative movement is not possible.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – command flags specifying the type of function call (E180X_COMMAND_FLAG_STREAM or E180X_COMMAND_FLAG_DIRECT) and for which axes the given values have to be applied (E180X_COMMAND_FLAG_AXIS_0, E180X_COMMAND_FLAG_AXIS_1, E180X_COMMAND_FLAG_AXIS_2, E180X_COMMAND_FLAG_AXIS_3); the additional flag E180X_COMMAND_FLAG_DONOTWAIT can be set to perform a movement parallel to other operations, for details please refer to description of function E180X_motion_stream_wait() below
pos0 – change the motion position for axis 0 (in unit mm or degrees) by the value given here; this value is used only when flag E180X_COMMAND_FLAG_AXIS_0 is set
pos1 – change the motion position for axis 1 (in unit mm or degrees) by the value given here; this value is used only when flag E180X_COMMAND_FLAG_AXIS_1 is set
pos2 – change the motion position for axis 2 (in unit mm or degrees) by the value given here; this value is used only when flag E180X_COMMAND_FLAG_AXIS_2 is set
pos3 – change the motion position for axis 3 (in unit mm or degrees) by the value given here; this value is used only when flag E180X_COMMAND_FLAG_AXIS_3 is set
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_move_abs_async(const unsigned char n,const unsigned int flags,const double pos0,const double pos1,const double pos2,const double pos3)

Start an asynchronous motion operation to the given absolute positions using at maximum the speeds specified with E180X_motion_set_speed(). Comparing to E180X_motion_move_abs(), this function:

- waits until a possibly running, previous motion operation at the same axes has completed
- starts this motion operation
- continues without waiting for the end of this motion, so other operations (except motions) can be executed in parallel

This command requires firmware version 15 or newer.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – command flags specifying the type of function call (currently only E180X_COMMAND_FLAG_STREAM is supported) and for which axes the given values have to be applied (E180X_COMMAND_FLAG_AXIS_0, E180X_COMMAND_FLAG_AXIS_1, E180X_COMMAND_FLAG_AXIS_2, E180X_COMMAND_FLAG_AXIS_3); the additional flag E180X_COMMAND_FLAG_DONOTWAIT can be set to perform a movement parallel to other operations, for details please refer to description of function E180X_motion_stream_wait() below
pos0 – absolute motion position for axis 0 (in unit mm or degrees); this value is used only when flag E180X_COMMAND_FLAG_AXIS_0 is set
pos1 – absolute motion position for axis 1 (in unit mm or degrees); this value is used only when flag E180X_COMMAND_FLAG_AXIS_1 is set
pos2 – absolute motion position for axis 2 (in unit mm or degrees); this value is used only when flag E180X_COMMAND_FLAG_AXIS_2 is set
pos3 – absolute motion position for axis 3 (in unit mm or degrees); this value is used only when flag E180X_COMMAND_FLAG_AXIS_3 is set
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_stream_wait(const unsigned char n)

The two motion functions E180X_motion_move_abs() and E180X_motion_move_rel() described above can be called with the command flag E180X_COMMAND_FLAG_STREAM set to ensure sequential operation within the normal stream of commands. In this mode processing of further commands is halted until the related motion operation has finished. Since the motion extension board uses an own micro-controller, here also parallel operations are possible: when for these motion functions the flag E180X_COMMAND_FLAG_DONOTWAIT is set together with E180X_COMMAND_FLAG_STREAM, operation does not wait but continues to process other data in stream. In such a case motion is performed in parallel to these operations done on the main laser controller.
This is true only for non-motion operations, before the next motion operation is called, the application has to wait until the previous one has been finished. Within a stream this has to be done by calling function E180X_motion_stream_wait(), it re-synchronises the stream with the parallel motion operation. So the rule is: as long as a motion operation was started with the combined flags E180X_COMMAND_FLAG_STREAM|E180X_COMMAND_FLAG_DONOTWAIT, a following call to E180X_motion_stream_wait() is mandatory before any other motion-function is used.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_stop(const unsigned char n)

Stop a currently running motion operation immediately. Since axes always perform a combined movement where all axes start and stop at the same time, the stop function always affects all axes which are moving. When E180X_stop_execution() is called instead, not only axis movements but also all other operations are stopped.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_get_pos(const unsigned char n,const unsigned char axisNum,double *pos)

Retrieves the current position of an axis. This command is always a direct command retrieving the current axis position. Thus it does not make use of command-flags.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
axisNum – the number of the axis (but not the axis-flags!) in range 0..3 where the current position has to be retrieved for
pos – pointer to a variable where the current position (in unit mm) of the axis with the number specified in axisNum has to be stored into; when the function does not return E180X_OK, this value is undefined and can't be used
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_reference(const unsigned char n,const unsigned int flags,const unsigned int mode,const double leaveDist,double speedStep0,double speedStep1)

Starts a referencing operation (=homing sequence) to have a defined position for the axis. The referencing sequence consists of following steps:

- move to reference switch (connected to reference-input) with first referencing speed speedStep0
- leave the reference switch by the given distance leaveDist
- move to reference switch (connected to reference-input) with second referencing speed speedStep1
- set the position of the referenced axis to -1 – this value can be used to check if referencing was successful or not, when E180X_motion_get_pos() returns a different value than -1 for the referenced axis, something went wrong and referencing failed

Parameters:

n – the 1-based board instance number as returned by E180X_set_connection()
flags – command flags specifying the type of function call (E180X_COMMAND_FLAG_STREAM or E180X_COMMAND_FLAG_DIRECT) and for which axes the given values have to be applied (E180X_COMMAND_FLAG_AXIS_0, E180X_COMMAND_FLAG_AXIS_1, E180X_COMMAND_FLAG_AXIS_2, E180X_COMMAND_FLAG_AXIS_3)
mode – specifies how referencing has to be done exactly, here a bunch of OR-concatenated flags can be handed over: one of the flags E180X_MOTION_REFSTEP_N (to search for the reference input in negative direction) or E180X_MOTION_REFSTEP_P (to search for the reference input in positive direction) which optionally can be combined with flag E180X_MOTION_REFSTEP_INV_SWITCH to have inverted logic on the reference input
leaveDist – distance (in unit mm or degrees) to move off the reference switch after the switch was found for the first time
speedStep0 – referencing speed (in unit mm/sec or degrees/sec) to find the reference switch for the first time (this value can be larger than speedStep1 but should be small enough to not to overrun the switch)
speedStep1 – referencing speed (in unit mm/sec or degrees/sec) to find the reference switch for the second time (this value should be smaller than speedStep0 and is responsible for the accuracy of the referenced position)
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

int E180X_motion_set_pos(const unsigned char n,const unsigned int flags,const double pos)

This function does not cause any movement but resets the current axis position(s) to a new value. It can be used e.g. after successful referencing to set the initial positions to some own values. All following movement operations then are done in respect to the position values given here.

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – command flags specifying the type of function call (E180X_COMMAND_FLAG_STREAM or E180X_COMMAND_FLAG_DIRECT) and for which axes the given values have to be applied (E180X_COMMAND_FLAG_AXIS_0, E180X_COMMAND_FLAG_AXIS_1, E180X_COMMAND_FLAG_AXIS_2, E180X_COMMAND_FLAG_AXIS_3)
pos – the new position value to be set for the specified axis/axes (in unit mm or degrees)
Return: E180X_OK or an E180X_ERROR_-return code in case of an error

20.1.7 Miscellaneous functions

int E180X_write(unsigned char n, unsigned int flags, unsigned int value)

Writes some specific data to outputs at E1803D controller. Here flags decides which output to use and value specifies what has to be written to this output. Additionally flags decides weather this is a stream-command (means it is executed at a point in stream that is relative to the other stream commands) or a direct command (means it is executed immediately on calling).

Parameters:
n – the 1-based board instance number as returned by E180X_set_connection()
flags – handling flags specifying the behaviour of this command, when E180X_COMMAND_FLAG_STREAM is set, it is used as stream command, E180X_COMMAND_FLAG_DIRECT specifies to execute it immediately and independent on current stream and execution state. Here exactly one of these flags can be used, it is not allowed to OR-concatenate them. Additionally exactly one of the following flags has to be set to specify which output need to be used to send the value to, this flag has to be OR-concatenated with one of the previously described ones:
E180X_COMMAND_FLAG_WRITE_LP8MO – set or unset MO-output of laser connector to a value of 1 or 0
E180X_COMMAND_FLAG_WRITE_LP8LATCH – set or unset latch-output of laser connector to a value of 1 or 0
E180X_COMMAND_FLAG_WRITE_LASERGATE – set or unset LaserGate-output to a value of 1 or 0, this functions should be used with jump or mark operations only since every switch from jump to mark (or vice

versa) still sets the LaserGate output automatically and therefore would overwrite own values set with this function
`value` – the value to be sent to the output specified by flags
Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

20.1.8 Writing of stand-alone data

Using E1803D Easy Programming Interface it is also possible to write stand-alone data which are not marked immediately but are stored either locally or on scanner controller's micro-SD-card. In this mode sending of vector data, scanner- and laser parameters looks exactly the same as for direct operation mode where data are marked immediately. The difference can be found in initialisation (which tells the software to not to mark these data but to store them for later use) and when dynamic data are created.

E1803D supports two types of writing of stand-alone data:

- sending them to the controller via Ethernet or USB connection where they are written to micro-SD-card and
- writing one or more files to the local file system which later have to be copied to the micro-SD-card of the controller manually.

The general procedure for sending stand-alone data to the controller's micro-SD-card has to look as follows:

1. The controller needs to be in idle-state, means it should not mark and should not have loaded an already existing .EPR file. This can be ensured by calling stand-alone command `clepr` with a filename for a file that does not exists on micro-SD-card. For more details please refer to "18.2 Stand-Alone Control Commands"
2. Configure the connection to E1803D controller by calling `E180X_set_connection()`, the returned board instance number has to be used for all following function calls.
3. Enable stand-alone write mode and specify the filename of the .EPR file to be created on micro-SD-card by calling `E180X_set_filepath()` with mode `E180X_FILEMODE_SEND`.
4. Send all laser- and scanner-parameters as well as vector data as usual.
5. Optionally: send information about dynamic contents of the .EPR file to be created by calling `E180X_dynamic_data()` optionally followed by some vector data followed by an other call to function `E180X_dynamic_data()` which ends this section of dynamic data (please refer function description below for details).
6. Wait until `E180X_get_card_state()` returns "busy"
7. Wait until `E180X_get_card_state()` returns "idle" or an error
8. End data transmission and finish created file by calling `E180X_close()`.

The general procedure for writing stand-alone data to the local filesystem has to look as follows:

1. Since writing of local data does not require a working connection to the controller card, it does not need to be configured and the special board instance number 0 has to be used for all following function calls.
2. Enable stand-alone write mode and specify the filename of the .EPR file to be created by calling `E180X_set_filepath()` with mode `E180X_FILEMODE_LOCAL`.
3. Send all laser- and scanner-parameters as well as vector data as usual.
4. Optionally: send information about dynamic contents of the .EPR file to be created by calling `E180X_dynamic_data()` optionally followed by some vector data followed by an other call to function `E180X_dynamic_data()` which ends this section of dynamic data (please refer function description below for details).
5. End data transmission and finish created file by calling `E180X_close()`.

The functions which are specific to writing of stand-alone data have to be used as follows:

`int E180X_set_filepath(unsigned char n,const char *fname,unsigned int mode)`
This function enables operation mode where all following data are not marked immediately but written into an .EPR stand-alone file. This mode stays active until next call of `E180X_close()`. It has to be called prior to `E180X_load_correction()`. Valid parameters and their meaning depends on the usage scenario:

- when sending stand-alone data to a connected controller which writes the .EPR file to the micro-SD-card directly:
  `n` – the 1-based board instance number as returned by `E180X_set_connection()`
  `fname` – name of the file as it has to appear on micro-SD-card of the controller in style "0:/filename.epr" where "0:/" is a fixed prefix specifying the micro-SD-card, "filename" is a free to choose name with recommended 8 characters at max and ".epr" is a fixed, mandatory file extension specifying an E1803 stand-alone file
  `mode` – set to `E180X_FILEMODE_SEND` to specify the data have to be sent to the controller
- when writing stand-alone data to the local filesystem (no controller card directly involved):
  `n` – board instance number, has to be set to 0 (as well as for all other function calls in this mode)
  `fname` – name of the file to be written, this has to be a valid path to a location on a local filesystem which is writeable and needs to have file extension ".epr"
  `mode` – set to `E180X_FILEMODE_LOCAL` to specify the data have to be written locally

Return: `E180X_OK` or an `E180X_ERROR_`-return code in case of an error

`int E180X_dynamic_data(unsigned char n,struct oapc_bin_struct_dyn_data2 *dynData)`

This function can be used to write dynamic data such as texts, serial numbers, barcodes which later can be changed during operation in stand-alone mode.
This function always has to be called in fixed sequences:

1. jump to the start position of the dynamic element by calling `E180X_jump_abs()`
2. first call of `E180X_dynamic_data(n,dynData)` with a valid `dynData` parameter describing the dynamic content and its capabilities
3. optionally and dependent on type of dynamic data that have to be created: some vector data which belong to the dynamic content and are required to build it up
4. second call of `E180X_dynamic_data(n,NULL)` with NULL handed over for parameter `dynData` to finish this element

A stand-alone file can contain up to ten dynamic data elements. So this function can be called up to ten times to create a new element on each call.

When this function is called, beside the .EPR-file an additional .DAT file is created which contains some specific data. During operation in stand-alone mode an other file with the same name and with extension .SER may be created which contains counting information of an included serial number. All these files belong together and deleting one or more of them may lead to unexpected results. When writing the data to local filesystem it also has to be ensured both, the .EPR and the .DAT file are copied to the controller later.

The structure `oapc_bin_struct_dyn_data2` is defined in file "oapc_libio.h" which is part of the OpenSDK. The general usage is described in OpenSDK manual, both are available for download at https://halaser.systems/download.php.

For E1803 scanner controller card following specific parameters and features of this structure can to be used:

Independent from what kind of dynamic element has to be created, following members of structure `oapc_bin_struct_dyn_data2` **always have to be filled** with data:

UID – and unique identifier which can be created out of a plain, human readable text which should be unique too and later can be used to access this specific element via stand-alone control commands; this identifier has to created out of the 8 bit ASCII character using following CRC-function:

```
#define POLY 0x82f63b78

/* CRC-32 (Ethernet, ZIP, etc.) polynomial in reversed bit order. */
unsigned int crc32b(const char *buf)
{
   int          k;
   unsigned int crc=0xFFFFFFFF;
   size_t       len=strlen(buf);

   while (len--)
```

```
        {
           crc^=*buf++;
           for (k=0; k<8; k++)
            crc=crc&1 ? (crc>>1)^POLY : crc>>1;
        }
        return ~crc;
    }
```

`uScaleX` – scaling factor in X-direction in unit 1/1000000
`uScaleY` – scaling factor in Y-direction in unit 1/1000000
`res1a`, `res1b`, `res2`, `res3`, `res4`, `res5`, `res6`, `res7` – these members are reserved for later use and all have to be set to 0

Every dynamic element can be a **serial number**. In such a case the serial number part of structure `oapc_bin_struct_dyn_data2` has to be filled with data:

`fmtString` – an ASCII text with a maximum length of `DYN_DATA_MAX_STRING_LENGTH` describing the format of the serial number/date/time in the dynamic element, here the same notation has to be used as it is known from the serial number input element of BeamConstruct (please refer to the related manual)
`snBeatCount` – specifies how much numbers of mark operations have to elapse before the serial number has to be incremented, here a value of 1 has to be given to increment on every operation
`snBeatOffset` – specifies a counting offset for the beat count parameter
`snIncrement` – specifies the step width by which a serial number has to be incremented
`snNumericBase` – the numeric base of the serial numbers to be displayed, default is 10 for decimal numbers
`snResetAtTime` – the time value at which the serial number has to be reset to it's initial value; set to a timestamp (in unit day of week/date/seconds) when it has to be reset at a given time
`snResetAtValue` – the numeric value at which the serial number has to be reset to it's initial value
`snFlags` – a set of OR-concatenated flags which further specifies handling of the serial number:
0x0002 – reset the serial number at a specific counting value specified by `snResetAtValue`
0x0004 – reset the serial number at a specific day of the week specified by `snResetAtTime`
0x0008 – reset the serial number at a specific date specified by `snResetAtTime`
0x0010 – reset the serial number at a specific time of the day specified by `snResetAtTime`
`snStartValue` – the initial- and reset-to-value of the serial number
`snMinDigits` – the minimum number of digits the serial number has to consist of
`timeOffset` – a static offset (in unit seconds) to be added to the time-part of the current element

Dynamic **text elements** additionally need to fill following parts of the same structure `oapc_bin_struct_dyn_data2`:

`fmtString` – an ASCII text with a maximum length of `DYN_DATA_MAX_STRING_LENGTH` which contains the text to be shown and which can be changed by appropriate stand-alone commands later; when used in combination with serial number data, here a format-string has to be given as described above
`type` – a number which specifies the font to be used for creating the dynamic texts, here one of following values can be used:
`0x01000000` – use “Rect Single” laser font
`0x02000000` – use “Rect Double” laser font
`0x03000000` – use “Roman Simple” laser font
`0x04000000` – use “Roman Double” laser font
`0x05000000` – use “Script Simple” laser font
`0x06000000` – use “Script Double” laser font
`0x07000000` – use “Script Complex” laser font
`0x08000000` – use “Times Simple” laser font
`0x09000000` – use “Times Bold” laser font
`0x0A000000` – use “Times Italic” laser font
`0x0B000000` – use “Times Italic Bold” laser font

`flags` – some OR-concatenated flags which specify orientation, alignment and style of the text to be generated, here no two flags of same type are allowed to be combined which would conflict with each other:
`0x00000000` – orient text left to right
`0x00010000` – orient text right to left
`0x00020000` – orient text top to bottom
`0x00030000` – orient text bottom to top
`0x00000000` – horizontally align to the left
`0x00000100` – centre-align horizontally
`0x00000200` – horizontally align to the right
`0x00000001` – style fixed char-size – all characters are forced to have same distance
`param1` – kerning value in unit 1/1000%
`param2` – reserved for future use, set to 0
`param3` – spacing in unit 1/1000%

Dynamic **DataMatrix barcode elements** require vector data to be sent between two calls of function `E180X_dynamic_data()`, these vector data describe the pattern which has to be marked to create one single element (means square) of the DataMatrix barcode. Such an element needs to incorporate all that is needed including laser- and scannerdata as well as vector data for outline and possible hatches. During stand-alone operation the barcode itself is created by combining these single elements at these positions, where a bit (=square) has to be set).
Additionally following data of the structure `oapc_bin_struct_dyn_data2` need to be filled for this type of element:

`fmtString` – an ASCII text with a maximum length of `DYN_DATA_MAX_STRING_LENGTH` which contains the text to be encoded as DataMatrix barcode and which can be changed by appropriate stand-alone commands later; when used in combination with serial number data, here a format-string has to be given as described above
`type` – set to 71 for DataMatrix barcode
`flags` – some OR-concatenated flags which further specify how the barcode has to be created, currently only one flag is supported:
0x0001 – create a square-shaped DataMatrix barcode instead of a rectangular one
`param1` – set to 0
`param2` – set to -1
`param3` – specifies the size to be generated (in range 2..30) and implicitly the error correction level
`quietZone` – zone the barcode has to be surrounded with, the value given here is the multiple of the width of a single token multiplied with 1000

20.1.8.1 Example

Following a (simplified) example in some pseudo-code is given which demonstrates the correct usage of the programming interface to write stand-alone data. The laser- and scanner-parameters are dropped in this example since they are not specific to this operation mode and always have to be set.

Example: A serial number in format “000/hh/mm” where “000” is a continuously increased number, “hh” is the current hour and “mm” is the current minute has to be encoded into a DataMatrix barcode which has a size of 25x25 mm and is positioned at -30x30 mm within a 100x100 mm working area that itself is aligned to coordinates -50,50

1. not shown here: initialisation of libE1803 (with evaluation of parameter `boardIdx`), sending of default scanner and laser data as usual
2. `E180X_jump_abs(boardIdx, -20132659, 20132659, 0)` // jump to the starting position of the DataMatrix barcode to be created
3. `E180X_dynamic_data(boardIdx, dynData)` // initiate the dynamic data sequence, here the members of dynData are set to following values:

`UID` = `2340633892` – CRC-value of element name “Barcode 1”
`fmtString` = `"$S/%I/%M"` – display serial number, hour and minute
`type` = `71` – DataMatrix barcode
`flags` = `1` – barcode forced to square
`param2` = `4294967295`
`param3` = `2`
`uScaleX` = `1029654`
`uScaleY` = `1029654`
`snIncrement` = `1`
`snNumericBase` = `10`
`snMinDigits` = `3`
all other values are set to 0

4. `E180X_jump_abs(boardIdx, 0, 0, 0)`
   `E180X_mark_abs(boardIdx, 1197222, 0, 0)`
   `E180X_mark_abs(boardIdx, 1197222, -1197222, 0)`
   `E180X_mark_abs(boardIdx, 0, -1197222, 0)`
   `E180X_mark_abs(boardIdx, 0, 0, 0)` // draw a single rectangle which describes one DataMatrix cell (in this example only the outline without any hatching is done, hatches would have to be added here too
5. `E180X_dynamic_data(boardIdx, NULL)` // end the sequence of dynamic data
6. `E180X_execute(boardIdx)`
7. Not shown here: waiting for card being busy, waiting for card being idle (which means writing of the Epr file to the microSD card has been finished), closing the connection to the controller

20.1.9 Error Codes

Most of the functions described above can return an error code in case an operation could not be completed successfully for any reason. So when it does not return with `E180X_OK` the error code informs about the reason for failure:

- `E180X_ERROR_INVALID_CARD` – a wrong or illegal card number was specified with function parameter `n`
- `E180X_ERROR_NO_CONNECTION` – a connection to card could not be established
- `E180X_ERROR_NO_MEMORY` – there is not enough memory available on the host
- `E180X_ERROR_UNKNOWN_FW` – card is running an unknown and/or incompatible firmware version
- `E180X_ERROR_TRANSMISSION` – data transmission to card failed
- `E180X_ERROR_FILEOPEN` – opening of a file failed
- `E180X_ERROR_FILEWRITE` – writing of data into a file failed
- `E180X_ERROR_INVALID_DATA` – data or parameters handed over to a function are invalid, out of range or illegal in current context
- `E180X_ERROR_UNKNOWN_BOARD` – trying to access a controller board that is not a motion controller
- `E180X_ERROR_FILENAME` – a file name handed over to a function was illegal, it is either too long, has an illegal or too long file extension, comes with too much sub-directories or contains illegal characters
- `E180X_ERROR_NOT_SUPPORTED` – the requested feature or function is not supported by the current firmware version
- `E180X_ERROR` – an other, unspecified error occurred
- `E180X_ERROR_STILL_IN_PROGRESS` – the requested function could not be executed because the controller is still busy, this happens e.g. in cases where a stand-alone operation is running on the controller which is not under control of the programming interface

20.2 RTC4 Compatibility Functions

Beside the easy programming interface described above, a bunch of additional functions is provided that are compatible to the ones known from RTC4 scanner controller card. So to use E1803D scanner card with existing code that supports the RTC4 scanner controller, following few steps have to be done:

- insert a call to `E180X_set_connection()` into existing code as very first in order to specify the communication connection for E1803D card (this is the only exception where an E1803D Easy Interface Function should be used together with an RTC4 Compatibility Function)
- recompile the existing RTC4-application so that it uses e1803inter.DLL/libe1803inter.so instead of RTC4DLL.dll/libslrtc4.so

In case E1803D card has to be operated with default connection settings, no recompilation is necessary, here e1803inter.dll/libe1803inter.so just has to be renamed to RTC4DLL.dll/libslrtc4.so.

Since most relevant RTC4 functions are already provided, majority of existing applications should work now without any further modifications. Due to the completely different concept of E1803D scanner controller, there are some differences to the original RTC4 programming interface which should be checked in case of some errors:

- all RTC4 functions that exist as list- and non-list-commands are treated like a list command
- list switch commands are ignored since E1803D does not make use of separated lists internally
- output of already sent marking data is started on calls to `n_execute_list()`,`execute_list()`, `n_set_end_of_list()` or `set_end_of_list()`
- some functions are not implemented or always return a default value (please refer below for a list of not implemented RTC4 functions)

Following functions are specific to RTC4 hardware or do not make sense when E1803D scanner card is used and therefore aren't supported:

```
n_load_z_table()
load_z_table()
n_set_defocus_list()
set_defocus_list()
n_set_offset_list()
set_offset_list()
n_laser_on_list()
laser_on_list()
n_set_list_jump()
set_list_jump()
n_set_input_pointer()
set_input_pointer()
n_list_call()
list_call()
n_list_return()
list_return()
n_z_out_list()
z_out_list()
n_timed_jump_abs()
timed_jump_abs()
n_timed_mark_abs()
timed_mark_abs()
n_timed_jump_rel()
timed_jump_rel()
n_timed_mark_rel()
timed_mark_rel()
n_set_fly_rot()
set_fly_rot()
n_fly_return()
fly_return()
n_calculate_fly()
calculate_fly()
n_select_cor_table_list()
select_cor_table_list()
n_set_wait()
set_wait()
n_simulate_ext_start()
simulate_ext_start()
n_set_pixel_line()
set_pixel_line()
```

```
n_set_pixel()
set_pixel()
n_set_extstartpos_list()
set_extstartpos_list()
n_laser_signal_on_list()
laser_signal_on_list()
n_laser_signal_off_list()
laser_signal_off_list()
n_set_io_cond_list()
set_io_cond_list()
n_clear_io_cond_list()
clear_io_cond_list()
n_list_jump_cond()
list_jump_cond()
n_list_call_cond()
list_call_cond()
n_save_and_restart_timer()
save_and_restart_timer()
n_set_ext_start_delay_list()
set_ext_start_delay_list()
n_set_trigger()
set_trigger()
n_arc_rel()
arc_rel()
n_arc_abs()
arc_abs()
drilling()
regulation()
flyline()
n_get_input_pointer()
get_input_pointer()
n_get_marking_info()
get_marking_info()
n_auto_change_pos()
auto_change_pos()
aut_change()
n_start_loop()
start_loop()
n_quit_loop()
quit_loop()
n_write_da_2()
write_da_2()
n_set_max_counts()
set_max_counts()
n_set_offset()
set_offset()
n_disable_laser()
disable_laser()
n_enable_laser()
enable_laser()
n_stop_list()
stop_list()
n_restart_list()
restart_list()
n_get_xyz_pos()
get_xyz_pos()
n_get_xy_pos()
get_xy_pos()
n_select_list()
select_list()
n_z_out()
z_out()
n_laser_signal_on()
```

```
laser_signal_on()
n_laser_signal_off()
laser_signal_off()
n_set_delay_mode()
set_delay_mode()
n_set_piso_control()
set_piso_control()
n_select_status()
select_status()
n_get_encoder()
get_encoder()
n_select_cor_table()
select_cor_table()
n_execute_at_pointer()
execute_at_pointer()
n_get_head_status()
get_head_status()
n_simulate_encoder()
simulate_encoder()
n_set_hi()
set_hi()
n_release_wait()
release_wait()
n_get_wait_status()
get_wait_status()
n_set_ext_start_delay()
set_ext_start_delay()
n_home_position()
home_position()
n_set_rot_center()
set_rot_center()
n_read_ad_x()
read_ad_x()
n_read_pixel_ad()
read_pixel_ad()
n_get_z_distance()
get_z_distance()
n_get_time()
get_time()
n_set_defocus()
set_defocus()
n_set_softstart_mode()
set_softstart_mode()
n_set_softstart_level()
set_softstart_level()
n_control_command()
control_command()
load_cor()
load_pro()
n_get_serial_number()
get_serial_number()
n_get_serial_number_32()
get_serial_number_32()
get_hi_data()
n_auto_cal()
auto_cal()
n_get_list_space()
get_list_space()
teachin()
n_get_value()
get_value()
set_duty_cycle_table()
n_move_to()
```

```
move_to()
getmemory()
n_get_waveform()
get_waveform()
n_measurement_status()
measurement_status()
n_load_varpolydelay()
load_varpolydelay()
n_write_da_2_list()
write_da_2_list()
```

20.3 USC1/2 Compatibility Functions (SCI interface)

Beside the easy programming interface described above a bunch of additional functions is provided that are compatible to the ones known from SCI programming interface used for USC1/2 scanner controller card. So to use E1803D scanner card with existing code that supports the USC1 or USC2 scanner controller, following steps have to be done:

- insert a call to `E180X_set_connection()` into existing code as very first in order to specify the communication connection for E1803D card (this is the only exception where an E1803D Easy Interface Function should be used together with an USC1/2 Compatibility Function)
- recompile the existing SCI-application so that it uses e1803inter.DLL instead of sc_optic.dll

In case E1803D card has to be operated with default connection settings, no recompilation is necessary, here e1803inter.dll just has to be renamed to sc_optic.dll.

Since most relevant SCI functions are already provided, majority of existing applications should work now without any further modifications. Due to the different concept of E1803D scanner controller, there are some differences to the original SCI programming interface which should be checked in case of troubles:

- all USC1/2 functions/types that exist as stream- and non-stream-variants are treated like a stream command
- output of already sent marking data is started on call to `ScSCIFlush()` latest
- some functions are not implemented or always return a default value (please refer below of a list of not implemented SCI functions)

Following functions are specific to USC1/2 hardware or do not make sense in relation to E1803D scanner card and therefore aren't supported:

```
long ScSCISetContinuousMode()
long ScSCIGetContinuousMode()
long ScSCIDevicePixelLine()
long ScSCIRasterPixelLine()
long ScSCIRasterStart()
long ScSCIRasterEnd()
long ScSCIGetDeviceName()
long ScSCIGetDeviceCaps()
long ScSCIGetDeviceData()
long ScSCISetDeviceData()
long ScSCIGetExternalTrigger()
long ScSCISetExternalTriggerCount()
long ScSCIGetExternalTriggerCount()
long ScSCISetEnableHead()
long ScSCIGetEnableHead()
long ScSCISetZField()
long ScSCIGetZField()
long ScSCIGetZGain()
long ScSCISetZGain()
long ScSCIGetHomePosition()
long ScSCIGetZHomePosition()
long ScSCISetHomePosition()
long ScSCISetZHomePosition()
long ScSCIGetZOffset()
long ScSCISetZOffset()
```

```
long ScSCISetZWorkingArea()
long ScSCIGetZWorkingArea()
long ScSCIGetHomeJump()
long ScSCISetHomeJump()
long ScSCIMaxExternalTriggerCount()
long ScSCIResetExternalTriggerCount()
long ScSCISetDeviceEnableFlags()
long ScSCIGetDeviceEnableFlags()
long ScSCIGetDevicePath()
long ScSCISetDeviceMiscValueD()
long ScSCIGetDeviceMiscValueD()
long ScSCISetHeadCount()
long ScSCIStreamInfo()
long ScSCIGetSpeed()
long ScSCISetSpeed()
long ScSCIGetStyleSet()
long ScSCISetStyleSet()
long ScSCISetLoopMode()
long ScSCIGetLoopMode()
long ScSCISetLoop()
long ScSCIGetLoop()
long ScSCISetMessageWindow()
long ScSCISetAxisState()
long ScSCIGetAxisState()
long ScSCISaveSettings()
long ScSCILoadSettings()
long ScSCIEditSettings()
long ScSCIUpdateDeviceStyle()
long ScSCIGetInterfaceVersion()
long ScSCIGetDebugMode()
long ScSCISetDebugMode()
long ScSCIGetIdentString()
long ScSCIGetDeviceMapLaserPort()
long ScSCISetDeviceMapLaserPort()
long ScSCIGetUSCInfoLong()
```

APPENDIX A – Wiring between E1803D and specific laser types

This appendix describes the wiring between the controller card and specific laser models when an E1803dock (as described in "15 E1803dock Extension Board") is not used or when it can't be used because the laser comes with a non-standard, proprietary control interface.

Wiring between E1803D and IPG YLP Series Type B, B1 and B2, GZTech YFPN series fiber laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	E1803D Connector	E1803D Connector / Pin	IPG Pin
LP0	Laser signal connector	Pin 1	Pin 1
LP1		Pin 3	Pin 2
LP2		Pin 5	Pin 3
LP3		Pin 7	Pin 4
LP4		Pin 9	Pin 5
LP5		Pin 11	Pin 6
LP6		Pin 13	Pin 7
LP7		Pin 15	Pin 8
MO / Main Oscillator		Pin 8	Pin 18
LP8 Latch		Pin 17	Pin 9
LaserA / Frequency		Pin 22	Pin 20
Laser Gate / Modulation		26 pin connector, pin 26	Pin 19
Alarm, one of DIn0…DIn7	Digital interface connector	Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 16
Alarm, one of DIn0…DIn7		Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 21
Pilot Laser, one of DOut0…DOut7		Pin 3, 5, 7, 9, 11, 13, 15 or 17	Pin 22 *)

*) may require additional power driver since some laser variants consume a current at this input which is higher than the maximum output allowed

In this wiring-scheme no GND-connections are listed, they have to be added in order to get valid and working connections.

Wiring between E1803D and JPT YDFLP series fiber laser (“MOPA”) or IPG YLP Series Type D fiber laser or Raycus RFL PMX/PQB Series fiber laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	E1803D Connector	E1803D Connector / Pin	JPT D-SUB25
LP0	Laser signal connector	Pin 1	Pin 1
LP1 / serial data		Pin 3	Pin 2
LP2 / serial clock		Pin 5	Pin 3
LP3		Pin 7	Pin 4
LP4		Pin 9	Pin 5
LP5		Pin 11	Pin 6
LP6		Pin 13	Pin 7
LP7		Pin 15	Pin 8
MO / Main Oscillator		Pin 8	Pin 18
LP8 Latch		Pin 17	Pin 9
LaserA / Frequency		Pin 22	Pin 20
Laser Gate / Modulation		26 pin connector, pin 26	Pin 19
LaserB / serial enable		Pin 19	Pin 22 *)
GND		Pin 2 or 23	Pin 10-15
Alarm, one of DIn0…DIn7	Digital interface connector	Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 16
Alarm, one of DIn0…DIn7		Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 21
Pilot Laser: in software choose LaserB as output			

*) for details regarding double-usage of this pin, please refer to the manual of the laser

Wiring between E1803D and IPG YLP Series Type E fiber laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Variant using E1803D with support for APD index setting via DB-25 serial data interface

Signal Name	E1803D Connector	Connector / Pin	IPG Pin
LP0	Laser signal connector	Pin 1	Pin 1
LP1		Pin 3	Pin 2
LP2		Pin 5	Pin 3
LP3		Pin 7	Pin 4
LP4		Pin 9	Pin 5
LP5		Pin 11	Pin 6
LP6		Pin 13	Pin 7
LP7		Pin 15	Pin 8
MO / Main Oscillator		Pin 8	Pin 18
LP8 Latch		Pin 17	Pin 9
LaserA / Frequency		Pin 22	Pin 20
Laser Gate / Modulation		26 pin connector, pin 26	Pin 19
Alarm, one of DIn0..DIn7	Digital interface connector	Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 16
Alarm, one of DIn0..DIn7		Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 21
Pilot Laser, one of DOut0, DOut1, DOut5..DOut7		Pin 3, 5, 13, 15 or 17	Pin 22 [1])
Serial Enable		Pin 7	Pin 24 [2])
Serial Clock		Pin 9	Pin 13 [2])
Serial Data		Pin 11	Pin 10 [2])

[1]) may require additional power driver since some laser variants consume a current at this input which is higher than the maximum output allowed
[2]) serial data transmission requires firmware version 2 or newer

In this wiring-scheme no GND-connections are listed, they have to be added in order to get valid and working connections.

Wiring between E1803D and IPG YLP Series Type G fiber laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	E1803D Connector	E1803D Connector / Pin	D-SUB25
LP0	Laser signal connector	Pin 1	Pin 1
LP1		Pin 3	Pin 2
LP2		Pin 5	Pin 3
LP3		Pin 7	Pin 4
LP4		Pin 9	Pin 5
LP5		Pin 11	Pin 6
LP6		Pin 13	Pin 7
LP7		Pin 15	Pin 8
MO / Main Oscillator		Pin 8	Pin 18
LP8 Latch		Pin 17	Pin 9
LaserA / Frequency		Pin 22	Pin 20
Laser Gate / Modulation		26 pin connector, pin 26	Pin 19
LaserB		Pin 19	Pin 22
GND		Pin 2 or 23	Pin 14
Alarm, one of DIn0...DIn7	Digital interface connector	Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 11
Alarm, one of DIn0...DIn7		Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 16
Alarm, one of DIn0...DIn7		Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 21
Pilot Laser: in software choose LaserB as output			

Wiring between E1803D and IPG YLR Series laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	Board	Connector / Pin	IPG Pin
AOut0 or AOut1	Laser signal connector	Pin 12 or pin 14	Pin 12
MO / Main Oscillator		Pin 8	Pin 18
Laser Gate / Modulation		Pin 26	Pin 15
Pilot Laser, one of DOut0..DOut7	Digital interface connector	Pin 3, 5, 7, 9, 11, 13, 15 or 17	Pin 17

In this wiring-scheme no GND-connections are listed, they have to be added in order to get valid and working connections.

Wiring between E1803D and IPG YLM Series laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	Board	Connector / Pin	IPG Pin
AOut0 or AOut1	Laser signal connector	Pin 12 or pin 14	Pin 8 [1])
MO / Main Oscillator		Pin 8	Pin 7
Laser Gate / Modulation		Pin 26	Pin 17
Pilot Laser, one of DOut0..DOut7	Digital interface connector	Pin 3, 5, 7, 9, 11, 13, 15 or 17	Pin 21
Laser ready, one of DIn0..DIn7		Pin 4, 6, 8,10, 12, 14, 16 or 18	Pin 19
RS232 serial interface	Serial interface connector	Pins 1, 2 and 3	RS232 interface

[1]) The E1803D analogue output provides signals in range 0..10 V while the IPG YLM laser input expects signals in range 0..4 V. To avoid hardware damage the signal level has to be limited by additional hardware measures, e.g. by a voltage divider

In this wiring-scheme no GND-connections are listed, they have to be added in order to get valid and working connections.

Wiring between E1803D and MaxPhotonics MFP fiber laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	E1803D Connector	E1803D Connector / Pin	MaxPhotonics D-SUB25
LP0	Laser signal connector	Pin 1	Pin 1
LP1		Pin 3	Pin 2
LP2		Pin 5	Pin 3
LP3		Pin 7	Pin 4
LP4		Pin 9	Pin 5
LP5		Pin 11	Pin 6
LP6		Pin 13	Pin 7
LP7		Pin 15	Pin 8
LP8 Latch		Pin 17	Pin 9
MO / Main Oscillator		Pin 8	Pin 18
LaserA / Frequency		Pin 22	Pin 20
Laser Gate / Modulation		26 pin connector, pin 26	Pin 19
GND		Pin 2 or 23	Pin 10-15
Alarm, one of DIn0…DIn7	Digital interface connector	Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 16
Alarm, one of DIn0…DIn7		Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 21
Pilot Laser, one of DOut0…DOut7		Pin 3, 5, 7, 9, 11, 13, 15 or 17	Pin 22

Wiring between E1803D and SPI G4 Pulsed Fibre Laser / TRUMPF TruPulse nano series

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	E1803D Connector	E1803D Connector / Pin	SPI Pin
LP0	Laser signal connector	Pin 1	Pin 17
LP1		Pin 3	Pin 18
LP2		Pin 5	Pin 19
LP3		Pin 7	Pin 20
LP4		Pin 9	Pin 51
LP5		Pin 11	Pin 52
LP6		Pin 13	Pin 53
LP7		Pin 15	Pin 54
MO / Laser Enable		Pin 8	Pin 7
LP8 Latch		Pin 17	Pin 23
LaserA / Pulse Trigger		Pin 22	Pin 13
AOut0 / Power		Pin 12	Pin 65
AOut1 / Simmer		Pin 14	Pin 64
LaserGate / Modulation		Pin 26	Pin 5
Alarm, one of DIn0…DIn7	Digital interface connector	Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 9
Pilot Laser, one of DOut0…DOut7		Pin 3, 5, 7, 9, 11, 13, 15 or 17	Pin 6

In these wiring-schemes no GND-connections are listed, they have to be added in order to get valid and working connections.

Wiring between E1803D and Raycus fiber laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	E1803D Connector	E1803D Pin	Raycus Pin
LP0	Laser signal connector	Pin 1	Pin 1
LP1		Pin 3	Pin 2
LP2		Pin 5	Pin 3
LP3		Pin 7	Pin 4
LP4		Pin 9	Pin 5
LP5		Pin 11	Pin 6
LP6		Pin 13	Pin 7
LP7		Pin 15	Pin 8
MO / Main Oscillator		Pin 8	Pin 18
LaserA / Frequency		Pin 22	Pin 20
Laser Gate / Modulation		Pin 26	Pin 19
Alarm, one of DIn0...DIn7	Digital interface connector	Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 16
Alarm, one of DIn0...DIn7		Pin 4, 6, 8, 10, 12, 14, 16 or 18	Pin 21

In this wiring-scheme no GND-connections are listed, they have to be added in order to get valid and working connections.

Wiring between E1803D and Raycus C500 Series laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	E1803D Connector	E1803D Pin	Raycus Pin
AOut0 or AOut1	Laser signal connector	12 or 14	22
GND		2 or 23	25
LaserA		22	BNC

Wiring between E1803 and DAVI D-Series RF CO_2 Laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	E1803D Connector	E1803D Pin	DAVI RJ45 Pin	DAVI RJ45 Wire Colour
LaserA / Frequency	Laser signal connector	22	1	orange/white
GND		2 or 23	8	brown
Laser Ready input, one of DIn0…DIn7	Digital interface connector	Pin 4, 6, 8, 10, 12, 14, 16 or 18	3	green/white
GND		2	6	green

Wiring between E1803 and Raycus RFL-C Series 24V Control CW Laser



PLEASE NOTE: this wiring description is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Laser models that require control signals with higher voltage levels than 5V (typically 16..24V) can't be connected directly and require additional hard- and software configuration steps. Following hardware configuration is required to operate such a laser model with the E1803D:

- configure the digital interface to operate in opto-insulated mode (opto-mode 0) and with external power supply (as described in section "7.8.2 Opto-Configuration Jumpers")
- connect one of the DOut-pins of the digital interface with the "MOD+" input (pin 15) of the laser
- add the "`lasergate`" parameter to the e1803.cfg to specify this specific DOut to be used as LaserGate (as described in section "7.5 microSD-Card")
- connect an other of the DOut-pins of the digital interface with the "Red Laser" input (pin 18) of the laser
- connect AOut0 or AOut1 of the Laser Interface with the "Analog" input (pin 12) of the laser

Based on this, following software configuration is required:

- configure the DOut line that is used for the pilot laser (aka "Red Laser") to be used for preview (e.g. in BeamConstruct at menu "Project" → "Project settings" → tab-pane "Scanner" → section "Pilot laser")
- configure the used analogue output to be used for power control (e.g. in BeamConstruct when E1803D is configured to be used as scanner card at menu "Project" → "Project settings" → tab-pane "Scanner" → button "Configure" → parameter "Additional power output")
- the laser-mode does not matter for this configuration as for all types of lasers that may be set for the E1803D the LaserGate signal is switched

In this wiring-description no GND-connections are listed separately, they have to be added in order to get valid and working connections.

Wiring between E1803D and GZTECH S355 series DPSS Laser

PLEASE NOTE: this wiring scheme is a non-binding policy, it may be incorrect due to changes in connected hardware. So in every case please check this table against specification and wiring documentation of the used laser!

Signal Name	E1803D Connector	E1803D Pin	GZTECH DB9 connector Pin	GZTECH Signal Name
LaserA / Frequency	Laser signal connector	22	3	PWM
GND		2 or 3	5	GND
LaserGate		26	2	Gate

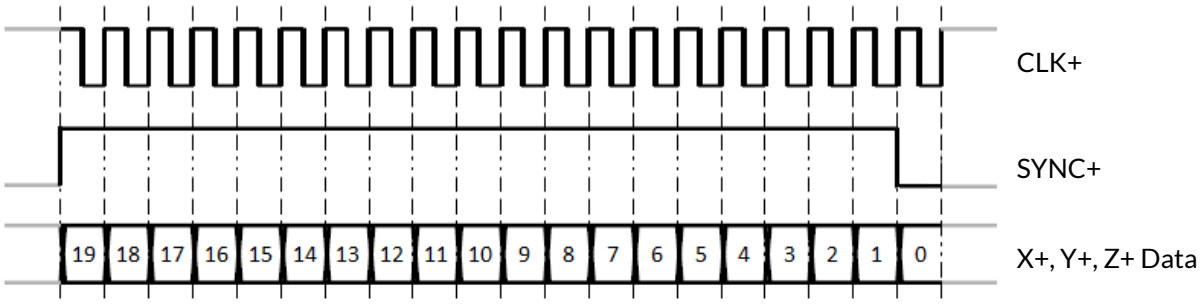
Within BeamConstruct following settings have to be done in order to control the laser:

- menu “Project” → “Project settings” → tab-pane “Scanner” → button “Configure” → parameter “Laser type” has to be set to value “YAG1”
- menu “Project” → “Pen settings” → tab-pane “General”: the “Frequency” slider has to be set to an appropriate value (typically 30..50 kHz, please refer to the manual of the laser for further details), the “Power” slider has no effect as no additional power output was configured in scanner controller settings
- menu “Project” → “Pen settings” → tab-pane “Laser”: the parameter “Pulse length” has to be set to an appropriate value (such as 1 usec, please refer to the manual of the laser for further details)

To use this laser model with BeamConstruct, HALsetup software package version 8.2-07 or newer is required.

APPENDIX B – XY2-100 / XY2-200 protocol description

Depending on the actual configuration, the data submitted at 26 pin or D-SUB25 connector of E1803D are conform to XY2-100 specification:



In standard 16 bit operating mode first three bits are set to 001, then 16 bit position data followed by a parity bit (even parity) are transmitted:

19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	D15..D0 position data																Pe

In enhanced XY2-100E 18 bit operating mode first bit is set to 1, then 18 bit position data followed by a parity bit (odd parity) are transmitted:

19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	D17..D0 position data																		Po

To use this mode, the related tune-value has to be set in configuration file (please refer to section “7.5 microSD-Card”)

In XY2-100 and XY2-100E mode one frame with 20 bits has a length of 10 usec which is similar to 100 kHz output clock.
For the XY2-200 and XY2-200E modes a frame with 20 bits has a length of 5 usec which is similar to 200 kHz output frequency.

APPENDIX C – XY3-100 protocol description

Depending on the actual configuration, the data submitted at 26 pin or D-SUB25 connector of E1803D are conform to XY3-100 specification. For details about the XY3-100 protocol, please refer to information given online at https://halaser.systems/compare.php#XY3

APPENDIX D – SL2-100 protocol description

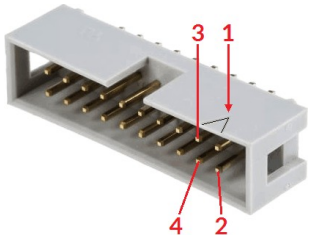
For information about the SL2-100 scanner protocol, please refer to information given online at https://halaser.systems/compare.php#XY3

APPENDIX E – RL3-100 protocol description

For information about the RL3-100 scanner protocol, please refer to information given online at https://halaser.systems/compare.php#XY3

APPENDIX F – IDC connector pin numbering

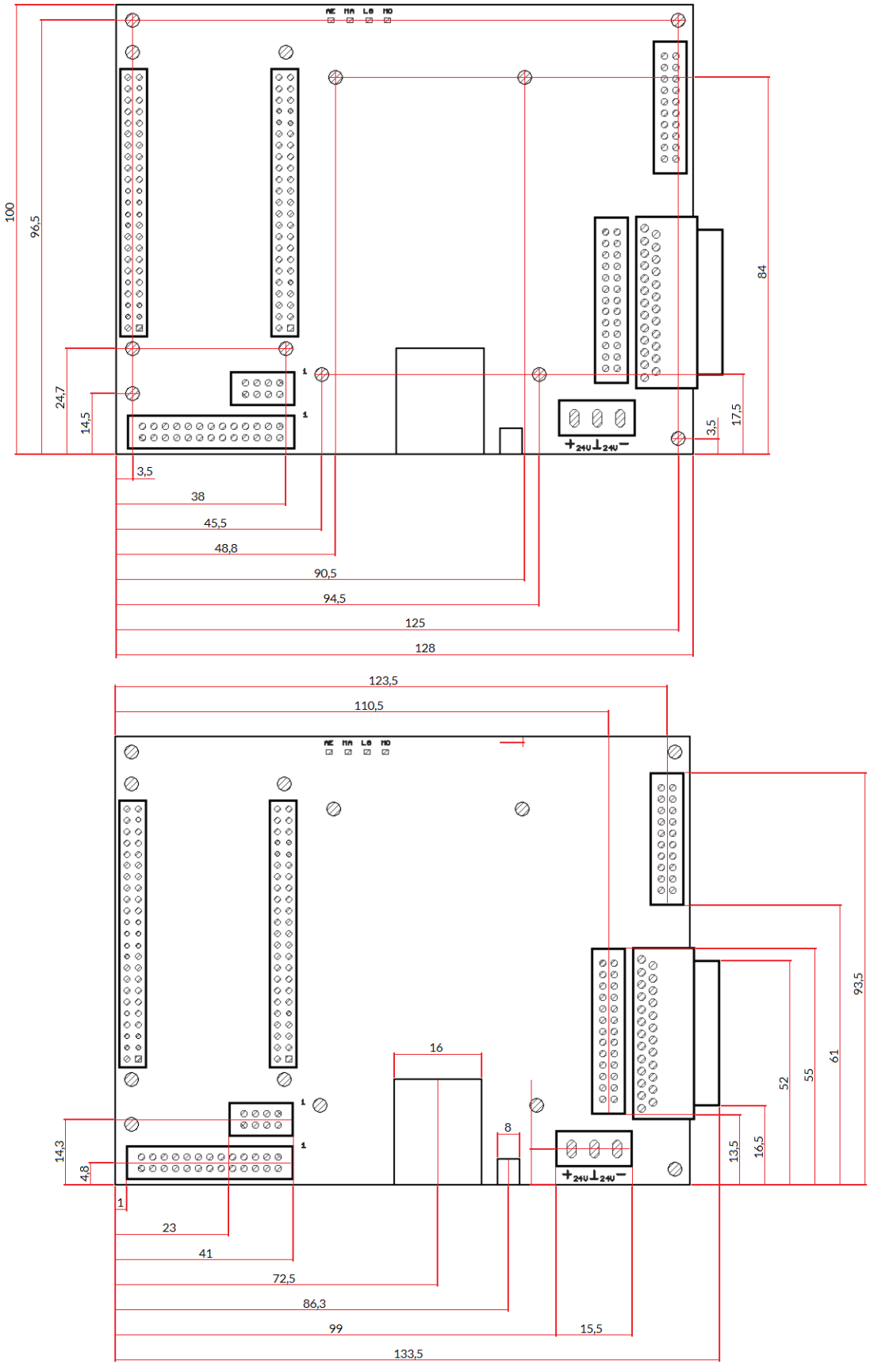
Pin numbering of the IDC connectors (according to pinout-tables shown in hardware description sections above) can be seen in below image:



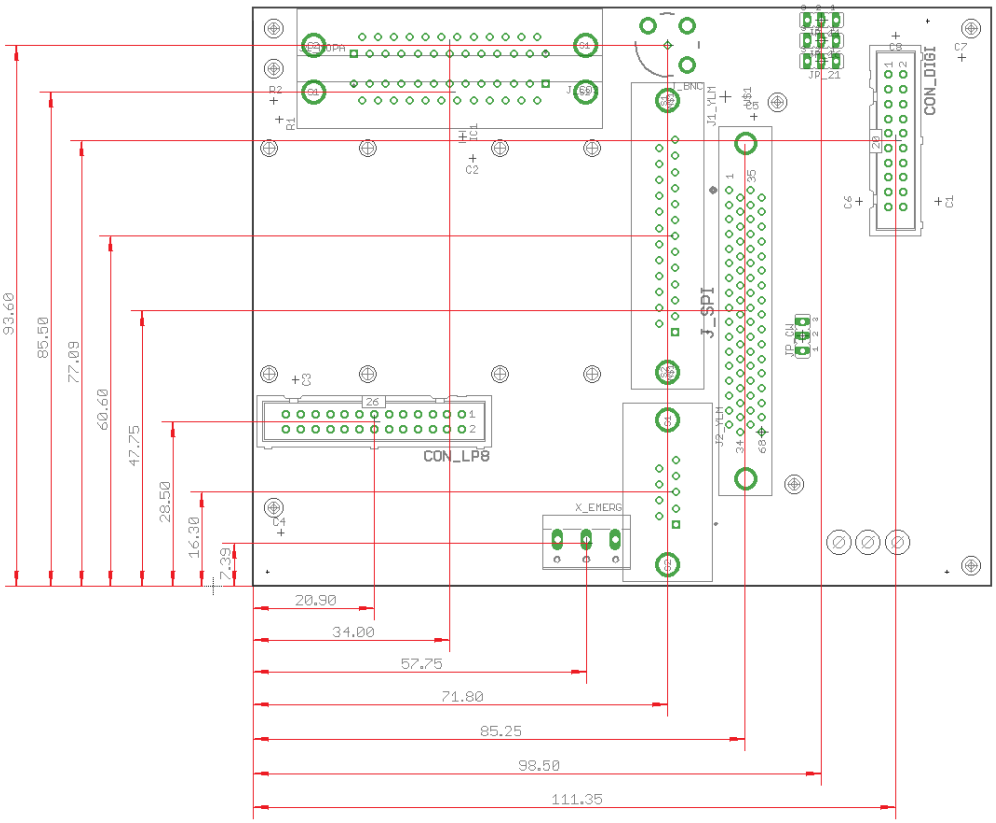
The first pin is marked by a small arrow in connector. Second pin is below of it, counting continues column-wise.

APPENDIX G – Mechanical Dimensions

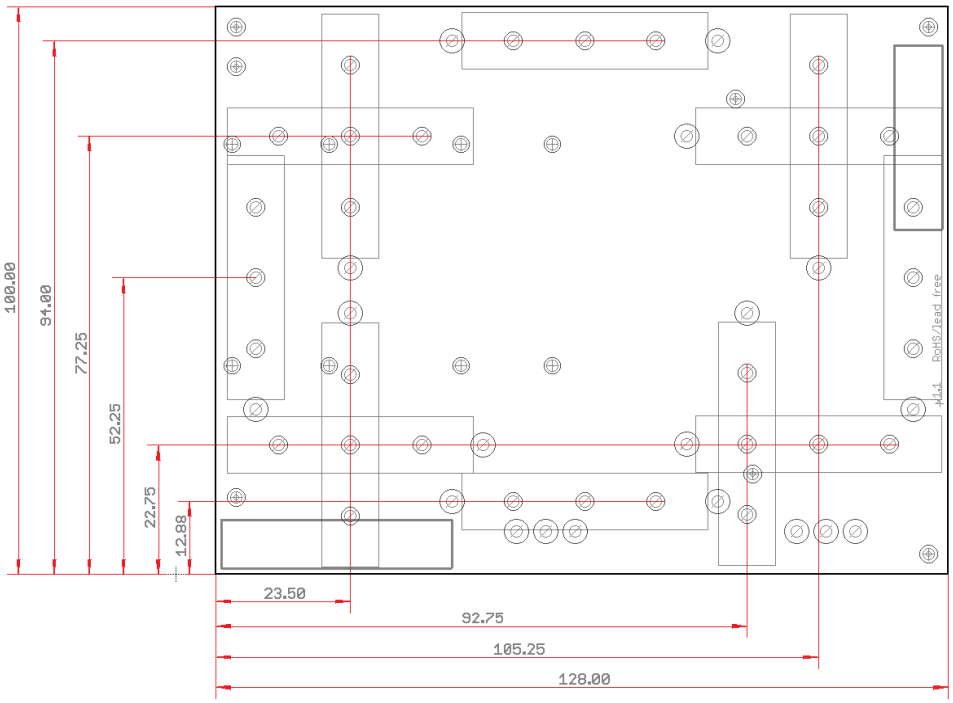
Mechanical dimensions, positions of connectors and holes of E1803D, all values are given in unit mm:



Mechanical dimensions and positions of connectors of E1803dock, all values are given in unit mm:



Mechanical dimensions, positions of mounting points and holes of E1803base, all values are given in unit mm:



Index

1

1:1....1
16-bit IO port....29
180X_mark_abs_list....115

2

24V....154
2D marking on-the-fly....38

8

8.3....41
80X_execute....103

A

A/E....19, 23
ADC....21
adcfreq....21
Alive....19, 75
AOut0....18, 35, 77, 154
AOut1....18, 35, 77, 154
auto....42, 78
autofile....23

B

barcode....42
barcodes....82f.
BeamConstruct....13, 41, 75
bitmap....117
bitmap lines....117
BNC....69f.
boot....29
busid....25, 76, 86

C

C45 rail....74
C45 rail adapter....74
C45 rail lock....74
cdepr....81
cdser....83
cdtl0....48, 85
cdtl1....48, 85
cecho....25, 76
cfror....77
cftim....84
cgana....78
cgbsr....77
cgbuf....82
cgcor....80
cgepr....81
cginp....76
cglog....85
cgmtx....77
cgser....83
cgsta....84
cgtim....84
cgtin....80
cgtxt....83
chalt....79
ciser....83

cjsor....23, 77
clepr....44, 81, 133
cliprot....31
clipwa....31
CLK....32f., 49f.
cmapz....31, 86
cmsor....23, 77
CNC....81
CO2....34, 69, 112
commands....76
configuration....21, 42, 46
continuous wave....68
continuously running frequency....113
control commands....76
correction table....21
corrtable....21, 80
corrtable0....21
cpuor....77
cpwor....23, 77
crrrr....85
crser....83
crtim....84
csbuf....82
cscnc....79, 95
cscor....21, 80
cslgt....85
cslmo....86
cslp8....86
csmtx....8, 78
csout....85
cspof....78
csser....83
cssta....85
cstal....78
cstat....79, 100
cstdy....84
csthr....84
cstim....83
cstmi....84
cstmo....84
cstop....79
cstrt....79
cstsc....84
cstxt....42, 82
cstyr....84
cswaf....30, 78
ctlxy....81
ctrig....79
current....52f., 55f.
cvers....76
CW....67f.
cxcnc....30, 80, 95, 99

D

DataMatrix....42, 82f., 136
DAVI....72
DAVI D-Series RF CO2....153
digidebc....24
digiinit....24
digimask....24

Digital Interface....14, 36
DIn....36
DIN rail....74
DIN/C45 rail adapter....74
DIN/C45 rail lock....74
Dot....47
dot marker....46
Dot Mode....47f., 85
dot peen....46
dotdist....46
dotfont0....46
dotfont1....46, 48
dotfont1y....46
dotmark....42, 46
dottime....47
DOut....36f.
DOut0....36
DOut7....36
Download new firmware....92
DYN_DATA_MAX_STRING_LENGTH....135f.

E

E1803base....162
E1803dock....162
E180X_ana_read....108
E180X_ana_read()....127
E180X_ana_write....121
E180X_close....102
E180X_close()....102, 133
E180X_COMMAND_FLAG_ANA_AOUT0....124
E180X_COMMAND_FLAG_ANA_AOUT1....124
E180X_COMMAND_FLAG_ASYNC....126
E180X_COMMAND_FLAG_DIRECT....113, 121, 123, 126, 128
E180X_COMMAND_FLAG_DONOTWAIT....131
E180X_COMMAND_FLAG_FREQ_LASERA....125
E180X_COMMAND_FLAG_HEAD_STATE_RAW....119
E180X_COMMAND_FLAG_PASSIVE....122, 127
E180X_COMMAND_FLAG_SCANNER_VAR_POLYDELAY....111
E180X_COMMAND_FLAG_STREAM....113, 121ff., 126, 128, 131
E180X_COMMAND_FLAG_SYNC....104
E180X_COMMAND_FLAG_UART1....126
E180X_COMMAND_FLAG_XYCORR_FLIPXY....109
E180X_COMMAND_FLAG_XYCORR_MIRRORX....109
E180X_COMMAND_FLAG_XYCORR_MIRRORY....109
E180X_COMMAND_FLAG_ZCORR_MIRRORZ....110
E180X_CSTATE_FILE_WRITE_ERROR....106
E180X_CSTATE_HALTED....106
E180X_CSTATE_MARKING....101f., 105f.
E180X_CSTATE_MARKING|E180X_CSTATE_PROCESSING....105
E180X_CSTATE_PROCESSING....101f., 105f.
E180X_CSTATE_SAC_CTLXY....82, 106
E180X_CSTATE_SAC_MARKING....82, 106
E180X_CSTATE_SAC_READY....82, 106
E180X_CSTATE_WAIT_EXTTRIGGER....106
E180X_CSTATE_WAIT_INPUT....106
E180X_CSTATE_WAS_START_PRESSED....106
E180X_CSTATE_WAS_STOP_PRESSED....106
E180X_CSTATE_WRITING_DATA....93
E180X_CSTATE_WRITING_DATA_ERROR....93
E180X_delay....106

E180X_digi_pulse() ... 122
E180X_digi_read ... 122
E180X_digi_read() ... 122
E180X_digi_set_mip_output ... 125
E180X_digi_set_mip_output() ... 23
E180X_digi_set_motf_powerctl() ... 124
E180X_digi_set_motf_sim() ... 123
E180X_digi_set_motf(... 123
E180X_digi_set_motf() ... 123f.
E180X_digi_set_wet_output ... 125
E180X_digi_set_wet_output() ... 23
E180X_digi_wait ... 123
E180X_digi_wait_motf ... 124
E180X_digi_wait_motf() ... 124
E180X_digi_wait() ... 123
E180X_digi_write ... 122
E180X_dynamic_data() ... 133f.
E180X_ERROR_FILENAME ... 137
E180X_ERROR_FILEOPEN ... 137
E180X_ERROR_FILEWRITE ... 137
E180X_ERROR_INVALID_CARD ... 137
E180X_ERROR_INVALID_DATA ... 137
E180X_ERROR_NO_CONNECTION ... 137
E180X_ERROR_NO_MEMORY ... 137
E180X_ERROR_TRANSMISSION ... 137
E180X_ERROR_UNKNOWN_BOARD ... 137
E180X_ERROR_UNKNOWN_FW ... 137
E180X_execute ... 101
E180X_execute() ... 22, 101, 103f.
E180X_FILEMODE_LOCAL ... 133f.
E180X_FILEMODE_SEND ... 133f.
E180X_get_card_state ... 105
E180X_get_card_state() ... 101ff.
E180X_get_free_space ... 106
E180X_get_head_state ... 118
E180X_get_library_version ... 107
E180X_get_library_version() ... 107
E180X_get_pos() ... 115
E180X_get_serial_number() ... 107
E180X_get_startstop_state ... 105
E180X_get_sync() ... 103
E180X_get_version ... 107
E180X_get_version() ... 107
E180X_halt_execution ... 104
E180X_halt_execution() ... 106
E180X_jump_abs ... 114
E180X_jump_abs_list ... 114
E180X_jump_abs_list() ... 114
E180X_jump_abs() ... 101, 124
E180X_LASERMODE_CO2 ... 112
E180X_LASERMODE_CRF ... 113
E180X_LASERMODE_DFREQ ... 113
E180X_LASERMODE_MOPA ... 98, 113
E180X_LASERMODE_YAG ... 98
E180X_LASERMODE_YAG1 ... 112
E180X_LASERMODE_YAG2 ... 112
E180X_LASERMODE_YAG3 ... 113
E180X_load_correction ... 108
E180X_load_correction() ... 101, 133
E180X_lp8_write ... 120

E180X_lp8_write_latch....120
E180X_lp8_write_latch()....120
E180X_lp8_write_mo....121
E180X_lp8_write_mo()....121
E180X_lp8_write_pilot....121
E180X_lp8_write()....120
E180X_mark_abs....114f.
E180X_mark_abs_list()....115
E180X_mark_abs()....101, 124
E180X_mark_pixelline....117
E180X_motion_digi_read()....127
E180X_motion_digi_write()....127
E180X_motion_get_pos()....131
E180X_motion_move_abs_async()....130
E180X_motion_move_abs()....129, 131
E180X_motion_move_rel()....130f.
E180X_motion_reference()....131
E180X_MOTION_REFSTEP_INV_SWITCH....132
E180X_MOTION_REFSTEP_N....132
E180X_MOTION_REFSTEP_P....132
E180X_motion_set_accel()....129
E180X_motion_set_limits()....128
E180X_motion_set_pos()....132
E180X_motion_set_pulsewidth()....128
E180X_motion_set_speed()....129
E180X_motion_stop()....131
E180X_motion_stream_wait()....129ff.
E180X_PIXELMODE_GATE_POWER_CONTROL....116
E180X_PIXELMODE_HW_POWER_CONTROL....116
E180X_PIXELMODE_JUMP_LEAVE_POWER....116
E180X_PIXELMODE_JUMP_N_SHOOT....116
E180X_PIXELMODE_NO_JUMPS....116
E180X_release_trigger_point....101, 104f.
E180X_release_trigger_point()....124
E180X_SCANNERMODE_SL....112
E180X_SCANNERMODE_XY2_100....111
E180X_SCANNERMODE_XY2_100E....112
E180X_SCANNERMODE_XY2_200....111
E180X_SCANNERMODE_XY2_200E....112
E180X_set_connection....102
E180X_set_connection()....101, 133f.
E180X_set_debug_logfile....103
E180X_set_filepath()....133
E180X_set_fpk....112, 119
E180X_set_laser....113
E180X_set_laser_delays....110
E180X_set_laser_mode....112
E180X_set_laser_mode()....112
E180X_set_laser_timing....119
E180X_set_laserb....113, 119
E180X_set_matrix....117
E180X_set_matrix()....109
E180X_set_password....102
E180X_set_pixelmode....116
E180X_set_pos....116
E180X_set_pos()....116
E180X_set_scanner_delays....111
E180X_set_scanner_delays()....111
E180X_set_scanner_mode()....111
E180X_set_speeds....110

E180X_set_standby....113, 119
E180X_set_standby()....119
E180X_set_sync()....103
E180X_set_trigger_point....101, 104
E180X_set_trigger_point()....103, 124
E180X_set_wobble....114
E180X_set_xy_correction....109
E180X_set_xy_correction()....109, 118
E180X_set_z_correction....109
E180X_set_z_correction()....109
E180X_stop_execution....104
E180X_stop_execution()....8, 104
E180X_switch_correction....109
E180X_switch_correction()....109
E180X_uart_read....126
E180X_uart_write....126
E180X_write....132
E180X_write()....132
electrostatic sensitive device....10
encoder....38
EPR....41, 81
Error....19, 75
errout....23
ESD....10
eth....29
Ethernet....13ff., 21, 29, 76, 102
extension connector....40
ExtStart....35, 47f., 79ff., 85, 104, 125
ExtStop....35, 47, 80, 85, 104

F

fiber....34
fiber laser....35, 143f.
fiber-laser....113
file extension....41
filename....41
firmware....20, 31, 107
font....41
fonts....46
FPK....70, 112

G

G-Code....79ff.
G0....96, 98
G1....96, 98
G2....96
G3....96
G4....96
G70....98
G71....98
galvos....32, 50
gateway....28, 84
GNDext....36, 39
GRBL....99
gw0....28
GZTECH S355 series DPSS....155
GZTech YFPN....60, 143

H

HALdrive....63, 65, 68, 71, 73f.
HALnode Compact Ethernet IO....29

haltedloop....22f., 43, 45, 78
haltedlooptimeout....22, 43
homing....53, 56, 131

I

idxselect....23, 44, 78, 81f.
in-polygon delay....111
Intelli-IO Extension Board....127
Intelli-IO Extension Board v1....51
Intelli-IO Extension Board v2....54
iobuff....23, 44, 82
iohaltedloop....22f., 43ff., 78, 81
iolatch....22, 44
ioselect....23, 41, 43f., 78, 81f.
iothres....22
IP....14f., 21, 102
ip0....21
IPG....34, 60, 143
IPG YLM....60, 64, 148
IPG YLP....60, 144ff.
IPG YLR....147

J

JPT....60
JPT YDFLP....60, 144
jump....114
jump delay....111, 114
jump speed....114
jumpspeed....110

L

laser off delay....111, 114
laser on delay....111, 115
Laser Signals....14, 34
LaserA....35, 112
LaserB....35, 112
LaserGate....20, 24, 35, 79, 132, 154
lasers....34
Latch....35, 132
LED....19
libslrtc4.so....138
Linux....16
loop....42, 78
LP8....27, 35

M

M2....79f., 97
M3....97
M5....97
M704....96
M705....98
M707....98
M709....97f.
M715....98
M718....98
machine network....14
Main Oscillator....20, 27, 35
mapz....31
mark....115
mark delay....111, 115
mark speed....115

Marking Active....20
marking data....86
marking in progress....125
Marking On-The-Fly....37f.
markspeed....110
matrix laser....46
matrix marking....46
matrix printing....46
MaxPhotonics....60, 149
MaxPhotonics MFP....60, 149
MFP....60
microSD....14, 20, 41, 75
microUSB....17
mipout....23
MO....27, 35, 132
MOPA....60, 144
motion....52, 56
motout....24

N

NCC....72
netmask....29, 84
nm0....29
node0....29
node1....29
NX-02....57, 59
NX-02 Extension Board....57

O

open collector....37
opto-insulated....36
opto-mode 0....154

P

passwd....22
pethd....29
pixel line....116
polydelay....111
position encoder....37
power driver....53, 56
Power supply....14, 17, 32ff.
PRO license....13, 17
programming interface....101
PWM....70, 112

Q

Q-Switch....112
QR....42, 82f.
Quick Start....75

R

Raycus....60, 151
Raycus RFL-C Series 24V Control CW....154
Raycus RFL-PMX....60
Raycus RFL-PQB....60
Raycus RFL-QCW....60, 64
reboot....85
Reference....52f., 56
referencing....131
RFL-PMX....60
RFL-PQB....60
RJ45....14

RL3-100 159
rotation 118
RS232 ..... 24f., 40, 49f., 64
RS485 ..... 24f., 40, 49f.
RTC4 ..... 137f.
RTC4DLL.dll 138
RX0 40
RX0- 40
RX0+ 40
RX1 49
RX1- 49
RX1+ 49

S

sc_optic.dll 141
scaling 118
scanhead 18, 32ff., 50, 118
scanner movement 110
Scanner Signals 14, 32
scanner speed 110
SCI 141
SDP Extension Board 59
serial interface 14, 24f., 40, 102
serial number 83
sha256 20
sha256.asc 20
signatures.bin 20
SL Extension Board 112
SL2-100 158
SNTP 84
SNTP time server 28
sntp0 28
sntp0offset 28
SPI 34, 66, 72, 150
SPI G4 66
stand-alone 41, 46, 78
standalone 22, 42, 46
STATUS 32f., 118
step/direction 53, 56
stepper 52, 56
stepper motor 53, 56
SYNC ..... 32f., 49f.

T

T1 ..... 97, 99
Telnet 76
time 83
truePulse nano 72
TRUMPF 66, 72
TruPulse nano 66
tune 26, 48
tunemarkout 25, 45
tunereadyout 25, 44
tunexy3 28
turn on laser 81
TX0 40
TX0- 40
TX0+ 40
TX1 49
TX1- 49

TX1+....49
Type B....143
Type D....144
Type E....145
Type G....146

U

u0bits....24
u0brate....24
u0bypass....25, 76, 86
u0parity....24
u0stop....24
u1bits....25
u1brate....25
u1parity....25
UART0....40
UART1....49f.
USB....13f., 17, 29, 75, 102
USC1....141
USC2....141
User LED....14

V

Vext....36, 39
Vext)....39

W

waiting for external trigger....125
waveform....35
wetout....23
Windows....15f.

X

X....32f., 49f.
XY2-100....111, 118, 156
XY2-100E....112, 156
XY2-200....111, 156
XY2-200E....112
XY3-100....33, 157

Y

Y....32f., 49f.
YAG....34, 69, 112
YDFLP....60
YLM....64
YLP....60

Z

Z....32f.

Extension Board....127

.

.bco....108
.bco,....21
.crt....21, 108
.ct5....21, 108
.ctb....21, 108
.fcr....108
.gcd....21, 108
.txt....21, 108
.ucf....21, 108

.xml.....21, 108